NIMBUS PC

RM BASIC

RESEARCH MACHINES

RM BASIC

PN14351

RM Basic
PN 14351

Copyright © 1985, Research Machines Ltd.

All rights reserved. Although customers may make copies of this manual for their own use, you may make no other form of copy of any part of it without our written permission.

MS-DOS is a registered trademark of Microsoft Corporation.

Because our policy is to improve our products and services continually, we may make changes without notice. We have tried to keep the information in this manual completely accurate, but we cannot be held responsible for the consequences of any errors or omissions.

Customers comments are of great value to us in improving our computer systems, publications and services. If you would like to make any comments, please use the reply-paid form at the back of the manual.

Author: Gerry Cooper.

Illustrations by Jane Hannah.

Cartoons by David Mostyn, Oxford.

Typeset by direct transfer from Research Machines Network to Linotron 202 at Oxford Publishing Services, Oxford.

Printed by The Hazell Press, Wembley.

Research Machines Limited, Mill Street, Oxford OX2 0BW
Tel. Oxford (0865) 249866

Preface

RM Basic is a programmable language that is ideal for beginners because it is easy to learn and easy to correct mistakes. But RM Basic is also a very useful language for experienced programmers because all the advanced facilities of Nimbus are easily available with the language.

RM Basic contains many features that allow advanced structured programming.

This manual is in three parts:

- a Concepts section
- a Keywords section
- appendices and index

The Concepts section consists of ten chapters that teach programming techniques, with many example programs. It is designed so that you can work through the chapters at your Nimbus, typing in examples and learning as you proceed.

The Keywords section is the reference part of the manual. It lists all the special RM Basic words in alphabetical order and it gives syntax and other details. There are examples showing how these words are used in programs.

In the third part of the manual you will find appendix A that explains error handling in RM Basic and lists all the error messages. Appendix B explains the mysteries of binary and hexadecimal number systems, and shows you how logical operations work. Last, but not least, is the index.

Contents
RM Basic Concepts

Chapter 1: Talking to RM Basic

Running RM Basic	1.2
— PRINT command	1.3
— Writing programs	1.4
— The INPUT command	1.6
Loop Structures	1.7
— The REPEAT/UNTIL loop structure	1.8
The READ and DATA commands	1.9
SET and ASK	1.9
The AUTO command	1.11
Saving your programs	1.12
Program interruption	1.13

Chapter 2: Changing Things

Correcting typing mistakes	2.1
The Editor	2.2
— Moving the cursor to the left	2.3
— Moving the cursor to the right	2.3
— Home and End	2.4
— Word by word	2.4
— Up and down	2.5
Summary of cursor movement keys	2.5
Deleting	2.6
— Getting out of delete mode	2.7
Modifying program lines	2.8
The EDIT command	2.9
— Changing your mind	2.10
Other useful commands	2.10

Chapter 3: Handling Words and Numbers

String constants 3.1
Numeric constants 3.2
— Integers 3.2
— Floating point numbers 3.3
— Scientific notation 3.4
— Integers or floating point numbers? 3.4
— Hexadecimal numbers 3.4
— How RM Basic writes numbers 3.5
Variables 3.5
— String and numeric variables 3.6
— Naming variables 3.7
Arrays 3.8
— Arrays with two subscripts 3.11
— Even more subscripts 3.11

Chapter 4: Expressing Yourself

Comparisons 4.2
String-joining (concatenation) 4.4
Logical operations 4.5
— Logical operators in expressions 4.7
Arithmetic operations 4.7
Functions 4.8
Order of execution 4.9

Chapter 5: Divide and Conquer

Procedures 5.1
— Defining a procedure 5.22
— Calling a procedure 5.2
— Parameters 5.3
— Using several parameters 5.5
— Passing arrays by reference 5.6
Functions 5.7
Leaving a procedure 5.9
Recursion 5.10
Global variables 5.11
Keeping a variable alive 5.13
Global arrays 5.14
Return and receive 5.15

Chapter 6: Quick on the Draw

Introduction	6.1
— The screen	6.1
— High resolution graphics mode	6.3
— Low resolution graphics mode	6.4
— Defining the graphics axes	6.6
Drawing slices	6.7
Changing the origin	6.8
Colour	6.9
— Selecting colours	6.11
— Graphics colour	6.12
— Dithered patterns	6.13
Other options	6.13
— Point styles	6.13
— Line styles	6.14
— Area filling styles	6.16
— Flood fills	6.18
Plotting characters	6.19
— Plotting options	6.19
Drawing over previous drawings	6.20
Repositioning and sealing the coordinates	6.22
— The AT option	6.23
— The SCALE option	6.24
Drawing areas	6.25
Changing the colour table	6.27
Using arrays	6.28

Chapter 7: A Cursory Glance

The text screen	7.1
— Moving the cursor	7.2
Writing areas	7.4
— Writing and drawing	7.7
Pen and paper colours	7.8
Text characters	7.10
— The GET cursor	7.11

Chapter 8: Mice and Joysticks

Introduction	8.1
Programming with a mouse	8.2
— Displaying the mouse cursor	8.2
— Using the buttons	8.4
— SET MOUSE secrets	8.6
Programming with a joystick	8.7
Other mouse and joystick keywords	8.8
— More adventurous programs	8.10

Chapter 9: Blowing Your Own Trumpet

Sound	9.1
— Setting up	9.1
— The NOTE command	9.1
— Glissandos	9.3
— Forte to piano	9.4
Sound Plumbing	9.5
— Tempo	9.6
Other sound options	9.7
— Three voices	9.7
— Envelopes	9.9
— Non-musical sounds	9.11
— Quarter semitones	9.12
— The NOISE command	9.13

Chapter 10: The Outside World

Channels	10.1
— Using a printer	10.2
Files	10.3
— Storing information	10.3
— Reading from a stored file	10.5
— Modifying files	10.6
Several channels	10.7
File-handling commands	10.8
Communicating with other external devices	10.9

RM Basic Keywords

Introduction	K.1
Syntax	K.2
— Graphics syntax	K.4
Remarks	K.4
Examples	K.4
Options	K.5
Associated Keywords	K.5
Summary of rules	K.5
— Numbers	K.5
— Strings	K.5
— Variable names	K.6
— Array names	K.6
— Assignment rules	K.6
Definitions	K.7

Appendix A: Error Messages and Warnings

Introduction	A.1
Error messages	A.2
— Commands	A.3
— Syntax	A.5
— Size or range problems	A.6
— Graphics	A.7
— Files and channels	A.7
— Warnings	A.8

Appendix B: Logical Matters

The binary system	B.1
The hexadecimal system	B.2
Negative numbers	B.3
True and false	B.4
Bit-by-bit operations	B.5
XOR drawing and writing	B.9

Appendix C: Configuration Files

Chapter 1
Talking to RM Basic

As with spoken languages, computer languages have a vocabulary that should be learnt and syntax rules that should be observed. Basic is a popular computer language. It is the first that most people learn, because it contains a lot of English words — such as PRINT, LIST, SAVE — and because the syntax rules are not too difficult to understand.

This chapter is not intended as a guide to these rules, but as an introduction to the things that you can do with RM Basic. Just type in the examples and see what happens. Details of the language are given in later chapters and, in reference form, in the Keywords section.

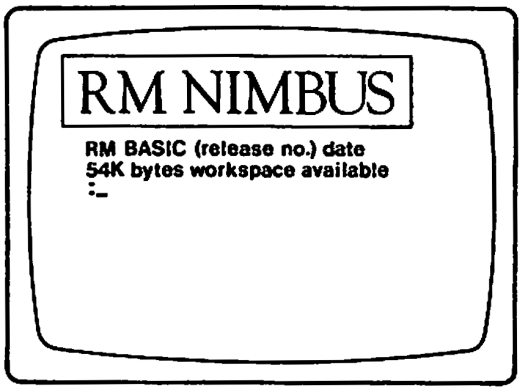
Running RM Basic

Switch on your Nimbus and follow the instructions in "Running Programs" in the Nimbus Owners Handbook to load the operating system and to insert your disk containing RM Basic. Now type:

```
BASIC <ENTER>
```

where <ENTER> means press the Enter key on the keyboard. If you are not sure about the keys, consult "Introducing Nimbus" in the Owners Handbook.

After an introductory message, you will see the : prompt and a thin, rectangular, flashing cursor on the screen.



The : prompt tells you that RM Basic is waiting for you to type something. The cursor is at the position that your entry will be displayed.

To show you what can be done, here are some examples that you can type in. You can use upper case or lower case letters and, if you make any typing errors, you can use the Backspace key to delete the mistake.

Make sure that you press the Enter key at the end of each line — this tells the computer you have finished the line.

```
PRINT "Welcome" <ENTER>

SET PEN 2 <ENTER>

LINE 0, 0; 100, 100 <ENTER>

SET MODE 40 <ENTER>

PRINT 10, 20, 30 <ENTER>

SET BORDER 2 <ENTER>

SET MODE 80 <ENTER>
```

Each line that you typed is called an *instruction*, consisting of either an RM Basic command and information for that command to use, or an RM Basic command on its own.

You entered the instructions in *direct mode*. This means that they were obeyed, or executed, as soon as <ENTER> was pressed. After execution of each instruction, the **:** prompt and cursor reappear to show that RM Basic is ready for more input.

The PRINT Command

You can use the PRINT command to display things on the screen. For example, if you type:

```
PRINT 43 <ENTER>
```

the number 43 is printed on the next line.

You can use this command to do arithmetic. For example, type:

```
PRINT 43 + 16 -7.5 <ENTER>
```

The computer calculates the result and displays it.

PRINT can also be used to display words. You have to place the words in quote marks. For example:

```
PRINT "Good morning" <ENTER>
```

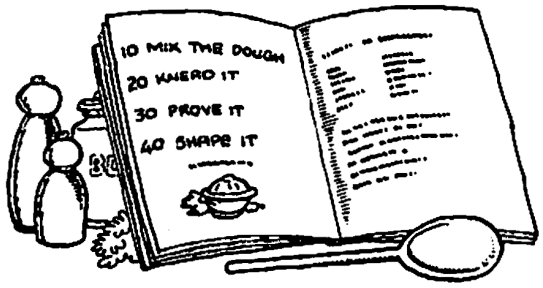
When you type this, the greeting will appear on the next line without the quote marks. When entering words in quotes, the characters are displayed exactly as you entered them.

Writing Programs

The instructions that have been used so far have been entered in direct mode and executed as soon as the Enter key was pressed.

After execution, the instruction is forgotten by the computer and, if you want to carry it out again, you have to re-enter it.

When you want to repeat actions, or carry out a sequence of actions, the instructions are entered in a program. You do this by putting a number — called a *line number* — at the start of the instruction. The line number informs the computer that the instruction is part of a program.



If you now type:

```
10 PRINT 43 + 16 -7.5 <ENTER>
```

you will see that the computer has behaved differently from before.

You only see that the cursor has moved to the start of the next line awaiting further instructions, but the computer has stored the instruction in memory ready to be executed on request.

Clear the screen of all the previous clutter with:

```
CLS <ENTER>
```

Now, if you type:

```
RUN <ENTER>
```

you will see that the action of your instruction has been performed. You have just run a program!

Each time you type **RUN** and press the Enter key, the action will be repeated.

If you type another useful direct mode instruction:

```
LIST <ENTER>
```

the program line will be displayed on the screen.

By now, you should have noticed the importance of the Enter key. It causes execution of direct mode instructions and it stores program lines in memory.

In all future examples, there is no reminder to press the Enter key.

The INPUT Command

PRINT allows you to display things on the screen. There is also a command called INPUT that allows you to enter information at the keyboard.

To see how this works, type in the following program of 6 lines (not forgetting to press the Enter key at the end of each line). Before you start, type:

```
NEW
```

to clear the memory of your first program.

Notice that the line numbers go up in tens. This is the usual practice; it allows additional lines to be put in later.

```
10 CLS : HOME
20 INPUT "How many miles have you travelled "; Miles
30 INPUT "How many hours did it take "; Hours
40 Speed := Miles/Hours
50 PRINT "Your speed was"; Speed; "miles per hour"
```

Now type **RUN** to run the program. The words:

```
How many miles have you travelled ?
```

will appear on the screen followed by the cursor. The question mark is produced because a semicolon was used after the quotation mark in line 20. If you do not want a question mark, use a comma.

You have to type in a response and press the Enter key. When you have done this, a second question appears and you must respond again. Nimbus will calculate the result and display it according to line 50 of your program.

Run the program again with different responses. The INPUT command allows data entry while a program is running.

Loop Structures

There are all sorts of occasions in programming where you will want to repeat instructions more than once. To do this, you set up a loop structure.

The FOR/NEXT loop structure is one way of executing lines repeatedly. You can see how it works from the next example program that is similar to the last one. The program finds out how many pupils there are in a class, asks you to enter each pupil's mark, and calculates the average mark:

```
10 Total := 0
20 INPUT "How many pupils "; Number
30 FOR I = 1 TO Number
40   INPUT "Pupil's mark "; Mark
50   Total := Total + Mark
60 NEXT I
70 Average := Total/Number
80 PRINT "The average mark is "; Average
90 END
```

When you run this program, it will ask how many pupils there are. Your response determines how many times the next question is displayed. If you said that there are 10 pupils, the question:

```
Pupil's mark?
```

will appear ten times. This happens because of the FOR/NEXT loop. As you enter the marks, they are added (line 50) and the program prints out the average mark when all the data has been collected.

After you have run this program, use LIST to display the program lines on the screen. The instructions of lines 40 and 50 have been indented automatically to show where the loop construct is situated.

The REPEAT/UNTIL Loop Structure

The FOR/NEXT loop structure repeats instructions for a specified number of times. Often, you will not want to specify how many times, or you may not know how many repetitions are needed.

This is where the REPEAT/UNTIL loop structure comes in handy. Here, the repetitions continue until the condition following UNTIL is true.

Here is a simple program that keeps repeating until you enter 0 to stop it. Don't forget to use NEW to clear previous programs before you enter this one.

```
10 Total := 0
20 REPEAT : REM Loop structure for data entry.
30   INPUT "Enter a number. Enter 0 to stop: ", N
40   Total := Total + N : REM Add the numbers.
50 UNTIL N = 0
60 PRINT "Total is"; Total
70 END
```

Run the program a few times, entering as many or as few numbers as you like.

The REM command, used in lines 20 and 40, introduces a remark or comment that explains what the program is doing. The remark has no effect on program execution.

You will see more examples of loop structures later.

The READ and DATA Commands

The READ command is similar to the INPUT command except that, instead of waiting for you to enter something at the keyboard, READ looks through the program for a DATA instruction containing data. This data is read and used in the program.

Type in and run the following program after clearing the memory with NEW.

```
10 CLS : HOME
20 READ A, B, C, D
30 SET CURPOS A, B
40 PRINT "Here"
50 SET CURPOS C, D
60 PRINT "There"
70 DATA 5, 5, 70, 25
```

The data from line 70 is read in by the READ command on line 20 and used in lines 30 and 50 to set the position of the cursor.

You may have noticed that line 10 is the same as in a previous example program. There are two instructions in this line, separated by a colon. This saves having a line for each short instruction.

The CLS instruction clears the screen and the HOME instruction positions the prompt and cursor at the top left-hand corner of the screen.

SET and ASK

You have just seen how the cursor position can be selected with SET CURPOS. RM Basic has a lot of features or attributes that can be selected with the SET command.

Here are some of them:

`SET BORDER`
Select the screen border colour.

`SET PLOT SIZE`
Select the size of characters displayed with the graphics command, PLOT.

`SET CURSOR`
Select the characteristics of the displayed cursor.

`SET CURPOS`
Select the cursor position on the screen.

`SET DEG`
Select degrees as the unit for measuring angles in trigonometrical work.

`SET MODE`
Select the screen display mode (40 or 80 characters per row).

`SET PEN`
Select the colour of text characters on the screen.

`SET PAPER`
Select the background colour of the screen.

`SET TIME`
Set the MS-DOS operating system clock.

`SET DATE`
Set the MS-DOS operating system date.

Full details of these and all other attributes are given in the Keywords section.

There is also an ASK command that allows you to find out what the current setting of any of these attributes is.

The AUTO Command

Instead of having to type your program line numbers, you can use AUTO to generate them automatically. After clearing memory with NEW, type:

```
AUTO
```

Line number 10 will be displayed followed by the cursor, showing that input is expected:

```
10_
```

Enter an instruction such as:

```
SET PEN 1
```

As soon as you press <ENTER>, the next line number will appear on the next line:

```
10 SET PEN 1
20_
```

As you enter further instructions, line numbers will continue to be generated.

When you reach the end of your program, you will want to get out of automatic line generation. Do this by pressing <ENTER> instead of an instruction. For example:

```
70 PRINT "Finished"
80 END
90 <ENTER>
:_
```

Saving Your Programs

Programs remain in the computer's memory until you switch off or type NEW. Then they are lost forever.

In the same way that music can be permanently recorded on tape or record, you can save your program on disk. Then, when you want to run the program again, you can load it back into memory from disk.

To record a program, use the SAVE command. This saves the program lines on disk as a file with the name that you give it. For example, if you are saving the program that calculates the average mark of a class of pupils, you could name it Mark. To save it under that name, simply enter:

```
SAVE "Mark"
```

File names can be up to eight characters in length (see "Operating System Concepts" in the Nimbus Owners Handbook).

If you now type DIR, you should see a file in the list called MARK.BAS. The BAS is a file extension that has been added automatically because Mark is a Basic file.

The RM Basic command for loading this or any other Basic file from disk is LOAD:

```
LOAD "Mark"
```

Program Interruption

After a program has successfully run, the : prompt and cursor reappear. If there is an error in the program, RM Basic will display:

- the line it was executing when the error was found
- an arrow pointing to the place, in line execution, when the error was found
- an error message giving information about the type of error and the : prompt and cursor are displayed.

You will learn how to edit your programs in the next chapter, and details of error messages are given in appendix A.

If you want to interrupt a running program, you can press the BREAK keys — Ctrl and Scroll Lock at the same time. This action is indicated by <BREAK> in the manual.

Chapter 2
Changing Things

This chapter describes editing techniques:

- correcting mistakes during entry of a line
- modifying program lines.

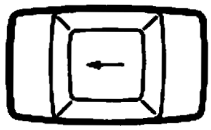
Correcting Typing Mistakes

When you are using direct mode, or typing in a line of program, or entering data at the request of an INPUT command, you can correct your mistakes before pressing <ENTER>. If you spot an error as soon as you have made it, you can delete the wrong character by pressing <BACKSPACE>.

For example, you may have typed Y instead of T:

```
10 PRINY_
```

You can remove the Y by pressing <BACKSPACE>:



```
10 PRIN_
```

Now, you can type T and enter the rest of the line.

Each time you press <BACKSPACE>, the character to the left of the cursor is removed and the cursor moves left one position. Any character, including a space, can be deleted.

Suppose you had gone a little further with the above line before noticing the mistake:

```
10 PRINY "Hello_
```

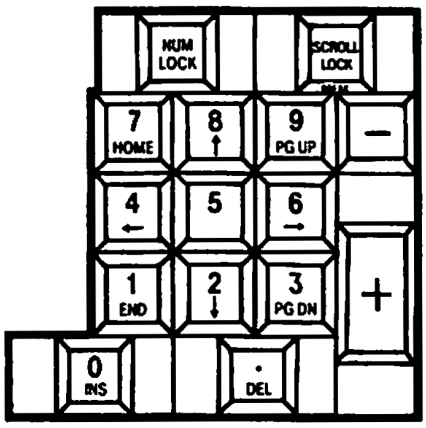
By pressing <BACKSPACE> eight times, you will delete eight characters:

```
10 PRIN
```

It would be much better if you could move the cursor over characters without deleting them. You can do this with the RM Basic editor.

The Editor

The numeric keypad on the right of the keyboard is used for editing. This section, and later ones, explain what you can do with each key.



Moving the Cursor to the Left

Returning to the above example:

```
10 PRINY "Hello_
```

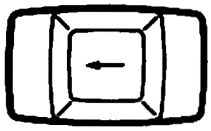
When you press <LEFT> the cursor moves one position to the left:

```
10 PRINY "Hell_o
```

If this does not happen and a 4 appears on the screen, delete the 4 and press <NUM LOCK> to change the keypad from numeric to editing mode. Then try again. Use <LEFT> to move the cursor to the space after PRINY:

```
10 PRINY_"Hello
```

Now, you can press <BACKSPACE> to delete the Y character:



```
10 PRIN_"Hello
```

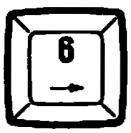
Notice that all characters on the right of the cursor have moved one position to the left.

When you type T, it will be inserted in the right place. All the characters on the right of the cursor move one position right:

```
10 PRINT_"Hello
```

Moving the Cursor to the Right

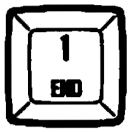
When you press <RIGHT>, the cursor moves one position to the right. Move the cursor back to the end of the line with this key.



As with all keys, you can speed up the action by holding down the key for repetition — assuming that auto repeat is set (see the KEYREP definition in the Keywords section).

Home and End

There is a more efficient method of moving to the end of the line — just press <END>.



This moves the cursor to the end of the text:

```
10 PRINT "Hello_
```

Now you can type in the rest of the line.

Often, there will be no need to go back to the end of the line after correcting a mistake. You can enter a line of text at any time by pressing <ENTER>. The whole of the text will be entered, wherever the cursor is.

To move the cursor to the beginning of text, press <HOME>:

```
10 PRINT "Hello
```

Word by Word

You can make the cursor jump over words. Pressing <PG UP> moves the cursor one word to the left:

```
50 INPUT "Enter a_number"
```

and <PG DN> moves the cursor one word to the right:

```
50 INPUT "Enter a number"_
```

Up and Down

Sometimes, you may be editing a long program line that takes up more than one screen line. For example:

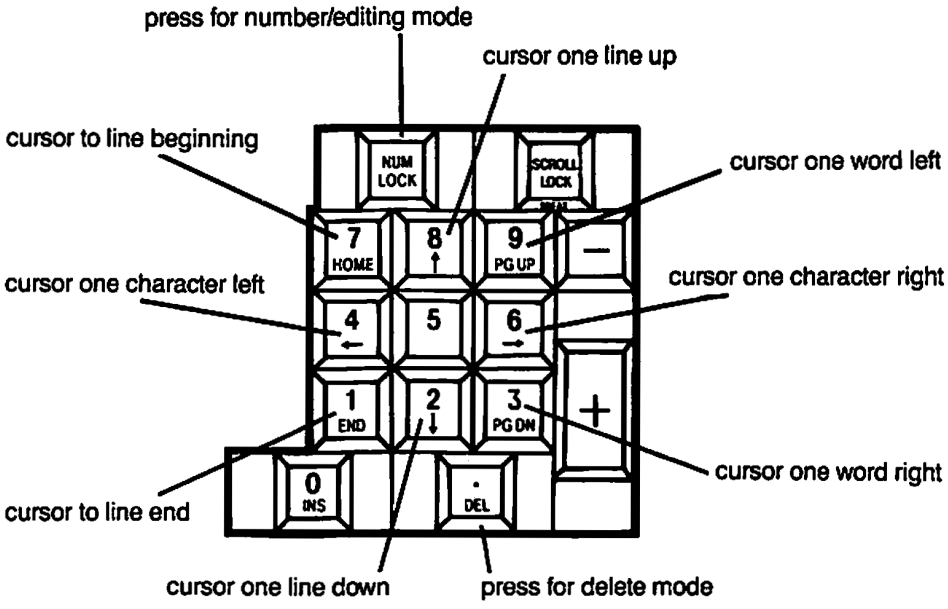
```
250 DATA January, February, March, April, May, June,
July, August_
```

If you press <UP>, the cursor jumps vertically up one line. In the above example, the cursor moves up to the u of January.

Similarly, by pressing <DOWN> you move the cursor vertically down one line.

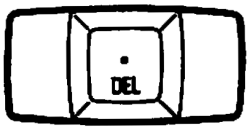
Summary of Cursor Movement Keys

Here is a summary of what each key does. The Del key puts you in delete mode — explained in the next section.



Deleting

The keys that you use to move the cursor can also be used to delete text. You convert the keys to delete mode by pressing <DEL> on the numeric keypad.



When you are in delete mode, the cursor changes to a flashing block. For example, you may have entered:

```
60 PRINT "How many books in the list_
```

when you decide to change the message. Press the Delete key, and the cursor will change:

```
60 PRINT "How many books in the list■
```

In delete mode, each key moves the cursor to the same place as before but the characters it passes over are deleted. So, if you press <PG UP>, a word to the left will be deleted:

```
60 PRINT "How many books in the■
```

The word left key is now the delete word left key. The action of all keys is similarly changed.

The <UP> and <DOWN> keys need some explanation. The <UP> key causes deletion of a line of characters preceding the cursor position.

For example, in the line containing months:

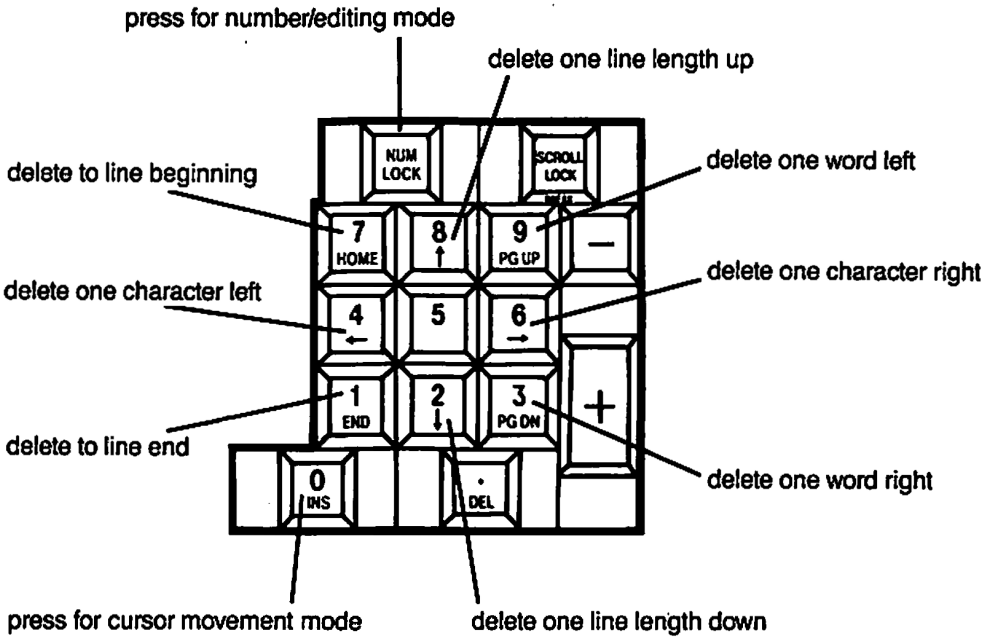
```
250 DATA January, February, March, April, May, June,
July, August■
```

After pressing <UP>, you will have:

```
250 DATA Jan■
```

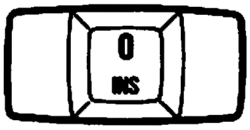
The <DOWN> key deletes a line in the opposite direction.

Here is a summary of key actions in delete mode:



Getting Out of Delete Mode

To get out of delete mode, either press <INS> on the numeric keypad or press any key on the main keyboard (except <BACKSPACE>).



Warning: Leave delete mode before using the numeric keypad keys to move the cursor. Otherwise you will continue deleting.

Modifying Program Lines

The previous section showed you how to correct mistakes before you press <ENTER>. This section describes how you can modify program lines that are in the computer's memory.

A program may have been typed in by you, or you may have loaded it from disk.

Use LIST to display all the lines of the program that you want to modify. For example:

```
10 REM calculation of the average number of letters
20 Total := 0
30 FOR R = 1 TO 5
40    INPUT "Enter a word: ", Word$
50    Total := Total + LEN(Word$)
60 NEXT 70 Average := Total / 5
80 PRINT "The average number of letters per word is";
Average
```

You can change any line simply by typing in a new line with the same line number. For instance, if you enter:

```
30 FOR R = 1 TO 10
```

and list the program again, you will see that ths line has replaced the old one.

To delete a line, type in the line number on its own:

```
30 <ENTER>
```

List the program once more. Line 30 has disappeared.

The EDIT Command

To change an existing line, use the EDIT command with the number of the line to be modified. For example, if you want to change line 70 in the previous program, type:

```
EDIT 70
```

The line will be displayed on the screen with the cursor at the start of the line:

```
_  70 Average := Total / 5
```

Now you can use the cursor control keys to move the cursor and to change 5 to 10. After editing, press <ENTER>.

The edited line will replace the old one, as long as you have not changed the line number. If you do change the line number during editing, the old line survives unchanged. The edited line is placed in the program in the position indicated by the new line number.

For example, you can call line 70 with the EDIT command and change the line number to 170. If you list the program, you will see that line 70 is still there, and you have added line 170, or replaced an old line 170 (if there was one).

Changing the line number is useful when you want to produce a new line that is either the same as or similar to a line that already exists. Rather than typing the new line, just use EDIT to create it from the old one.

Be careful. If you change the line number to one that is already in your program, that line will be replaced.

If you enter EDIT without specifying a line number, the line containing the most recent error (if there is one) is displayed.

Changing Your Mind

Sometimes, when you have called up a line with the EDIT command and made some changes, you may decide that you prefer the line as it was. You can remove all the changes and restore the line to its called-up state by pressing the Esc key.

This key can also be used when you are entering a line for the first time. On pressing Esc, what you have entered is cleared.

If you want to abandon a line, either on entering or on editing, press <BREAK>:

- if you are editing, you leave the line and changes to it are forgotten
- if you are entering a line, you leave it and that line is ignored. An existing line (in memory) with the same number is not affected

Other Useful Commands

There are two other commands that are useful for modifying programs:

DELETE
For deleting lines.

RENUMBER
For renumbering lines.

Details can be found in the Keywords section.

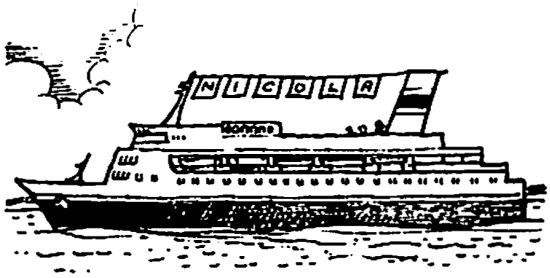
Chapter 3
Handling Words and Numbers

This chapter gives details of how words and numbers are handled in your programs.

The words and numbers that we are familiar with are treated as constants by RM Basic. Constants are classified into two types:

- strings (see the next section)
- numbers

String and numeric constants are stored in things called variables and arrays. These are explained later in this chapter.



String Constants

A string is a sequence of characters (up to 32000). Normally, you use strings to enter words or sentences into the computer. The characters of the string must be enclosed in double quotation marks:

```
"Nicola"
"Do you want to try again? Enter Y or N:"
"24 books"
```

When writing string constants to screen, printer, or file, RM Basic removes the enclosing quotation marks. So, if you enter:

```
PRINT "21 North Street, Southtown"
```

you will see on your screen:

```
21 North Street, Southtown
```

Numeric Constants

A numeric constant is a positive or negative number. If you do not supply a sign, the number is assumed to be positive.

There are three types of numeric constants:

- integers
- floating point numbers
- hexadecimal numbers

Integers

An integer is a whole number without a decimal point.

Integers can be in the range –32760 to +32760.
For example:

```
913+6-27858
```

Floating Point Numbers

Floating point numbers are made up of an integer part, and a fractional part with a decimal point. Either part can be omitted when you enter the number:

```
332.13
.8217
-73
```

Floating point numbers can be in the range 1.2×10^{-38} to 3.4×10^{38}. They are stored to a precision of six significant figures:

```
1.53440     3471.65     17.2168
```

When writing such numbers, RM Basic does not show a trailing zero in the fractional part:

```
1.5344
```

Numbers such as .8127 are written as:

```
0.8127
```

If a number such as 56 is assigned to a floating point variable, it is stored as 56.0000 but written as:

```
56
```

However, a number outside the integer range, such as 765432, is written as:

```
765432.0
```

Scientific Notation

Floating point numbers either larger than 9999999 or smaller than 0.1 are displayed in scientific notation — a floating point number (the *mantissa*) followed by the letter e and a power of 10 (the *exponent*). Ten million is represented by:

```
1e+07
```

Integers or Floating Point Numbers?

Floating point numbers take up more room in memory and the computer takes longer to do arithmetic with them.

Normally, you only use floating point numbers where you want the fractional values that are not given with integers, or numbers larger than can be stored as integers.

Hexadecimal Numbers

Hexadecimal numbers are preceded by an ampersand character (**&**). They consist of a sequence of digits — 0 to 9 and A to F:

```
&F6     &32
```

Appendix B gives details of hexadecimal numbers.

How RM Basic Writes Numbers

When you ask RM Basic to write numbers (to the screen, to a printer, or to a file), a positive number always has a space in front of it and a negative number has a minus sign instead of a space:

```
PRINT 45 : PRINT -45

 45
-45
```

Any kind of number always has a space following it:

```
PRINT 20.6; 43

 20.6  43
```

There are two spaces between these numbers because one is the space following 20.6, and the other is the space preceding 43. These rules do not apply when numbers are in strings:

```
PRINT "20.6 43"

20.6 43
```

Variables

Variables are names used to represent values used in a Basic program. The term 'variable' is used because you can change the value without changing the name. Only one value can be stored in a variable at any time.

You can name a variable and assign a value to it in two forms of an instruction:

```
[LET] Surname$ := "Jones"
[LET] Surname$ = "Jones"
```

Both these instructions create a variable called Surname$ and stores the value, Jones, in it. Surname$ will keep this value until you change it.

The LET keyword is often used for assignment. In RM Basic, it is optional.

You can use either := or = for assignment, but the use of the equals sign is very confusing in this situation, where it is not being used in the normal way. Consequently, := is used as the assignment operator throughout this manual.

The = character is used as a test for equality in instructions of this kind:

```
IF Month$ = "January" THEN PRINT "Expect snow" ELSE
PRINT "Don't expect snow"
```

This tests if Month$ equals January. If it does, we are told to expect snow. Otherwise, no snow.

String and Numeric Variables

The $ on the end of Surname$ is a *type identifier*; it informs the computer that you are creating a string variable. This can store only string values.

When you want to create an integer variable, put a % character at the end of the name:

```
Age% := 12
```

A variable without any type identifier is a floating point numeric variable:

```
Age := 12 or Age := 12.5
```

You can assign integer values to floating point variables and floating point values to integer variables. The following rules apply:

- the number is stored in the form defined by the variable
- when a floating point number is assigned to an integer variable, the decimal digits are removed:

```
A% := 65.7 : B% := -6.7
PRINT A%; B%

 65 -6
```

- an integer number can be assigned to a floating point variable:

```
A := 32
PRINT A

 32
```

Naming Variables

Variable names must begin with a letter, then you can use up to 31 characters made up of letters, numbers, or underscore characters (_).

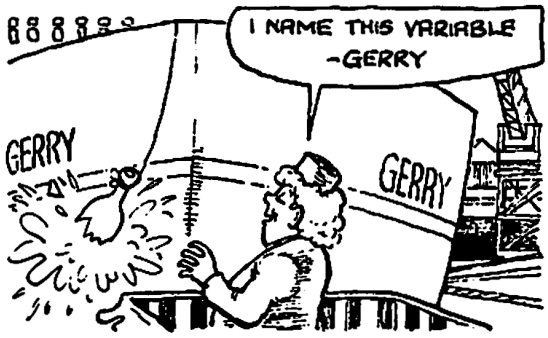
Your program will be easier to understand and read if you choose meaningful names. Here are some suggestions:

```
Day_Of_Month%

Name_Of_Pupil$

Answer
```

Throughout this manual, variable names are shown in the above form — lower case letters starting with a capital — to distinguish them from keywords.



A variable name may not be a keyword, but it can contain embedded keywords:

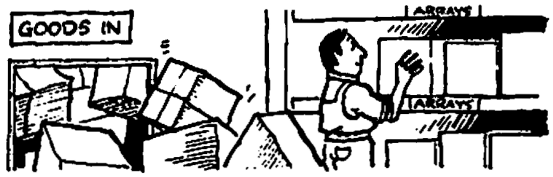
- List% is not allowed
- List_Of_Things_To_Do% is allowed

Arrays

A collection of several items of the same type can be stored under one name:

NAME	Item 1
	Item 2
	Item 3
	Item 4
	Item 5

This sort of construction is called an *array*. It is useful because large quanities of related information can be handled tidily.



Naming an array is subject to the same rules as naming a variable, including the use of type identifiers on the end of the name to signify the type of the elements of the array.

The DIM command is used to declare the name and type of an array, and the number of items in it. For example:

```
DIM Day$(6)
```

This creates a string array called Day$() with seven array elements, Day$(0), Day$(1), Day$(2), ..., Day$(6).

Each array element is a variable. Initially each element is empty, but a value can be assigned to it in the same way as in the previous section:

```
NEW
10 DIM Day$(6)
20 Day$(0) := "Sunday"
30 PRINT Day$(0); " is a day of rest"
```

The subscript need not be a numeric constant; you can have a variable or expression within the brackets.

For instance, you can assign values to array elements by putting a variable in the brackets in a loop structure:

```
DIM Day$(6)
10 FOR I% = 0 TO 6
20    READ Day$(I%)
30 NEXT I%
40 DATA "Sunday","Monday","Tuesday"
50 DATA "Wednesday","Thursday"
60 DATA "Friday","Saturday"
```

Each time through the loop, the next day of the week is read from the DATA instructions and assigned to the current array element — "Sunday" to Day$(0), "Monday" to Day$(1), and so on.

The following example shows the use of an integer array for storing the maximum daily temperature for every day of the year:

```
DIM Max_Daily_Temp%(365)
```

This creates an integer array with 366 array elements:

```
Max_Daily_Temp%(0)
Max_Daily_Temp%(1)
:
:
Max_Daily_Temp%(365)
```

Each day of the year is identified by the subscript number; so the 1st January reading is stored in Max_Daily_Temp%(0), and the 1st February reading uses Max_Daily_Temp%(31), and so on.

The final reading of the year, on 31st December, will be stored in Max_Daily_Temp%(364), and that leaves one unused array element (except in a leap year).

Arrays with two subscripts

To make it easier to see what day each array element represents, you can use an array with two subscripts:

```
DIM Max_Daily_Temp%(31,12)
```

This produces more than sufficient array elements. Each one has two subscripts; the first one refers to the day of the month, the second one refers to the month.

The array is big enough to hold 13 months (0 to 12) with 32 days (0 to 31) in each month. However, by not using the elements with 0 subscripts, the subscripts can be directly related to the calendar year.

Similarly, because some months are less than 31 days long, a few array elements, representing days at the end of the month, are never used.

The 1st January reading is stored in Max_Daily_Temp%(1,1), 1st February's in Max_Daily_Temp%(1,2), 31st December's is Max_Daily_Temp%(31,12), and so on.

Even more subscripts

This array can be enlarged further to cover temperature measurements over a number of years:

```
DIM Max_Daily_Temp%(31, 12, 10)
```

This creates array elements with 3 subscripts; the third subscript refers to the year. You can store data in this array, covering readings taken over a period of up to 11 years. An 11 year collection of data will consist of over 4000 measurements, taking up 8K of memory so you may exceed memory limits.

You can have more than three subscripts in an array, but you will need to take care.

Chapter 4
Expressing Yourself

This chapter gives details of expressions and how to use them to make RM Basic do what you want it to do.

Generally, where the computer is expecting a number or a string from an RM Basic instruction, you can provide an *expression* instead.

An expression is like a mathematical equation in which the computer is given information that needs to be evaluated:

```
PRINT X - Y

FOR I% = N% TO N% + 27
:
:
NEXT I%
```

You can assign an expression to a variable or array element. For example:

```
Age% := 15 + 30 / 2
```

Your computer will work out the value, then assign it to Age%.

In RM Basic an expression can be a number, or a string, or a variable. Also, it can be a collection of these separated by operators:

```
Sum% := 43 + 12 + Amount1 + Amount2%
```

You can do several types of operations in an expression:

- comparisons
- string-joining
- logical operations
- arithmetic operations
- functions

Comparisons

You can compare two expressions or values by placing a comparison operator between them:

```
IF A > B THEN PRINT "A is greater than B"
```

Here, the value of A is compared with the value of B.

If A is larger than B, the result is TRUE. Otherwise, the result is FALSE. The PRINT instruction will only be executed when the result is TRUE. Otherwise, the next line will be executed.

There are seven comparison operators:

Operator	*Type of Comparison*
=	Equality
<> or ><	Inequality
<	Less than
>	Greater than
<= or =<	Less than or equal to
>= or =>	Greater than or equal to
==	Interestingly equal to

The compared values must be both numbers or both strings.

In string comparisons, the one that would appear first in dictionary order has a value less than that of the other.

As strings can contain upper case and lower case letters, numbers, and punctuation characters, Nimbus dictionary order is decided by the codes of the characters of the string. Character codes are given in "Technical Data" in Reference section of the Nimbus Owners Handbook.

Lower case letters are greater than their upper case equivalents, and numbers contained in strings are less than letters. For example:

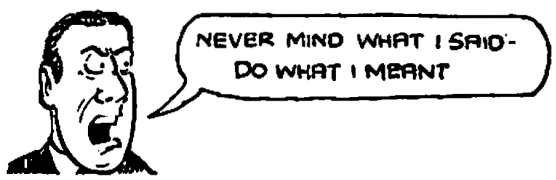
"Freda" is greater than "Fred"
"banana is greater than "BANANA"
"Class A" is greater than "Class 1"

The interestingly equal operator is very useful with strings. TRUE is given when two strings are spelt the same, whether upper case or lower case letters:

"banana" is interestingly equal to "BANANA"

You can use == to find out if two floating point numbers are the same. Sometimes, floating point numbers give inaccuracies in cumulative operations (where the printed number may not change while the binary value does).

The interestingly equal test compares the decimal output values — the values displayed by RM Basic — rather than the stored binary values. This avoids problems in testing with the equality operator where the binary values are compared.



Here is an example program that uses a number comparison (line 60) and a string comparison (line 110) to decide which sequence of instructions will be followed. It also uses the RND function in line 20 to generate random numbers (see the Keywords section for details).

```
NEW
 10 REM A simple addition test.
 20 A% := RND(50) : B% := RND(50)
 30 REM The question is repeated until you get it
 40 REM right or give up.
 50 REPEAT
 60    PRINT "What is"; A%; "plus"; B%;
 70    INPUT Answer%
 80    IF Answer% = A% + B% THEN PRINT "Correct" :
END
 90    REM Your answer is compared with the sum of
the numbers.
 80    REM if you are right, the program ends.
 90    REM Otherwise...
100    PRINT "Wrong. Do you want to try again (yes/no) ";
110    INPUT Reply$
120 UNTIL Reply$ == "no"
130 PRINT A%; "plus"; B%; "is"; A% + B%
140 END
```

String-joining (concatenation)

You can join two or more strings to produce a new string. The operator is the plus symbol (+).

Enter and run the following program:

```
NEW
10 A$ := "0x"
20 B$ := "ford"
30 C$ := A$ + B$
40 PRINT C$
```

In line 30, a single string, "Oxford", is created by joining A$ and B$.

Here are two more examples of string-joining:

```
PRINT "Hubble" + "-" + "bubble"
Hubble-bubble

PRINT "1234" + "5678"
12345678
```

Logical Operations

In the comparisons section, you saw how comparisons can be used to control the sequence of actions in IF ... THEN constructs and REPEAT/UNTIL structures. The logical operators, AND and OR, can be used to connect two or more comparisons for a single decision.

For instance:

```
IF Name$ = "Foster" AND Address$ = "Gloucester"
THEN Weather$ := "Rain"
```

When both the value of Name$ is "Foster" and the value of Address$ is "Gloucester", "Rain" is assigned to Weather$. If either of the comparisons is FALSE, the assignment does not take place.

Here is an example of the use of OR:

```
IF Forecast$ = "Rain" OR Forecast$ = "Showers"
THEN PRINT "Take an umbrella"
```

Only one comparison has to be TRUE for the advice to be given.

You can combine comparisons and logical operators in quite lengthy expressions. For example:

```
IF (A% = 10 AND B% = 20) OR (A$ = "Ten" AND B$ =
"Twenty") THEN X% := Y%
```

All comparisons are performed and, if one bracketed expression is TRUE, the instruction following THEN is executed.

Returning to the array, Max_Daily_Temp%(), the program segment below will assign keyboard-entered temperature readings to the array.

The inner REPEAT/UNTIL loop requests dates and checks that they are valid. The outer loop assigns data to the array and checks whether you want to enter more data.

```
100 DIM Max_Daily_Temp%(31, 12)
110 REPEAT : REM While Further data is available.
120   REPEAT : REM Until a valid date is entered.
130     INPUT "Which day "; Day%
140     INPUT "Which month "; Month%
150   UNTIL (Day% >= 1 AND Day% <= 31) AND (Month%
      => 1 AND Month% <= 12)
160   INPUT "Enter reading: ", Reading%
170   REM Maximum temp recorded on that day.
180   Max_Daily_Temp%(Day%, Month%) := Reading%
190   INPUT "Anymore (y/n) "; Answer$
200 UNTIL Answer$ == "N"
```

Logical Operators in Expressions

Because logical operators use bit-by-bit comparison — see appendix B — they can be used in expressions as well as in decision operations. For example:

```
A% := 5 AND 2
```

assigns 1 to A% because a bit-by-bit AND comparison of 5 and 2 produces 1.

Arithmetic Operations

Arithmetic operators are used in expressions to perform operations on numeric constants or variables. For example:

```
A := 6 * 13
```

The * operator specifies multiplication, so 78 is assigned to A.

The following arithmetic operators are available:

Operator	*Operation*	*Example*
↑	exponentiation	A ↑ B
+	addition	A+B
-	subtraction	A-B
*	multiplication	A*B
/	division	A/B
\	integer division	A\B
MOD	integer remainder	A MOD B
-	unary minus	-A
+	unary plus	+A

In integer division the operands are converted to integers (if within the range, –32760 to +32760), and division is performed. The quotient is converted to an integer. MOD gives the integer remainder of integer division.

The unary minus operator reverses the sign of a number. The unary plus operator leaves the number as it was.

When arithmetic operations are performed on expressions containing integer numbers and floating point numbers, the result is always a floating point number.

Functions

There are pre-defined functions that perform specific tasks such as calculating the sine of an angle (SIN) and calculating the number of characters in a string (LEN). Details of all pre-defined functions are given in the Keywords Section.

You can also define your own functions using the FUNCTION/ENDFUN structure (see chapter 5).

These functions are used in expressions in the same way as constants or variables are used. For example:

```
A := B + SQR(C)
```

Order of Execution

When there are several types of operators in an expression, the order in which operation is performed is decided by the following priority list:

SIN(x)	Functions
+	String concatenation
↑	Exponentiation
— +	Unary minus and plus
* / \ MOD	Multiplication and division
+ —	Addition and subtraction
< <= >= > = <> ==	Relational operators
NOT	Unary NOT (see appendix B)
AND	logical AND
OR	logical OR
XOR	logical XOR (see appendix B)

The top of the list is highest priority. In an expression with operators of equal priority, the operations are performed from left to right.

If you want to change the order of operation, use brackets. Operations within brackets are performed first.

For example, consider these two instructions:

```
A% := 10 + 3 * 6

A% := (10 + 3) * 6
```

In the first one, 28 is assigned to A% because 6 * 3 is performed first, giving 18, and adding 10 to that gives 28.

In the second one, 78 is assigned to A% because 10 + 3 is performed first, and 13 * 6 is 78.

If you want to use exponentiation on a negative number, you should place the number in brackets because exponentiation has a higher priority than unary minus:

```
PRINT -3↑2
```

gives –9, and:

```
PRINT (-3)↑2
```

gives 9.

Inside brackets, the usual order of operations is maintained.

Chapter 5
Divide and Conquer

Problems are more manageable when they are broken down into smaller parts that can be easily solved. This rule is particularly applicable to program writing.

Procedures

Before you start writing a program, it is useful to think about what you want to do. Try to split the job into progressively smaller units so that each one consists of a small number of instructions.

This helps too, if you want to modify the program. It is much easier to make changes to small units than to complicated programs.

As an example of a small unit, you may want to draw a greeting on the screen. Here is a program that does this using the graphics command, PLOT (see chapter 6).

```
NEW

250 CLS
260 SET MODE 80
270 FOR I% = 0 TO 3
280   PLOT "Hello", 250, 120 DIRECTION I% SIZE 2
290 NEXT I%
```

Defining a Procedure

To create a command that will execute these instructions, you can make them into a procedure. Do this by placing PROCEDURE at the start and ENDPROC at the end. Give the procedure a name; we have used Greeting:

```
250 PROCEDURE Greeting
260 CLS
270 SET MODE 80
280 FOR I% = 0 TO 3
290 PLOT "Hello!", 250, 120 DIRECTION I% SIZE 2
300 NEXT I%
310 ENDPROC
```

Calling a Procedure

Now, Greeting is a command that can be used in exactly the same way as you use any of the standard RM Basic commands. For instance, you can type:

```
Greeting <ENTER>
```

The instructions contained between PROCEDURE and ENDPROC will be executed, and the greeting will be drawn in four directions on the screen.

The procedure can be executed from within a program. For example:

```
10 INPUT "Do you want to say hello (Yes/No): ";
Answer$
20 IF Answer$ == "Yes" THEN Greeting
```

All but the smallest of programs will contain several procedures. It is best to place the procedure definitions at the end of the program after the controlling part that calls the procedures.

This is illustrated in the following program that uses some more graphics commands:

```
NEW
10 REM Draw a house using 3 procedures
20 CLS
30 SET MODE 40
40 Draw_Walls
50 Draw_Windows
60 Draw_Door
70 END
80 PROCEDURE Draw_Walls
90 AREA 0, 0; 130, 0; 130, 110; 0, 110 BRUSH 10
100 AREA 0, 110; 130, 111; 90, 160; 75, 160 BRUSH 9
110 AREA 75, 145; 90, 145; 90, 160; 75, 160 BRUSH 6
120 ENDPROC
130 PROCEDURE Draw_Windows
140 AREA 0, 0; 20, 0; 20, 20; 0, 20 AT 13, 20 BRUSH 14
150 AREA 0, 0; 20, 0; 20, 20; 0, 20 AT 97, 20 BRUSH 14
160 AREA 0, 0; 30, 0; 30, 30; 0, 30 AT 13, 70 BRUSH 14
170 AREA 0, 0; 30, 0; 30, 30; 0, 30 AT 87, 70 BRUSH 14
180 ENDPROC
190 PROCEDURE Draw_Door
200 AREA 55, 0; 75, 0; 75, 40; 55, 40 BRUSH 5
210 LINE 60, 15; 70, 15; 70, 20; 60, 20; 60, 15
BRUSH 0
220 ENDPROC
```

Lines 20 to 70 contain the instructions that execute the program. Lines 80 to 220 contain procedure definitions.

This program selects 40-character screen mode (line 30). To return to 80-character mode, type SET MODE 80.

Parameters

The inside of a procedure is a private area, and the variables and arrays are independent of those in the main program, even if the same name is used in each area.

So, the values of variables and arrays outside the procedure are not affected by what happens inside the procedure. You can imagine that there is a border around it.

Often, we control a procedure's action by giving it a value that has to cross the border. This is where parameters are used.

A parameter is a variable or array element that needs a value from outside. It is defined in the procedure definition.

For example, here is a procedure that calculates the factorial of the number supplied to it:

```
1000 PROCEDURE Factorial X
1010 Sumx := 1
1020 FOR Counter% = 1 TO X
1030   Sumx := Sumx * Counter%
1040 NEXT Counter%
1050 PRINT Sumx
1060 ENDPROC
```

When you call Factorial, it can accept a value that will be assigned to the variable, X. For instance:

```
Factorial 6 <ENTER>
```

prints 720.

Adding a parameter has increased the flexibility of the procedure. It can be called from several parts of the program, taking a different value each time.

Using Several Parameters

You can define procedures with more than one parameter. Just name the parameters in the procedure definition, separating them with commas.

The following procedure produces a format for a library reminder slip. You can change the book title and author, and the fine for the overdue period in each case.

You will see that exclamation marks are used in the PRINT instructions to produce blank lines.
PRINT ! gives one blank line and PRINT !!!
gives three blank lines.

```
NEW
100 PROCEDURE Overdue Title$, Author$, Fine
110 SET MODE 80
120 PRINT TAB(15); "OXFORD LIBRARY" !
130 PRINT TAB(16); "Overdue Book" !
140 PRINT TAB(35); DATE$ !!!
150 PRINT Title$ !
160 PRINT Author$ !
170 PRINT TAB(10); "This book is overdue. If you"
180 PRINT TAB(10); "return it by the end of the week,"
190 PRINT TAB(10); "the fine will be"; Fine; "pence."
200 PRINT !
210 PRINT TAB(15); "The Librarian"
220 ENDPROC
```

In your procedure call, be careful to use numeric expressions where you have defined numeric variables as parameters, and string expressions corresponding to string parameters. However, where you have defined integer or numeric variables as parameters, a numeric expression of either type will be accepted. These are the same assignment rules that were outlined in chapter 3.

Passing Arrays by Reference

You may want to use a set of values in a procedure from an array in the main program. This is easily done in RM Basic because the array reference system is used. If you place an array name followed by empty brackets as a parameter, the procedure can access any array that you name in the procedure call:

```
PROCEDURE Draw_House Info()
```

If you have an array, House1() for example, in the main program, the procedure is able to use all its contents:

```
Draw_House House1()
```

With this passing mechanism, the size of the array and the number of dimensions does not matter — as long as the procedure can cope.

The following procedure assigns 100 to all the elements of any two dimensional array passed to it. The BOUNDS function (see Keywords section) is used to find out the size of each dimension:

```
500 PROCEDURE Hundred Array_Name()
510 FOR Row% = 0 TO BOUNDS(Array_Name(), 1)
520   FOR Column% = 0 TO BOUNDS(Array_Name(), 2)
530     Array_Name(Row%, Column%) := 100
540   NEXT Column%
550 NEXT Row%
560 ENDPROC
```

When using the array reference system, you must assign an array of the same numeric type as that declared in the procedure definition. So, for example, you cannot pass an integer array to a procedure expecting a floating point array.

Functions

RM Basic has built-in functions, such as COS and LEFT$. If you want to define your own functions, you can do so in a similar way to defining your own procedures.

Defined functions and procedures have much in common, but there are some differences that are noted in this section.

Such functions help to break your program into manageable parts. If you have a frequently used calculation, you can write it as a function that can be used throughout your program.

To define a function, give it a name and parameters, and use the enclosing keywords, FUNCTION and ENDFUN. For example:

```
200 FUNCTION Number_Check(A)
210 IF A > 100 THEN RESULT 100
220 IF A < 50 THEN RESULT 0
230 RESULT A
240 ENDFUN
```

Number-Check is now a function with all the properties of a built-in function. For example, it can be called in an assignment instruction:

```
X := 30 + Number_Check(500)
```

The value of 130 is assigned to X because Number Check returns a value of 100.

As soon as a RESULT command is reached, a value is returned to the expression from which the function was called. The value returned is the value of the expression that follows RESULT.

Any number of RESULT commands can occur in a defined function. Once one RESULT is executed, the others are ignored.

When you call a defined function, always put brackets round the value(s) passed to it so that RM Basic knows exactly what you want the function to accept. When there are no parameters, call the function with empty brackets. Here is a function with no parameters:

```
500 FUNCTION Yes_Or_No$()
510 PRINT "Please enter Y or N"
520 REPEAT
530 Answer$ := GET$() : REM Wait For a key depression
540 UNTIL Answer$ == "Y" OR Answer$ == "N"
550 RESULT Answer$
560 ENDFUN
```

You would call this function whenever you want a yes or no answer to a displayed question.

The function ensures that a valid answer is given before returning that answer. For example:

```
IF Yes_Or_No$() == "Y" THEN Main_Program
```

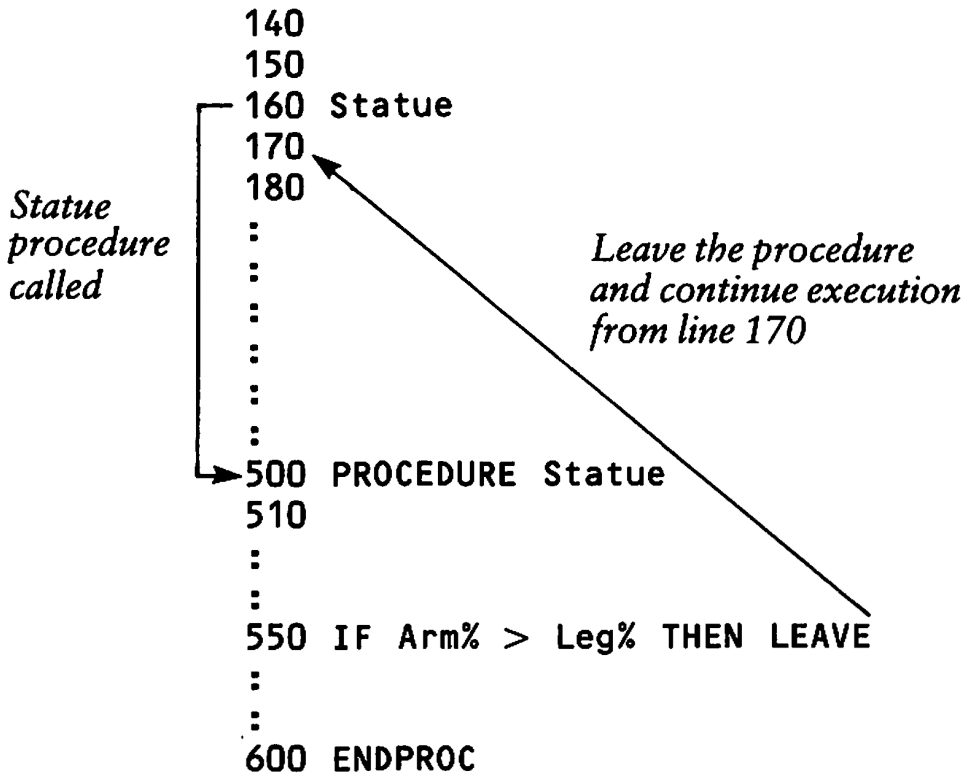
The Main_Program procedure is executed only when "Y" is returned by the Yes_Or_No$ function.

As with procedures, the array reference system can be used to allow functions to access arrays in the main program.

Leaving a Procedure

You have already seen several examples of procedures. In all of them, calling the procedure caused all instructions to be executed until ENDPROC was reached.

Often, you will want to leave a procedure without executing all its instructions. You can do this with the LEAVE command which does not return a value, but otherwise works in a procedure in the same way that RESULT works in a function.



Here is a procedure that uses the LEAVE command. The example also shows how a procedure can call another procedure or defined function. In this case, the procedure calls the Yes_Or_No$ function that you met in the previous section.

```
250 PROCEDURE Questions
260 INPUT "What is your age "; Age%
270 IF Age% <11 OR Age% >16 THEN LEAVE
280 PRINT "Do you watch ""Blue Peter"" ?"
290 IF Yes_Or_No$() == "N" THEN LEAVE
300 PRINT "Do you watch ""Coronation Street"" ?"
310 IF Yes_Or_No$() == "N" THEN LEAVE
320 PRINT "Do you watch ""The News"" ?"
330 IF Yes_Or_No$() == "Y" THEN PRINT "WHAT ABOUT YOUR
HOMEWORK ?"
340 ENDPROC
```

Recursion

A procedure or defined function can call itself. This is known as *recursion*. For example:

```
10 PROCEDURE Count_Up Number%
20 IF Number% >= 100 THEN PRINT : LEAVE
30 PRINT Number%;
40 Count_Up Number% + 1
50 ENDPROC
```

Try entering:

```
Count_Up 60
```

and observe what happens. In effect, the Count_Up procedure is called 41 times, each time with a different value passed to it. The procedure call in line 40, although a recursive call, is just like using any other command.

Global Variables

We have seen how information can be transferred across procedure borders with parameters. There is a more direct way that allows variables or arrays to be accessible both outside and inside the border.

Variables are normally local to one area of the program and, for example, on exit from a procedure the values of variables in the procedure are lost.

However, a variable can be created as a global variable with the GLOBAL command. This global variable can now be recognised in any part of the program when it is identified by a GLOBAL declaration.

Global variables and arrays do not lose their values when the program crosses a border, but their values are only known if they are identified.

Main Program	*Procedure*	*Defined Function*
GLOBAL Result%	GLOBAL Result%	
Create a global variable called Result%.	Acknowledge the existence of the global variable, Result%	No identification so the global variable is not known here.

Here is a program that gives the the sequence of lights for a traffic light. The main program reads the data into an array called Traffic$ then calls the Change_Lights procedure to produce the correct sequence. The global variable, Lights, is created in the main program and declared in the procedure.

```
100 GLOBAL Lights
110 DIM Traffic$(4)
120 FOR Sequence = 0 TO 4
130   READ Traffic$(Sequence)
140 NEXT Sequence
150 DATA Red, Red and Amber, Green, Amber
160 Lights := 0
170 FOR I% = 1 TO 5
180   PRINT Traffic$(Lights),
190   Change_Lights
200 NEXT I%
210 END
300 PROCEDURE Change_Lights
310 GLOBAL Lights
320 Lights := Lights + 1
330 IF Lights > 4 THEN Lights = 0
340 ENDPROC
```

If you run the program, the correct sequence will be displayed. However, if you remove line 100 and run the program again, you will get Red, Red, Red, Red, and Red.

This is because, with the removal of the GLOBAL instruction of line 10, the Lights variable in the main program is now local and it is not altered by calling the procedure.

The Lights global variable in the procedure changes its value each time the procedure is called, but the value of the Lights local variable does not change from 0 (assigned to it in line 160).

The following procedure calls another procedure (that you met at the start of this chapter) to work out the number of permutations and combinations of two entered numbers. Numbers of permutations and combinations are given by the mathematical expressions:

$$ {}^{n}P_{r} = \frac{n!}{(n-r)!} \qquad {}^{n}C_{r} = \frac{n!}{r!*(n-r)!} $$

The variable, Sumx, is used in both procedures. It is created as a global variable in Permcomb, and it is declared global in Factorial.

It is advisable to put your GLOBAL instructions in the first lines of your program, and/or in the first line of your procedure declaration, because if you try to declare a variable global when the variable has already been used, an error will occur.

```
300 PROCEDURE Permcomb N, 310 GLOBAL Sumx
320 Factorial N : REM Call Factorial procedure for N!
330 Nfact := Sumx
340 Factorial R : REM Call Factorial procedure for R!
350 Rfact := Sumx
360 Factorial N -R : REM Call Factorial procedure for (N-R)!
370 Nrfact := Sumx
380 Perm := Nfact/Nrfact
390 Comb := Nfact / Rfact * Nrfact
400 PRINT "Permutation is"; Perm
410 PRINT "Combination is"; Comb
420 ENDPROC
500 PROCEDURE Factorial X : REM Obtain factorial of X
510 GLOBAL Sumx : Sumx := 1
520 FOR Counter = 1 TO X
530   ON ERROR PRINT "Number too big" : END
540   Sumx := Sumx * Counter
550 NEXT Counter
560 ENDPROC
```

Keeping a Variable Alive

On exit from a procedure or defined function, the values of local variables are lost. However, the value of a global variable is not lost when the program leaves the procedure.

This can be useful when a procedure is called several times. You can change the value of a global variable each time that the procedure is called, and retain the changed value until the next call.

An example of this is given in a program at the end of chapter 8.

Global Arrays

If you are creating a global array, it must be dimensioned with the DIM command. Be careful that you do not try to re-dimension the array by re-calling a procedure:

```
500 PROCEDURE Negative
510 GLOBAL Results()
520 DIM Results(10)
530 FOR I% = 0 TO 10
540   Results(I%) := -1
550 NEXT I%
560 ENDPROC
```

The first time you call this procedure, Results() will be created and dimensioned as a one-dimensional array with elements 0 to 10. If you call this procedure again, an error will occur.

There are two ways of overcoming this problem:

- dimension your global array in an initialisation procedure that is only called once
- dimension your global array at the start of your program

Return and Receive

When you have gained experience in using procedures in your programs, you may find a few cases where the inability of a procedure to return a collection of values is a problem.

You can overcome this difficulty with the RETURN and RECEIVE facility. You place RECEIVE at the end of a procedure call and follow it with a series of variable names. This indicates the names of the variables into which any returned values will be assigned.

The values you wish to return are placed in the procedure definition. They follow the RETURN keyword and can be expressions of any form.

The first program below counts the number of vowels and consonants in a word. It uses the LEN, MID$, and INSTR string-handling functions. See the Keywords section for details of how they work.

In line 180 the Word$ parameter is passed to the procedure, Letter_Count. This instruction also specifies the variable names — Vowel% and Consonant% — that will receive the values returned by the procedure.

Line 500 contains the procedure name, the parameter name, and the variable names of the returned values.

The second program sorts strings in an array into alphabetical order. It uses a procedure that calls another procedure using RETURN and RECEIVE to swap values passed to it.

```
100 REM Program to print out the number of vowels and
110 REM consonants in a word.
120 PRINT "Word", "Vowels", "Consonants"
130 FOR Word_Num% = 1 TO 5
140   READ Word$
150   Letter_Count Word$ RECEIVE Vowel%, Consonant%
160   PRINT Word$, Vowel%, Consonant%
170 NEXT Word_Num%
180 DATA Train, Yacht, Bicycle, Aeroplane, Coach
190 END
500 PROCEDURE Letter_Count Phrase$ RETURN Nv%, Nc%
510 Nv% := 0 : Nc% := 0
520 FOR Count% = 1 TO LEN(Phrase$)
530   L$ := MID$(Phrase$, Count%, 1)
540   IF INSTR("AEIOUaeiou", L$) THEN Nv% = Nv% + 1
ELSE Nc% := Nc% + 1
550 NEXT Count%
560 ENDPROC
```

```
100 DIM Friends$(4)
110 FOR I% := 0 TO 4 : REM Read data into array.
120   READ Friends$(I%)
130 NEXT
140 DATA "John", "Anthony", "Tony", "Shaun", "David"
150 Sort Friends$(), 5 : REM Sort data.
160 FOR I% = 0 TO 4 : REM Print sorted data.
170   PRINT Friends$(I%)
180 NEXT
190 END
750 PROCEDURE Sort Name$(), N%
760 FOR J% = N% -1 TO 1 STEP -1
770   FOR K% = 1 TO J%
780     IF Name$(K%) > Name$(K% + 1) THEN Swap
Name$(K%), Name$(K% + 1) RECEIVE Name$(K%), Name$(K%
+ 1)
790   NEXT I%
800 NEXT J%
810 ENDPROC
900 PROCEDURE Swap A$, B$ RETURN B$, A$
910 ENDPROC
```

Chapter 6
Quick On The Draw

Introduction

This chapter gives details of the graphics features that are available in RM Basic on Nimbus. They are fast, colourful, and simple to use.

Graphics, in RM Basic, is a term covering the drawing of any pictures, graphs, shapes or diagrams on the screen — using a multitude of colours (or shades of grey on a monochrome monitor).

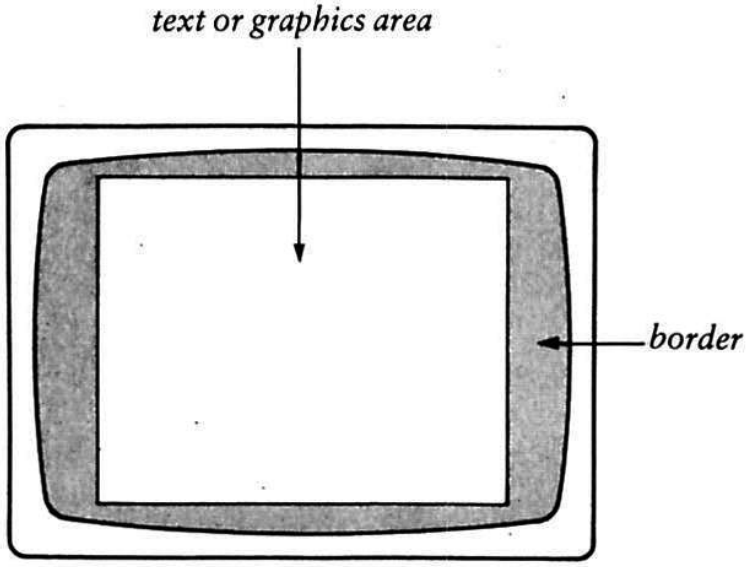
The Screen

So that you can understand how graphics works, here is some information about the screen of your monitor.

Obviously, if you have a monochrome monitor, it cannot generate colours. Instead, when you try out the examples in this chapter, you will see shades of grey.

The screen has two areas:

- a rectangular area in which text is written and graphics is drawn
- a border area around the rectangular area



When you enter RM Basic, you cannot see this border because it is the same colour as the central area. To show that there is a border, change the colour:

```
SET BORDER 2
```

and you will see a red border.

You can choose any one of sixteen colours for this area of the screen (see BORDER attribute in the Keywords section).

The writing and drawing area of the screen consists of lots of dots called *pixels* that can be lit up in different colours.

There are two modes of screen operation, each using a different number of pixels:

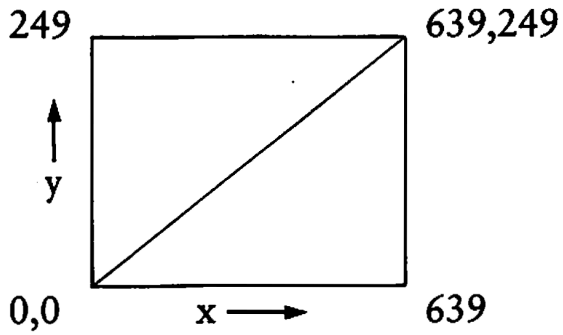
- low resolution graphics — where fewer but larger pixels are used, allowing all colours to be selected
- high resolution graphics — where more but smaller pixels are used and fewer colours are available

High Resolution Graphics Mode

On entering RM Basic, the screen is in high resolution graphics mode. This is also known as 80-character mode because each row of the screen can accommodate up to eighty characters of text.

When you are using graphics, the screen can be thought of as a piece of graph paper with x and y axes that have their origin at the bottom left corner.

The initial setting of the high resolution graphics axes are 0 to 639 in the x direction and 0 to 249 in the y direction.



In the graphics commands, you use x and y coordinates to indicate points on the screen. For example:

```
LINE 0, 0; 639, 249
```

draws a line from the origin (0, 0) to the top, right corner (639, 249). Each pair of x,y coordinates must be separated by a semicolon.

You can draw a line that goes through several points by specifying more coordinates:

```
LINE 20, 20; 100, 100; 50, 50; 0, 100
```

This draws one line from the point 20, 20 to the point 0, 100 passing through the points 100, 100 and 50, 50.

If you want to draw a rectangle, use the AREA command with coordinates for the four corners. For example:

```
AREA 50, 50; 100, 50; 100, 100; 50, 100
```

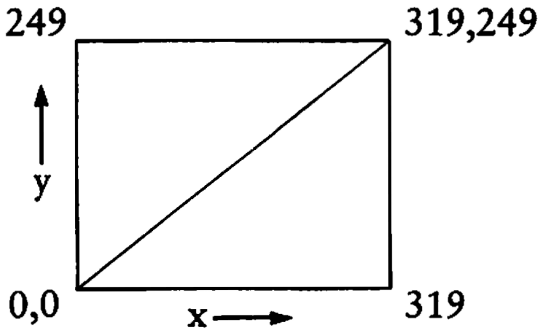
Low Resolution Graphics Mode

Low resolution graphics mode, or 40-character mode, is selected by:

```
SET MODE 40
```

Type in this instruction. The screen will be cleared and the background colour will change from blue to black. The prompt and cursor will be at the top, left corner.

The instruction has also changed the setting of the graphics x axis to the low resolution value — 0 to 319. The y axis is still 0 to 249.



Now, the instruction to draw a line from the origin to the top, right corner is:

```
LINE 0, 0; 319, 249
```

Type in this line; notice how the characters are wider in this mode. This is because the maximum is 40 characters per line, so each character takes up twice as much space as those of 80-character mode.

Here is a program that uses the AREA command to draw six triangles.

Each time round the loop, a solid triangle is drawn by joining the three points, specified by three pairs of x and y coordinates, and filling in the enclosed area:

```
NEW
10 SET MODE 40
20 FOR I% = 25 TO 275 STEP 50
30    AREA I%, 100; I% + 20, 100; I% + 10, 120
40 NEXT I%
```

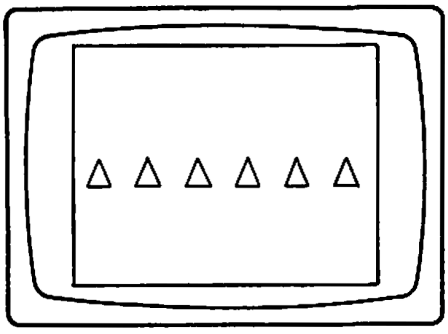
When you run this program, line 10 clears the screen and puts you in low resolution graphics mode (if you are not already in it).

Now use EDIT to change line 10 to:

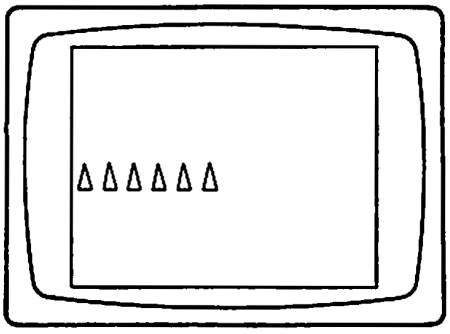
```
10 SET MODE 80
```

and run the program again.

Line 10 selects high resolution mode and changes the axes, so the shape of the triangles has changed.



low resolution mode



high resolution mode

The height has not changed because the units of the y axis are the same in both modes, but the width and spacing has changed because the units of the x axis are different.

Defining the Graphics Axes

You can define the units of the x and y axes in the SET MODE command. For example:

```
SET MODE 80, 149, 99
```

This selects high resolution mode and specifies that the units of the x axis are 0 to 149, and that the units of the y axis are 0 to 99.

Being able to change the axes is useful when you want to run a program in both modes without shape problems.

As in the following program, you can change the axes in one mode to match those in the other mode. Or, you can specify the same axes for both modes.

The program uses the CIRCLE command. This draws a circle of specified radius (using the y axis units) with its centre at the specified point given by the x, y coordinate pair.

The program calls a procedure to draw five circles in low resolution, then changes to high resolution with redefined axes, and calls the procedure again.

The command SET DEG TRUE is issued at the start of the program to work in degrees and not radians.

```
NEW
100 SET DEG TRUE : SET MODE 40
110 Draw_Circles : REM Draw low res. circles.
120 REM A loop to give time to see the effect.
130 FOR J% =1 TO 1000 : NEXT J%
140 REM Change mode and axes.
150 SET MODE 80, 319, 249
160 Draw_Circles : REM Draw high res. circles.
170 END
500 PROCEDURE Draw_Circles
510 FOR I% = 25 TO 275 STEP 50
520   CIRCLE 20, I%, 120
530 NEXT I%
540 ENDPROC
```

When you run this program, you will see that the circles are drawn in the same positions on the screen in both modes.

Drawing Slices

The SLICE command is similar to the CIRCLE command with the addition of two angles. SLICE is followed by:

- the radius (a in the following diagram)
- the angle from the y axis to start drawing (c)
- the angle from the y axis to stop drawing(d)
- the x,y coordinate of the centre of the circle (x, y)

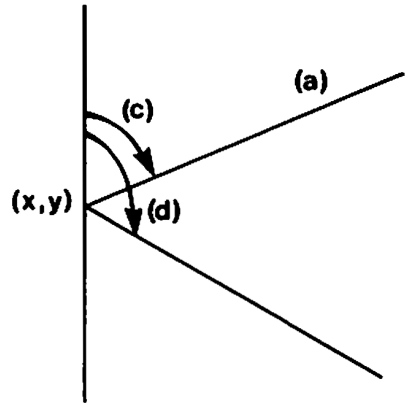
For example:

```
SLICE 50, 0 TO 90, 150, 100
```

You can draw several slices, or circles, of one radius by specifying more than one coordinate pair:

```
SLICE 20, 45 TO 135, 50, 50; 100, 100; 150, 150
```

Angle 0 is vertically up and angles increase as they turn clockwise:



Changing The Origin

Whenever you enter high or low resolution graphics, the origin of the x and y axes is automatically set to be at the bottom left corner of the screen.

You can change the origin to any position on the screen with the SET ORIGIN command. This does not change the scale of the axes. It simply moves the x and y axes with respect to the new origin position.

For example, enter and run this program:

```
10 SET MODE 40
20 LINE 5, 20 ; 205, 200
30 SET ORIGIN 150,100
40 LINE 5, 20 ; 205, 200
50 END
```

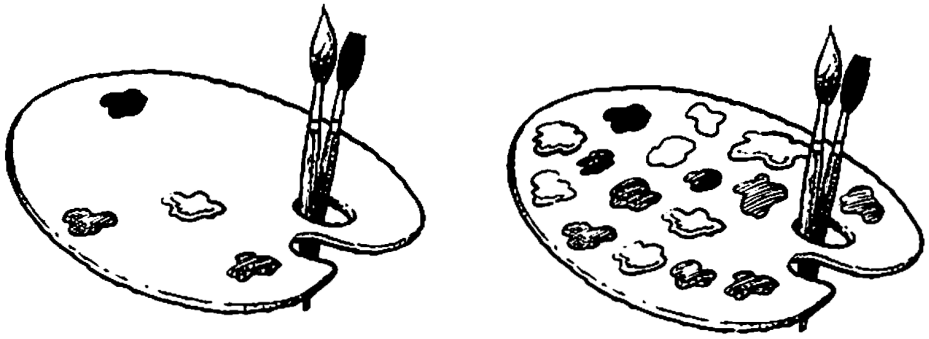
Though the coordinates in lines 20 and 40 are the same, those of line 20 have an origin at the bottom, left corner of the screen and those of line 40 have an origin at point 150, 100. So, two lines of the same length and direction are drawn.

The following program shows how changing the origin can help with graph plotting. It uses the POINTS command that draws points, and the PLOT command that draws characters. Details of these are given later in this chapter.

```
NEW
110 REM Draw a y versus x curve.
120 SET MODE 40, 50, 50
130 SET ORIGIN 25, 25
140 LINE -25, 0; 25, 0
150 LINE 0, -25; 0, 25
160 PLOT "x axis", 15, -2
170 PLOT "y axis", 1, -25
180 FOR X = -8 TO 8 STEP 0.5
190Y = X * X -3 * X -10
200POINTS X, Y
210 NEXT X
```

Colour

As stated earlier, when you are using high resolution graphics you can choose from a smaller range of colours (4) than when in low resolution graphics (16).



The following *colour tables* shows what colours are available in the two modes when you enter RM Basic, and their associated numbers.

High Resolution Mode Colour Table

Number	*Colour*	*Monochrome*
0	dark blue	grey level 1
1	dark green	grey level 4
2	light red	grey level 10
3	white	white

Low Resolution Mode Colour Table

Number	*Colour*	*Monochrome*
0	black	black
1	dark blue	grey level 1
2	dark red	grey level 2
3	purple	grey level 3
4	dark green	grey level 4
5	dark cyan	grey level 5
6	brown	grey level 6
7	light grey	grey level 7
8	dark grey	grey level 8
9	light blue	grey level 9
10	light red	grey level 10
11	light purple	grey level 11
12	light green	grey level 12
13	light cyan	grey level 13
14	yellow	grey level 14
15	white	white

Selecting Colours

The screen background colour is called the *paper* colour, the colour of text characters is called the *pen* colour, and the colour of graphics operations is called the *brush* colour.

When you enter RM Basic, the following colours are automatically selected:

	high resolution graphics (80-character) mode	*low resolution graphics (40-character) mode*
paper colour	0 (dark blue)	0 (black)
pen colour	3 (white)	15 (white)
brush colour	3 (white)	15 (white)

You can change the paper colour with the SET PAPER command:

```
NEW
10 SET MODE 80
20 FOR I% = 1 TO 3
30   SET PAPER I% : CLS
40   FOR J = 1 TO 1000 : NEXT J
50 NEXT I%
60 SET PAPER 0
```

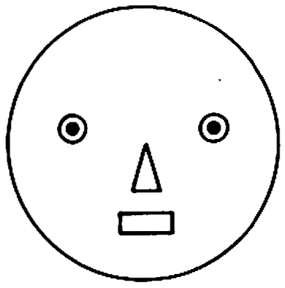
After you have run this program, change line 10 to SET MODE 40 and change line 20 to FOR I% = 1 TO 15. Run the program again.

Similarly, you can change the pen colour if the selected mode with the SET PEN command:

```
SET MODE 40
SET PEN 11
```

The prompt and cursor are now coloured light purple, and displayed text will be light purple until either you change the pen colour or you issue a SET MODE command. Once you use SET MODE, the colours are reset to the default state.

Graphics Colour



Brush colour selection works in the same way.

```
SET MODE 40
SET BRUSH 14
CIRCLE 20, 50, 50; 300, 100; 250, 150
```

You will see three yellow circles.

Again, the brush colour remains yellow until you change it or until you change the mode.

However, for a temporary change, you can use the BRUSH option with a graphics command to change the brush colour while the command is executed:

```
NEW
10 REM Draw a Face From outer space.
20 SET MODE 40
30 REM The Face and hair.
40 CIRCLE 50, 160, 125 BRUSH 10
50 LINE 160, 175; 167, 178; 158, 182; 166, 186
BRUSH 14
60 REM The eyes.
70 CIRCLE 5, 135, 130; 185, 130
80 CIRCLE 2, 135, 130; 185, 130 BRUSH 1
90 REM The nose.
100 AREA 160, 130; 157, 120; 163, 120 BRUSH 6
110 REM The mouth.
120 AREA 153, 110; 167, 110; 167, 105; 153, 105
BRUSH 6
```

This produces a face of several colours.

The default brush colour is 15 (white), and this is used in the CIRCLE command of line 40 where a BRUSH option is not specified. In lines 20, 30, 50, 60, and 70, the BRUSH option is used to change the colour for the duration of the instruction.

Dithered Patterns

In graphics, you can also select *dithered patterns* with SET BRUSH or with the BRUSH option. A dithered pattern is a colour pattern made up of a mixture of differently coloured dots.The pattern appears as a new colour or as a pattern of shades of colours.

In both high and low resolution mode, there are sixteen pattern numbers — 128 to 143. The first eight (128 to 135) are pre-defined, and the second eight can be defined by you with SET PATTERN (see the Keywords section). An example of pattern definition is given at the end of this chapter.

To see one of the pre-defined patterns, change BRUSH 10 to BRUSH 134 in line 40 of the face-drawing program.

Dithered patterns cannot be used with the PLOT and POINTS commands. If these commands are executed with a BRUSH setting in the range 128 to 143, RM Basic automatically converts the brush number to a number in the pure colour range.

Other Options

RM Basic gives great flexibility of drawing styles for points, lines, and areas. The system works in a similar way to the colour system. You can set a style or you can use a style option in an instruction.

Point Styles

There are five points styles:

style number	*style*
1	. (default)
2	+
3	*
4	0
5	x

You use SET POINTS STYLE to select one of the above styles:

```
SET POINTS STYLE 3
```

The following program uses this facility and the random number generating function, RND:

```
NEW
10 SET MODE 40, 99, 99
20 SET POINTS STYLE 4
30 FOR I%=1 TO 200
40    POINTS RND(100), RND(100)
50 NEXT I%
```

You use the STYLE option in a POINTS command:

```
POINTS 10, 10; 100, 50; 100, 100 STYLE 4
```

In the graph drawing program on page 6.9, you can change line 200 to:

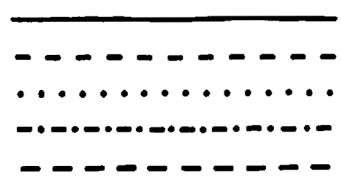
```
200 POINTS X, Y STYLE 2 BRUSH 3
```

This gives light purple crosses instead of white dots.

The example shows that you can use more than one option in a command.

Line Styles

You can select five defined line styles with SET LINE STYLE:



These styles are selected in a LINE command with the STYLE option. For example:

```
LINE 0, 0; 50, 50 STYLE 3
```

Styles 2 to 5 are made up of dots and/or dashes and spaces. The colour of the spaces is normally left unchanged (the current background colour), but you can choose a colour for the spaces by specifying a brush colour number after the style number.

For example, the following instruction draws a line of style 3 in the current brush colour with yellow spaces:

```
LINE 0, 0; 50, 50 STYLE 3, 14
```

Specifying either no brush colour or –1 leaves the colour below the spaces unchanged:

```
LINE 0, 0; 50, 50 STYLE 3, -1
```

You can also define your own line style with the STYLE 6 option:

```
SET LINE STYLE 6, " .. .. . "
```

You can create a string of dots and spaces up to sixteen characters in length.

The program below draws six horizontal lines to show the five standard line styles and the one defined previously as style 6. The dots/dashes of each line are drawn with a colour; all spaces are in the default colour (black).

```
100 REM Examples of line styles.
110 SET MODE 40
120 FOR Example% = 1 to 5
130   SET LINE STYLE Example%
140   Draw_Line Example% * 2 + 1, Example%
150 NEXT
160 SET LINE STYLE 6, " .. .. .. .."
170 Draw_Line 14, 6
180 REM Return line style to the default setting.
190 SET LINE STYLE 1 : SET CURPOS 0, 24
200 END
500 PROCEDURE Draw_Line Brush_Colour, Line_Style
510 Ypos = 250 -(Line_Style * 35)
520 PLOT "Line style" + STR$(Line_Style), 10, Ypos
+ 10
530 LINE 10, Ypos; 310, Ypos BRUSH Brush_Colour
540 ENDPROC
```

Area Filling Styles

The SET FILL STYLE command selects a style to fill in an area defined by the AREA, SLICE, and CIRCLE commands. The STYLE option gives a temporary area filling style.

SET FILL STYLE or STYLE can be followed by 1, 2 or 3:

- 1 (the default) selects a solid colour. Solid colours are either pure colours or dithered patterns.

- 2 selects a *hatching pattern.* This is two colours in a 16 pixel by 16 pixel repeated pattern. Compared with dithered patterns, these are large and bold.
- 3 selects drawing only the outline of the area, leaving the interior untouched.

Here is an example of the STYLE option:

```
AREA 50, 50; 100, 100; 318, 5 BRUSH 14 STYLE 3
```

In low resolution mode, this draws a triangle outline in yellow.

To specify a hatching pattern, use STYLE 2 followed by a number (in the range 0 to 5) that indicates which of the six patterns (see FILL STYLE in the Keywords section).

Each pattern uses the current brush colour and a second colour (for background) that you specify with a third number. For example:

```
CIRCLE 20, 50, 50 STYLE 2, 3, 2
```

draws a circle filled with hatching pattern 3, using the current brush colour and brush colour 2.

Specify –1 for the third number if you want to leave the background colour untouched.

The following instructions produce a rectangle filled with hatching pattern 5 using red and green:

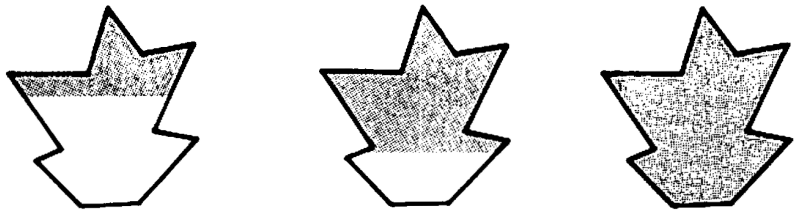
```
SET MODE 80
AREA 250, 0; 350, 0; 350, 150; 250, 150 BRUSH 2 STYLE
2, 5, 1
```

The next program displays all six hatching patterns in slices, using the current brush colour (white) and all possible second colours (except dithering patterns):

```
100 SET MODE 40 : SET DEG TRUE
110 SET ORIGIN 160, 125
120 A := 0
130 FOR Hatch% = 0 TO 5
140   FOR Colour2% = 0 TO 15
150     SLICE 120, A TO A + 3.75, 0, 0 STYLE 2,
Hatch%, Colour2%
160     A := A + 3.75
170   NEXT Colour2%
180 NEXT Hatch%
```

Flood Fills

When you need to fill in an irregular area it is easiest to use the FLOOD command. You specify a point in an area and the whole of that area is flooded with whatever colour you choose.



1 2 3

In its simplest form FLOOD is followed by an x and y coordinate pair giving a point in the area that you want to flood. For example:

```
FLOOD 180, 125
```

The current brush colour will be the filling colour unless you specify otherwise with the BRUSH option. For example:

```
CIRCLE 75, 200, 125 STYLE 3
FLOOD 150, 120 BRUSH 129
```

This floods the circle with dithered pattern 129.

Flooding stops when a plotted boundary or drawing area edge is reached. You can use the SET FLOOD EDGE command or the EDGE option to set a colour boundary:

```
10 SET MODE 40
20 CIRCLE 50, 150, 125 BRUSH 4
30 CIRCLE 50, 120, 125 BRUSH 2
40 FLOOD 120,125 BRUSH 3 EDGE 1, 4
```

This example draws a green circle and an overlapping red one. Then the whole of the screen except the green crescent is flooded with purple.

Plotting Characters

Plotting a string of characters on screen uses the PLOT command. In its simplest form, PLOT is followed by the string of characters to be plotted, and the x and y coordinates at which plotting starts. The string of characters is positioned horizontally. For example:

```
PLOT "Hello", 10, 20
```

plots Hello on the screen at the specified starting point and using the current brush colour.

Hello can be plotted at other points on the screen as well by adding more coordinate pairs. As with other graphics commands, you can use the BRUSH option. For example:

```
PLOT "Goodbye", 10, 20 ; 100, 50 ; 200, 30 BRUSH 2
```

plots Goodbye in red in three places on the screen.

You can only supply one string of characters to be plotted in each PLOT command.

Plotting Options

There are special options for plotting characters that work in the same way as other graphics options. You can select them with the SET *attrib* command, or you can temporarily specify them as an option in a PLOT command:

Selection	*Option*	*Action*
SET PLOT DIRECTION	DIRECTION	change plotting direction
SET PLOT SIZE	SIZE	change character size
SET PLOT FONT	FONT	change character font

Details of these features are given in the Keywords section, but here is an example:

```
SET MODE 80
PLOT "Upside down", 350, 125 DIRECTION 2 SIZE 4
```

Drawing Over Previous Drawings

When you are using graphics commands, there are two available drawing modes:

- destructive overdrawing (over plotting) — the default
- non-destructive overdrawing (XOR plotting)

With destructive overdrawing, you draw over anything that is already there, and the colour of the new drawing replaces the covered part the old one. Non-destructive overdrawing merges the colours of the new and old.

You can set the drawing mode with the SET OVER command or the OVER option using TRUE for destructive overdrawing and FALSE for non-destructive overdrawing.

FOR example, the following program draws two green circles, then draws two smaller circles over them using brush colour 2. The left smaller circle is drawn with destructive overdrawing to produce a red circle. The right one uses non-destructive overdrawing to produce a

brown circle:

```
10 SET MODE 40 : SET OVER TRUE
20 CIRCLE 20, 100, 125 BRUSH 4
30 CIRCLE 10, 100, 125 BRUSH 2
40 CIRCLE 20, 200, 125 BRUSH 4
50 CIRCLE 10, 200, 125 BRUSH 2 OVER FALSE
```

Non-destructive overdrawing has a very useful property. You can draw something, then redraw it by repeating the command — and it will disappear:

```
10 CIRCLE 50, 100, 100 OVER FALSE
20 FOR I = 1 TO 500 : NEXT
30 CIRCLE 50, 100, 100 OVER FALSE
```

The simple program below shows how this property can be used to obtain movement:

```
NEW
10 SET MODE 40 : SET BRUSH 14 : SET OVER FALSE
20 I% = 0
30 REPEAT
40   PLOT "MAGIC", I%, I%
50   FOR J% = 1 TO 150 : NEXT
60   PLOT "MAGIC", I%, I%
70   I% := I% + 10
80 UNTIL I% = 240
```

The following program gives another example of movement. Enter and run it, and see what happens. Press <BREAK> to stop the program:

```
100 SET MODE 40, 105, 60
110 Centre% := 50
120 Red% := 50 : Ylw% := 37 : Grn% := 24
130 REM Draw traffic light casing and leg.
140 LINE 40, 57; 40, 18 : LINE 40, 57; 60, 57
150 LINE 60, 57; 60, 18 : LINE 40, 18; 60, 18
160 AREA 48, 18; 52, 18; 52, 0; 48, 0
170 PLOT "STOP", Centre% -4.2, Red% -1.5 BRUSH 15
```

```
180 PLOT "GO", Centre% -4.5, Grn% -1.5 BRUSH 15
190 REM Draw traffic lights in sequence.
200 SET CURSOR 0 : SET OVER FALSE
210 CIRCLE 5, Centre%, Red% BRUSH 10
220 REPEAT
230    CIRCLE 5, Centre%, Ylw% BRUSH 14
240    Dummy$ := GET$(100)
250    CIRCLE 5, Centre%, Grn% BRUSH 12
260    CIRCLE 5, Centre%, Red% BRUSH 10
270    CIRCLE 5, Centre%. Ylw% BRUSH 14
280    Dummy$ := GET$(150)
290    CIRCLE 5, Centre%, Ylw% BRUSH 14
300    CIRCLE 5, Centre%, Grn% BRUSH 12
310    Dummy$ := GET$(100)
320    CIRCLE 5, Centre%, Red% BRUSH 10
330    CIRCLE 5, Centre%, Ylw% BRUSH 14
340    Dummy$ := GET$(150)
350 UNTIL FALSE
```

You can use the OVER TRUE or FALSE option in any graphics command that draws on the screen. This includes the BLOCK COPY and BLOCK WRITE commands (but not BLOCK READ).

BLOCK COPY can be used to copy an area of graphics to another part of the screen. BLOCK WRITE draws an area that has been stored in memory with BLOCK READ.

You are advised to use only destructive overdrawing if you are intending to flood fill areas. Non-destructive overdrawing can leave gaps in boundaries.

Repositioning and Scaling the Coordinates

AT and SCALE are two useful keywords that you can use in graphics commands to modify the coordinates temporarily.

AT specifies a temporary origin, and SCALE specifies temporary scaling of the x and y axes. The AT and SCALE options are placed immediately after the coordinates and before any other options.

The AT Option

After specifying coordinates for graphics drawing, you can move the position of what is drawn on the screen by repositioning the first coordinate pair. All the remaining coordinates will move to keep the original shape intact.

The new starting position is indicated by a pair of coordinates following the keyword AT. For example:

```
LINE 10,10; 20,20 AT 50,50
```

would produce a line between 50,50 and 60,60.

It is very useful to have the new starting position as a variable so that the shape can be easily and quickly reproduced in different places on the screen.

When you have a lot of coordinates, it is advisable to use integer values with AT, giving quicker execution.

Here is an example of the use of AT:

```
10 SET MODE 80
20 FOR X% = 60 TO 600 STEP 60
30   FOR Y% = 20 TO 220 STEP 20
40   AREA 5, 10; 25, 10; 25, 22; 5, 22 AT X%, Y%

     NEXT Y%
BRUSH RND(3)
50
60 NEXT X%
70 SET CURPOS 0, 24
80 END
```

The SCALE Option

Scaling to below or above an original drawing size is easily done using the SCALE option with all commands using coordinates (except BLOCK commands).

Two parameters follow SCALE, specifying the amount of scaling in both axes. The first parameter scales the drawing along the x axis and the second parameter scales it along the y axis. If you only put one parameter, both the x and y directions are scaled by the same amount. Again, integer values will improve the speed.

Try this simple use of the scale command:

```
10 SET MODE 40
20 AREA 50, 50; 200, 50; 200, 200; 50, 200 BRUSH 7
30 AREA 50, 50; 200, 50; 200, 200; 50, 200 SCALE 0.5
BRUSH 14
40 END
```

The yellow rectangle is a half-scale version of the grey one.

Because of the scaling of the axes, the smaller rectangle is drawn at a different area of the screen, even though the coordinates are the same in the two instructions. This is because SCALE 0.5 has temporarily multiplied the coordinates by a half.

To have a scaled version drawn in the same area as the original drawing, the origin must be the central point. For example, the scaled rectangle is nested inside the original when the origin is 125, 125:

```
10 SET MODE 40
20 SET ORIGIN 125, 125
30 AREA -75, -75; 75, -75; 75, 75; -75, 75 BRUSH 7
40 AREA -75, -75; 75, -75; 75, 75; -75, 75 SCALE 0.5
BRUSH 14
```

The following program uses one AREA instruction with SCALE to draw rectangles of different sizes:

```
NEW
500 SET MODE 40
510 REPEAT
520   SET CURSOR 0 : REM Switch GET cursor off.
530   X = RND(319) : Y = RND(249)
540   SET ORIGIN X, Y : REM Random origin.
550   AREA 0, 0; 20, 0; 20, 20; 0, 20 SCALE X/30, Y/30
BRUSH RND(16)-1
560 UNTIL GET(1)
570 SET CURSOR 1 : REM Switch GET cursor back on.
580 SET MODE 80
590 END
```

If you use AT and SCALE in the same command, the temporary origin is set up then scaling is done on the temporary coordinates.

Drawing Areas

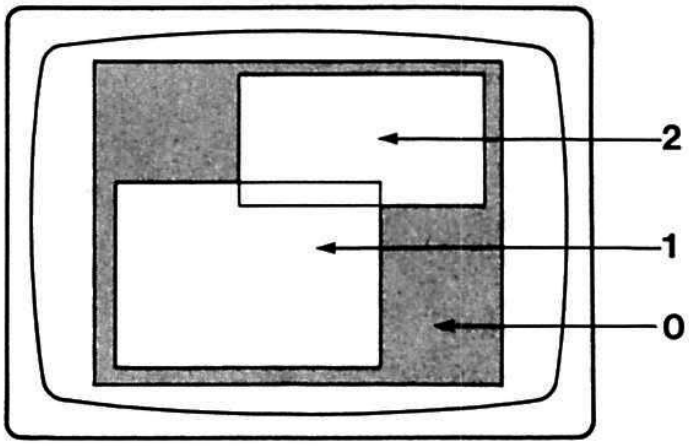
It has been mentioned that a border exists around the graphics drawing area. The area contained by this border is known as drawing area 0 — the whole graphics screen. The bottom-left coordinates are 0, 0 and the top-right ones are 319, 249 or 639, 249 depending on whether you are in low or high resolution mode.

Within drawing area 0 you can define up to 9 other drawing areas using the SET DRAWING command. The areas will be squares or rectangles because they are defined by corner coordinates. Each drawing area is given a number. For example:

```
SET DRAWING 1 TO 10, 10; 249, 150
SET DRAWING 2 TO 100, 130; 300, 230
```

defines drawing area 1 with bottom, left corner at 10, 10 and top, right corner at 150, 150, and area 2 with bottom, left corner at 50, 50 and top, right at 100, 100.

On screen, drawing areas 0, 1 and 2 would be:



When you want to restrict graphics or text to a defined area, call up the respective drawing area number. Another SET DRAWING command is used, but it is followed only by the number of the drawing area you want. For example:

```
200 SET DRAWING 1 TO 10, 10; 150, 150
210 SET DRAWING 2 TO 50, 50; 100, 100
220 SET DRAWING 1 : LINE 20, 20; 130, 130
230 SET DRAWING 2 : LINE 20, 20; 130, 130
```

This will attempt to draw the same length line in drawing area 1 and drawing area 2. The line will fit into drawing area 1 but it is curtailed by drawing area 2.

When you issue the CLG command, it clears the current drawing area; if the current drawing area is 2, CLG clears that area only.

You can find out the current drawing area using the ASK DRAWING command.

Changing The Colour Table

The entries in the low and high resolution colour tables can be changed with the SET COLOUR command.

For example, if you are working in 80-character (high resolution) mode, you may be tired of the four colours in the initial colour table. Perhaps you would prefer the following colour table:

Number	*Colour*
0	purple
1	brown
2	dark grey
3	yellow

This involves changing the colours associated with each number, using the SET COLOUR command. You specify the colour table number that you want to change, and the new colour that you want to assign to that number.

The new colour is specified by the relevant number from the list given in the COLOUR definition (see the Keywords section):

```
SET COLOUR 0 TO 3 : SET COLOUR 1 TO 6
SET COLOUR 2 TO 8 : SET COLOUR 3 TO 14
```

You can specify two flashing colours rather than one colour, and you can specify a flashing rate.

For example:

```
SET COLOUR 2 TO 10, 2, 4
```

sets colour number 2 to light red (10) flashing fast (2) with dark green (4).

The following modification to the outer space face program

of the Graphics Colour section produces a winking E.T.

```
NEW
 10  REM Draw a Face From outer space.
 20  SET MODE 40
 25  SET COLOUR 3 TO 1, 2, 15
 30  REM The Face and "hair".
 40  CIRCLE 50, 160, 125 BRUSH 10
 50  LINE 160, 175; 167, 178; 158, 182; 166, 186
     BRUSH 14
 60  REM The eyes.
 70  CIRCLE 5, 135, 130; 185, 130
 80  CIRCLE 2, 135, 130 BRUSH 1
 85  CIRCLE 2, 185, 130 BRUSH 3
 90  REM The nose.
100  AREA 160, 130; 157, 120; 163, 120 BRUSH 6
110  REM The mouth.
120  AREA 153, 110; 167, 110; 167, 105; 153, 105
     BRUSH 6
```

Colour number 3 of the 80-character table is changed to dark blue flashing with white (line 25). This colour is selected for the right eye in line 85.

You can use the MIX function in the SET COLOUR command to select the colour by monitor gun settings.

Using Arrays

Arrays are very useful for holding predefined shapes such as squares, rectangles or ducks. It is easy to put coordinate lists into arrays, and the commands POINTS, PLOT, LINE, CIRCLE, SLICE, AREA and FLOOD can all use arrays of coordinates.

The array reference system (see chapter 5) is used to assign an array to the graphics command.

If you decide to put the coordinates of several points

into an array, the data is arranged in a specific manner:

- The first element must be the total of the numbers of points to be plotted.
- The remaining elements are alternate x and y coordinate values positioning the points.

For example an array Dots% might hold the following values:

```
Dots%(0) = 2
Dots%(1) = 20
Dots%(2) = 10
Dots%(3) = 250
Dots%(4) = 100
```

This instruction plots the two points held in the Dots%() array.

```
POINTS Dots%()
```

Here is a program that draws a man. The trunk and legs coordinates are read into two arrays:

```
100 REM Draw a man.
110 SET MODE 40, 249, 249
120 DIM Trunk%(28), Legs%(14)
130 REM Read the body data.
140 FOR A% = 0 TO 28
150   READ Trunk%(A%)
160 NEXT A%
170 FOR B% = 0 TO 14
180   READ Legs%(B%)
190 NEXT B%
200 DATA 14, -20, 0, -20, 35, -35, -15, -50, -10, -25
210 DATA 70, -8, 70, -8, 75, 8, 75, 8, 70, 25, 70, 50
220 DATA -10, 35, -15, 20, 35, 20, 0
230 DATA 7, 20, 0, 35, -125, 20, -125, 0, -25, -20
240 DATA -125, -35, -125, -20, 0
250 REM Set up the dithered pattern for skin colour.
260 SET PATTERN 136, 1 TO 10, 15, 10, 15
270 SET PATTERN 136, 2 TO 15, 10, 15, 10
280 SET PATTERN 136, 3 TO 10, 15, 10, 15
290 SET PATTERN 136, 4 TO 15, 10, 15, 10
300 REM Start drawing.
310 SET ORIGIN 125, 225
320 REM Draw head.
330 CIRCLE 20, 0, 0 BRUSH 136
340 CIRCLE 5, -7, 5 : CIRCLE 5, 7, 5
350 CIRCLE 2, -7, 5 BRUSH 9
360 CIRCLE 2, 7, 5 BRUSH 9
370 POINTS 0, 0 STYLE 2 : LINE -2, -9; 2, -9
380 SET ORIGIN 125, 130
390 REM Draw body.
400 AREA Trunk%() BRUSH 14 : AREA Legs%() STYLE 2, 2
410 SET DEG TRUE
420 REM Draw hands.
430 SLICE 8, 70 TO 250, 45, -12 BRUSH 136
440 SLICE 8, 110 TO 290, -45, -12 BRUSH 136
450 END
```

Chapter 7
A Cursory Glance

This chapter gives details of how you control cursor movement and text handling on the screen.

Text refers to letters, numbers, symbols, and all the special characters of the two character sets described in the Technical Data section of your Owners Handbook.

Text is different from graphics, though you can use the special characters to draw pictures — a sort of low resolution graphics — and you can change the colour of both the characters and the background.

As with graphics drawing areas, you can divide your screen into writing areas.

The Text Screen

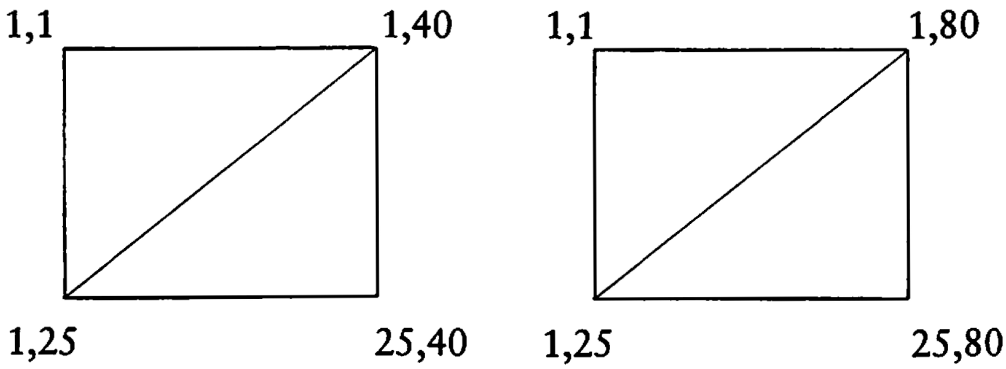
When you display characters on the screen using PRINT or other non-graphics commands, you are using text mode.

In this mode, the screen can be considered as a sheet of paper divided into columns and rows. In 80-character mode, there are 80 columns and 25 rows, so you can have lines of up to 80 characters:

```
SET MODE 80
PRINT "In 80-character mode, this sentence fits on one
line."
```

The *top left* corner of the screen (known as the home position) is at column 1, row 1.

In 40-character mode, there are 40 columns and 25 rows:



The lines can contain 40 characters that are broader:

```
SET MODE 40
PRINT "In 40-character mode, this sentence fits on two
lines."
```

Moving The Cursor

Whichever mode you are in, you can move the cursor to a specific row and column of the screen using the SET CURPOS command. For example, the following commands move the cursor to column 34, row 12 and print some text:

```
10 SET MODE 40
20 SET CURPOS 30,12
30 PRINT "HELLO WORLD"
40 FOR I = 1 TO 1500 : NEXT
50 SET MODE 80
60 SET CURPOS 30, 12
70 PRINT "HELLO WORLD"
80 END
```

After you have run the program, you will see that the prompt and cursor are positioned at the start of row 13 — where the program left the cursor.

You can also move the cursor relative to the current cursor position using the MOVE command. For example, the following will move the cursor down by three rows after each piece of text is written, starting at column 5:

```
10 SET MODE 80
20 FOR X = 1 TO 5
30   MOVE 5, 3
40   PRINT "HELLO WORLD"
50 NEXT X
```

If you wish, you can now find out where the cursor is by using ASK CURPOS:

```
ASK CURPOS X, Y
PRINT X; Y
```

X gives the column number; Y gives the row number.

Two useful commands for cursor control are CLS and HOME. CLS clears the screen and HOME moves the cursor to column 1, row 1:

```
CLS : HOME
```

You can use CLS –1 to clear the upper part of the screen — all characters to the left and above the current cursor position; CLS 1 clears everything to the right and below.

For example, type in and run this program:

```
NEW
10 CLS : HOME
20 FOR Column% = 1 TO 20
30   PRINT Column%
40 NEXT
```

You should have the numbers 1 to 20 down the left side of the screen with the prompt and cursor on the next row.

Type in the following:

```
MOVE 0, -10 : CLS -1
```

This moves the cursor up 10 rows and deletes everything to the left and above the new cursor position. You should now see:

```
:_2
 13
 14
 15
 16
 17
 18
 19
 20
:MOVE 0, -10 : CLS -1
```

The cursor was moved up 10 rows to row 12, not row 11, because you move the cursor down to row 22 when you press <ENTER> to enter the MOVE and CLS instructions.

CLL can be used to clear all or part of a line. It works in a similar way to CLS.

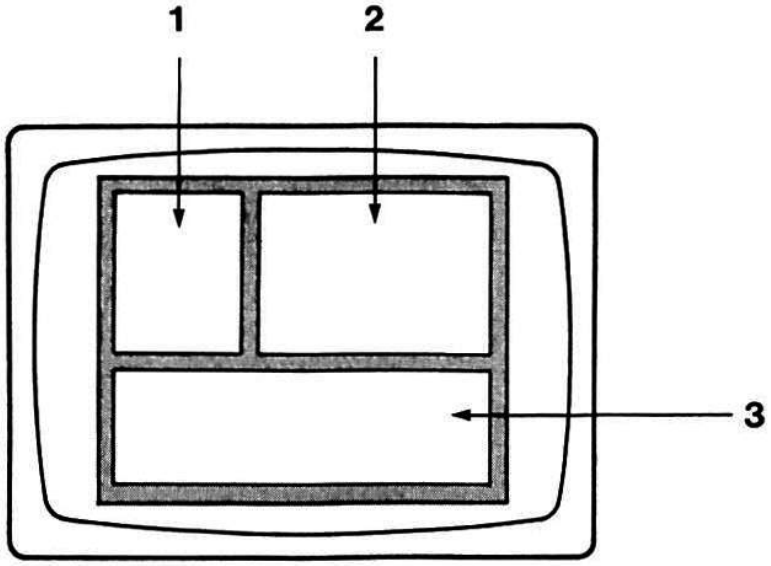
Writing Areas

In the same way that there are graphics drawing areas, you can divide your screen into up to nine writing areas (writing area 0 is always the whole screen).

You define the position and size of a writing area with the SET WRITING command. This works in a similar way to the SET DRAWING command except that the areas are defined in terms of column and row position.

For example, the following commands define the writing areas shown below:

```
SET WRITING 1 TO 5, 2; 28, 15
SET WRITING 2 TO 30, 2; 78, 15
SET WRITING 3 TO 5, 17; 78, 24
```



In each of these lines, the first number is the number of the writing area. The four numbers after TO are the numbers of the left column, the top row, the right column, and bottom row of the writing area.

You can select a writing area by using the SET WRITING command without any row and column numbers.

For example, the following program defines two writing areas at the start, and selects them later:

```
100 SET MODE 80
110 SET WRITING 1 TO 1, 1; 80, 20
120 SET WRITING 2 TO 1, 21; 80, 25
130 SET WRITING 2
140 SET CURPOS 24, 23
150 PRINT "To stop the program, press any key."
160 SET CURSOR 0
170 REPEAT
180   SET WRITING 1
190   SET CURPOS RND(70), RND(19)
200   SET PEN RND(3)
210   PRINT "RM Nimbus"
220 UNTIL GET(1)
230 SET CURSOR 1
240 SET MODE 80
250 END
```

Outputting commands, such as PRINT and LIST, when used after SET WRITING, write to the writing area that you selected. This also applies to CLS and CLL.

Also, in a similar way to the graphics options system, you can specify the writing area in an output command. This temporarily changes the writing area for the duration of the command.

The tilde (~) symbol, followed by the writing area number, is placed after the command:

```
PRINT ~2, X * X
```

Output commands like PRINT and LIST can be used in this way, and so can CLS and CLL. They operate from the last position of the cursor in the previous use of the specified writing area (the top, left corner if you have not used it before).

Other screen commands, such as MOVE and CURPOS, only work in the currently selected writing area.

Here is a program that defines four drawing areas. The program is listed in area 2, the results are output in area 3, program execution is traced in area 4, and line 90 is displayed for editing in area 1:

```
300 SET MODE 80
310 SET WRITING 1 TO 1, 22; 80, 25
320 SET WRITING 2 TO 41, 1; 80, 20
330 SET WRITING 3 TO 1, 2; 35, 12
340 SET WRITING 4 TO 1, 15; 30, 20
350 LIST ~2
360 TRACE ~4 1
370 FOR Num = 1 TO 8
380   PRINT ~3 Num, Num * Num, Num * Num * Num
390 NEXT Num
400 TRACE 0
410 SET WRITING 1
420 EDIT 380
```

When you have run this program, you can modify line 380 and run it again, observing the effects of your change.

Writing And Drawing

You can use writing areas and drawing areas for a combination of text and graphics.

The following program draws graphics in a drawing area and lists the program in a writing area:

```
NEW
500 SET MODE 80
510 SET DRAWING 1 TO 0, 0; 319, 249
520 SET WRITING 1 TO 42, 1; 80, 25
530 LIST ~1
540 REPEAT
550 SET CURSOR 0 : REM Switch GET cursor off.
560 X = RND(319) : Y = RND(249)
570 SET ORIGIN X, Y : REM Random origin.
580 SET DRAWING 1
590 AREA 0, 0; 20, 0; 20, 20; 0, 20 SCALE X/30, Y/30
BRUSH RND(4)-1 STYLE 3
600 UNTIL GET(1)
610 SET CURSOR 1 : REM Switch GET cursor back on.
620 SET MODE 80
630 END
```

Pen And Paper Colours

You saw in chapter 6 how the graphics drawing colour can be specified with SET BRUSH command, screen background or paper colour can be specified with SET PAPER, and the text colour can be specified with SET PEN.

All three types are selected from the same two colour tables (high resolution or 80-character and low resolution or 40-character). This means that redefinition of the tables with SET COLOUR affects brush colour, paper colour, and pen colour.

Unlike brush colours, paper and pen colours cannot be selected as options in a command. You can only select them with SET PAPER or SET PEN.
Also, there are no dithered patterns.

The next program is an example of the use of SET PAPER and SET PEN:

```
NEW
10 SET MODE 40
20 SET WRITING 1 TO 1, 1; 19, 25
30 SET WRITING 2 TO 21, 1; 40, 25
40 SET PAPER 1 : SET WRITING 1 : CLS
50 SET PAPER 2 : SET WRITING 2 : CLS
60 SET PAPER 6 : SET PEN 2
70 PRINT ~1 "WRITING AREA 1"
80 SET PAPER 5 : SET PEN 1
90 PRINT ~2 "WRITING AREA 2" : SET PEN 15
```

After this program has been run, you will see a multi-coloured cursor. This is because the cursor is displayed using XOR plotting.

If you run into difficulties when experimenting with paper and pen colours, just type SET MODE 40 or 80 to reset the colours.

As with brush colour, if you want to change a colour in the colour table, you can re-define with SET COLOUR.

For example, if you are in 80-character mode,the following command changes colour number 3 (normally white) to yellow, slow flashing with magenta:

```
SET COLOUR 3 TO 14, 1, 11
```

The following program makes characters flash between brown (number 6) and dark grey (number 8) at a fast rate:

```
10 SET MODE 40
20 SET COLOUR 3 TO 6, 2, 8
30 SET PEN 3
40 PRINT "Flashy"
50 SET PEN 15
```

Remember that colour 3 in the 40-character table remains flashing brown and dark grey until you reset the colours.

Text Characters

There are two character sets that can be used for text display. Both sets (256 characters and their identifying codes) are shown in chapter 1 of Technical Data in the Owners Handbook.

Towards the end of that chapter there is an RM Basic program that uses SET CHARSET to select the character set and the CHR$ function with PRINT to display all but the first 32 characters.

The PUT command can also be used to display characters by selecting the character code:

```
10 SET MODE 80
20 SET WRITING 1 TO 5, 2; 28, 15
30 SET WRITING 2 TO 30, 2; 78, 15
40 SET WRITING 3 TO 5, 17; 78, 24
50 FOR I% = 1 TO 350 : PUT ~1 178 : NEXT I%
60 FOR J% = 1 TO 700 : PUT ~2 177 : NEXT J%
70 FOR K% = 1 TO 650 : PUT ~3 176 : NEXT K%
80 END
```

Some of the characters coded 0 to 31 are control characters that produce actions rather than characters. Details of these are given in the above-mentioned chapter.

You can set the control character flag off with an escape sequence (see Technical Data) to obtain a few more characters. However, you will find that RM Basic will set the flag on occasionally.

You can select whether characters are to be displayed normally or underlined with the SET UNDERLINE command.

As in graphics, characters can be written in either over mode or XOR mode. The selection is made with the SET CHAROVER command.

The GET Cursor

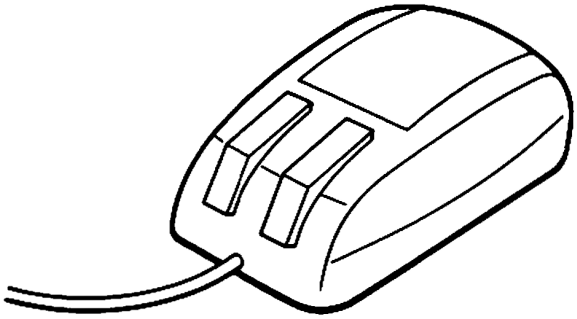
The cursors used with the RM Basic editor are always underline and block. The only other time a cursor is used is with the GET function when it is waiting for input. This can have its form changed by use of the SET CURSOR command.

Chapter 8
Mice and Joysticks

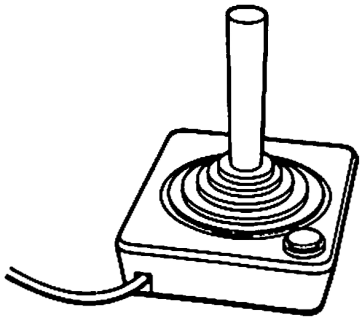
This chapter shows how to write programs that use input from a mouse or a joystick (or two joysticks).

Introduction

A *mouse* is a hand-held control unit that is about the same size as a mouse. You move the unit over a flat surface to alter its position that is monitored by the computer.



A *joystick* is a control unit consisting of a base with a moveable stick. You control its coordinates by moving this stick like a gear lever.



Frequently, programs plot a cursor on the screen at the position corresponding to the coordinate position of the mouse or joystick(s).

A mouse has two buttons and a joystick normally has one. When a button is pressed or released, a signal is sent to Nimbus. Your program can read the signal and take action accordingly.

The RM mouse and any standard joystick will plug into the socket on the Nimbus rear panel labelled "Mouse or Joy stick". RM Basic supports the simultaneous use of two joysticks. A joystick adapter is required for the socket to accept the connection of two joysticks.

Programming with a Mouse

After connecting a mouse, you have to inform Nimbus that you want to use the mouse for input. This is done by issuing the command:

```
SET MOUSE
```

If you want to turn off input from a mouse (this may increase program execution speed slightly), then use:

```
SET MOUSE OFF
```

Displaying the Mouse Cursor

The mouse does not automatically display a cursor on the screen. Your program has to track the position of the invisible cursor and display a cursor of your choice.

Mice and joysticks use the graphics system of x and y coordinates for screen position, and the ASK MOUSE command reports the current position of the invisible cursor:

```
ASK MOUSE Xpos%, Ypos%
```

This position can be used to draw a cursor. For example, to draw a cross, use:

```
POINTS Xpos%, Ypos% STYLE 2
```

Combine these commands into a simple program:

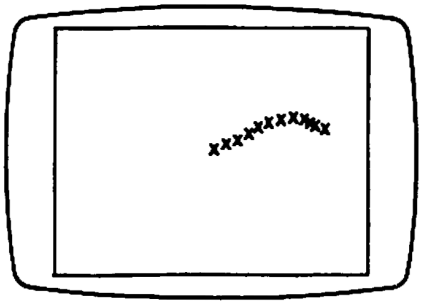
```
10 SET MODE 40
20 SET MOUSE
30 ASK MOUSE Xpos%, Ypos%
40 POINTS Xpos%, Ypos% STYLE 2
```

You will see a cross in the middle of the screen. But, if you move the mouse, the cross will remain stationary. This is because the above program asks for the mouse cursor position only once.

To track the movement, you have to obtain the mouse cursor position continuously:

```
NEW
10 SET MODE 40: SET CURSOR 0
20 SET MOUSE
30 REPEAT
40    ASK MOUSE Xpos%, Ypos%
50    POINTS Xpos%, Ypos% STYLE 2
60 UNTIL GET(1)
70 SET MODE 80
```

Now, when you move the mouse, a stream of crosses will be dislayed on the screen following the mouse movement. Stop the program by pressing a key on the keyboard.



Normally, you want a single cursor that shows you the mouse cursor position at any time, not a continuously drawn cursor.

To prevent the "cursor trail", you can use XOR plotting of the cross. By using the OVER FALSE option in line 50, then by repeating the instruction in line 60, the desired effect is obtained:

```
10 SET MODE 40: SET CURSOR 0
20 SET MOUSE
30 REPEAT
40    ASK MOUSE Xpos%, Ypos%
50    POINTS Xpos%, Ypos% STYLE 2 OVER FALSE
60    FOR Time = 1 TO 10 : NEXT
70    POINTS Xpos%, Ypos% STYLE 2 OVER FALSE
80 UNTIL GET(1)
90 SET MODE 80
```

This has produced a single cursor, but it flashes. To stop this flashing, make the following modification to the program:

```
100 SET MODE 40 : SET CURSOR 0
110 SET MOUSE
120 Xold% := -1 : Yold% := -1
130 REPEAT
140    ASK MOUSE Xpos%, Ypos%
150    POINTS Xold%, Yold% STYLE 2 OVER FALSE
160    POINTS Xpos%, Ypos% STYLE 2 OVER FALSE
170    Xold% := Xpos% : Yold% := Ypos%
180 UNTIL GET(1)
190 SET MODE 80
```

Using the Buttons

Having successfully tracked mouse movement, you can now make use of the mouse buttons.

As well as the current cursor position, ASK MOUSE reports whether or not the buttons are pressed (see the MOUSE definition in the Keywords section). For example:

```
ASK MOUSE Xpos%, Ypos%, Button%
```

When the right-hand button of the mouse is pressed, the value assigned to Button% is 1.

The above program is modified to make use of this by adding Button% to line 140 and by adding a new line:

```
165 IF Button% = 1 THEN CIRCLE 10, Xpos%, Ypos%
```

When you run this program, a circle will be drawn whenever the right-hand button is held down.

The final modification to the program, shown below, draws circles of different sizes and colours when the right-hand button is pressed, and it draws stars of different colours when the left-hand button is pressed. You stop the program by pressing both buttons.

```
100 SET MODE 40: SET MOUSE
110 Paint% := 0 : Xold% := -1 : Yold% = -1
120 POINTS Xold%, Yold% STYLE 2 OVER FALSE
130 REPEAT
140   ASK MOUSE Xpos%, Ypos%, Button%
150   Paint% := (Paint% + 1) MOD 16
160   POINTS Xold%, Yold% STYLE 2 OVER FALSE
170   POINTS Xpos%, Ypos% STYLE 2 OVER FALSE
180   IF Button% = 1 THEN CIRCLE Paint% * 4, Xpos%,
Ypos% BRUSH Paint%
190   IF Button% = 2 THEN POINTS Xpos%, Ypos% STYLE 3
BRUSH Paint%
200   Xold% := Xpos% : Yold% := Ypos%
210 UNTIL Button% = 3 : SET MODE 80
```

SET MOUSE Secrets

While running the above programs, you may have noticed that the initial position of the cross is always at the centre of the screen. You can change this position by specifying the start point in the SET MOUSE command. For example, change line 100 to:

```
SET MODE 40 : SET MOUSE 0, 0
```

Run the program and you will see that the initial position of the cursor is now at the bottom, left of the screen (with coordinates 0, 0).

If you do not specify initial coordinates, the cursor is positioned either at the centre of the screen or at the previous position if you have not reset the system since last using the mouse.

SET MOUSE can also be used to change the sensitivity of the mouse. This indicates how far the mouse can be moved before it reports its position (low sensitivity means large movement).

The fourth and fifth arguments of SET MOUSE indicate sensitivity in the x and y directions (a third argument must be present but is ignored). Values can be in the range 0 to 255. For example:

```
SET MOUSE 0, 0, 0, 1, 1
```

Sensitivity is highest with 1 and decreases as the value increases. Zero sensitivity is given by 0. The default setting is 1, 1.

Try different values in the above program and see what happens.

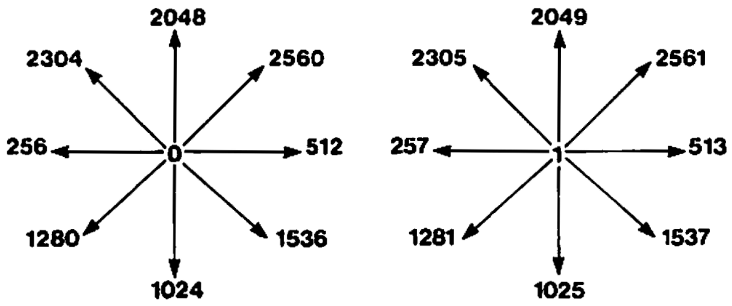
Programming With a Joystick

Programming with a joystick is very similar to programming with a mouse. The SET JOYSTICK command initiates the use of joysticks, sets the initial position of the joystick cursor, and sets joystick sensitivity (increasing values give *more* sensitivity).

The ASK JOYSTICK command is similar to ASK MOUSE except that information about button state and stick direction is provided in the third variable. For example:

```
ASK JOYSTICK Xpos%, Ypos%, Button%
```

The value assigned to Button% depends on whether or not the button is pressed and on the stick position:



If you want to find out if the button is pressed, use the AND logical operator (see appendix B) to combine Button% with 1. This will discard the direction information:

```
IF Button% AND 1 THEN CIRCLE 10, Xpos%, Ypos%
```

The combination gives TRUE when the button is pressed.

Here is a program that uses this technique. It draws circles when the stick is pointing in a south-west direction and stops when you press the button:

```
NEW
100 SET MODE 40: SET JOYSTICK
110 Xold% := -1 : Yold% := -1
120 REPEAT
130    ASK JOYSTICK Xpos%, Ypos%, Button%
140    POINTS Xold%, Yold% STYLE 2 OVER FALSE
150    POINTS Xpos%, Ypos% STYLE 2 OVER FALSE
160    IF Button% = 1280 THEN CIRCLE 20, Xpos%, Ypos%
BRUSH RND(15) STYLE 2, RND(6)-1, 15
170    Xold% := Xpos% : Yold% := Ypos%
190 UNTIL BUTTON% AND 1
200 SET MODE 80
```

Other Mouse and Joystick Keywords

You have seen how the ASK SOUND and ASK JOYSTICK commands give information about cursor position and button state. RM Basic also contains the POSX(), POSY(), and BUTTONS() functions that provide this information.

You place a number in the brackets to indicate a control unit:

- 0 indicates a mouse
- 1 indicates joystick 1
- 2 indicates joystick 2

For example:

```
BUTTONS(1)
```

provides the current button state of a joystick (or number 1 joystick if you are using two). The function returns 0 if the button is not pressed or 1 if it is pressed.

POSX() and POSY() provide the current x and y coordinates of the specified control unit. There are two other functions, JOYX() and JOYY(), that return the current x-axis and y-axis directions of the specified joystick.

Here is an example. The following program works with mouse or joystick:

```
100 SET MODE 40 : SET CURSOR 0
110 INPUT "Mouse or joystick (M or J) "; Answer$
120 IF Answer$ == "M" THEN SET MOUSE : Argument% := 0
ELSE SET JOYSTICK : Argument% := 1
130 Paint% := 0 : Xold% := -1 : Yold% = -1 : CLS
140 SET POINTS STYLE 2 : SET PLOT SIZE 2
150 REPEAT
160    Paint% := Paint% MOD 15 + 1
170    Button% := BUTTONS(Argument%) : Xpos% :=
POSX(Argument%) : Ypos% := POSY(Argument%)
180    POINTS Xold%, Yold% OVER FALSE
190    POINTS Xpos%, Ypos% OVER FALSE
200    IF BUTTONS(Argument%) THEN PLOT "Wham", Xpos%,
Ypos% BRUSH Paint% SIZE 2
210    Xold% := Xpos% : Yold% := Ypos%
220 UNTIL GET(1)
230 SET MODE 80
```

Press the right-hand button of a mouse or the button of a joystick to draw on the screen. Press any key on the keyboard to leave the program.

Because you cannot obtain simultaneous readings of position and button status, these functions give less accurate readings than those returned by ASK MOUSE or ASK JOYSTICK. However, they can be easier to use.

More Adventurous Programs

The following program uses two procedures — one to draw circles with a mouse, and the other to make the circles change colour. Press the right-hand mouse button to draw circles and the left-hand button to change their colour.

```
 10 GLOBAL Button%
20 SET MODE 40 : SET MOUSE 0, 0
30 SET POINTS STYLE 2
40 REPEAT
50    Rope
60 UNTIL Button% = 3
70 SET MODE 80 : END
80 :
90 PROCEDURE Rope
100 GLOBAL Button%
110 Xold% := -1 : Yold% := -1 :Paint% := 1
120 REPEAT
130    ASK MOUSE Xpos%, Ypos%, Button%
140    POINTS Xold%, Yold% BRUSH 3 OVER FALSE
150    POINTS Xpos%, Ypos% BRUSH 3 OVER FALSE
160    IF Button% = 1 THEN CIRCLE 10, Xpos%, Ypos%
BRUSH Paint%
170    IF Button% = 2 THEN Change_Colour
180    IF Button% = 3 THEN LEAVE
190    Xold% := Xpos% : Yold% := Ypos% : Paint% :=
(Paint% + 1) AND 15
200 UNTIL FALSE
210 ENDPROC
220 :
230 PROCEDURE Change_Colour
240 GLOBAL Colr%
250 REPEAT
260    FOR Ink% = 1 TO 15
270       Colr% := (Colr% + 1) AND 15
280       SET COLOUR Ink% TO Colr%
290    NEXT Ink%
300    IF BUTTONS(0) AND 1 THEN LEAVE
310 UNTIL FALSE
320 ENDPROC
```

You can leave the program by pressing both buttons.

The global variable, Colr%, is an example of the use of the global declaration within a procedure to retain the variable value (referred to at the end of chapter 5). This ensures that colour change always follows on from where it stopped after the previous procedure call. Finally, the following program is an example of how a mouse can be used to choose from a selection of options on the screen (a menu).

Four procedures are employed. The first, "Screen-Setup", produces the options and instructions on the screen. Then "Select" controls mouse movement and traps mouse button or keyboard interruptions — used to control the program flow.

The "Choice" procedure finds the mouse cursor position after selection, and sends the appropriate values to the procedure, "Display". This controls what appears on the screen.

```
10 GLOBAL Key% : SET CURSOR 0
20 REPEAT
30    SET MODE 40, 99, 99
40    Screen_Setup
50    Select
60 UNTIL Key% = 27
70 SET MODE 80 : END
80 :
90 PROCEDURE Screen_Setup
100 FOR A% = 3 TO 83 STEP 26
110    AREA 0, 0; 0, 20; 25, 20; 25, 0 AT A%, 70 STYLE
3 BRUSH 1
120 NEXT
130 RESTORE 180
140 FOR I% = 1 TO 8
150    READ Word$, Wordx%, Wordy%
160    PLOT Word$, Wordx%, Wordy%
170 NEXT I%
```

```
180 DATA "Poppy",6,82,"red",8,77,"Oxford",32,82,
    "blue",34,77,"Sea"
190 DATA 61,82,"green",59,77,"Magic",85,82,
    "magenta"82,77
200 SET WRITING 1 TO 1, 10; 40, 25
210 PRINT~1!!! "Move the mouse cursor to the desired
220 PRINT~1 "box and press the right-hand button."!!
230 PRINT~1 "To leave your choice, press the left-"
240 PRINT~1 "hand button."!
250 PRINT~1 "To leave the program, press ESC."
260 ENDPROC
270 :
280 PROCEDURE Select
290 GLOBAL Key%
300 SET MOUSE 55, 60
310 Xold%:= -1 : Yold%:= -1
320 REPEAT
330    ASK MOUSE Xpos%, Ypos%, Button%
340    POINTS Xold%, Yold% STYLE 2 OVER FALSE
350    POINTS Xpos% : Ypos% STYLE 2 OVER FALSE
360    IF Button% = 1 THEN Choice : LEAVE
370    Xold% := Xpos% : Yold% := Ypos% : Key% := GET(1)
380 UNTIL Key%
390 ENDPROC
400 :
410 PROCEDURE Choice
420 ASK MOUSE Xpos%, Ypos%
430 IF (Xpos% > 3 AND Xpos% < 28) AND (Ypos% > 60 AND
    Ypos% < 85) THEN Display 8, 2, 10, 6
440 IF (Xpos% > 29 AND Xpos% < 54) AND (Ypos% > 60 AND
    Ypos% < 85) THEN Display 15, 1, 9, 7
450 IF (Xpos% > 55 AND Xpos% < 70) AND (Ypos% > 60 AND
    Ypos% < 85) THEN Display 4, 5, 13, 12
460 IF (Xpos% > 81 AND Xpos% < 99) AND (Ypos% > 60 AND
    Ypos% < 85) THEN Display 0, 3, 11, 14
470 ENDPROC
```

```
480 PROCEDURE Display A%, B%, C%, D%
490 SET MODE 80 : SET MOUSE
500 SET COLOUR 0 TO A% : SET COLOUR 1 TO B% : SET
COLOUR 2 TO C% : SET COLOUR 3 TO D% : CLS
510 REPEAT
520   PLOT "Hi there", RND(639), RND(249) BRUSH RND(3)
SIZE RND(4) DIRECTION RND(4)-1
530 UNTIL BUTTONS(0) AND 2
540 ENDPROC
```

Chapter 9
Blowing Your Own Trumpet

Sound

Your Nimbus has extensive sound capabilities and a built-in speaker. Using RM Basic, you can produce music by specifying the pitch, duration and volume of notes. Also, you can produce beeps as warnings or as sound effects in games programs.

This section gives details of how to program sound operations.

Setting Up

To switch Nimbus sound operations on or off, use the SET SOUND command:

`SET SOUND TRUE` enables sound output.

`SET SOUND FALSE` disables sound output.

When you are programming with sound, always put SET SOUND TRUE at the beginning of the program. Sound output may have been switched off by a previous program.

The Note Command

You use the NOTE command to produce musical notes or non-musical noises. You will learn about noises later. Here, we concentrate on music.

First, enter and run this short program:

```
10 SET SOUND TRUE
20 NOTE 148, 100
```

You should hear a note that plays for one second.

The first number after NOTE specifies the pitch of the note. This number can be in the range 1 (very low pitch) to 388 (very high pitch). The second number specifies the duration of the note in centiseconds (100 centiseconds equals 1 second).

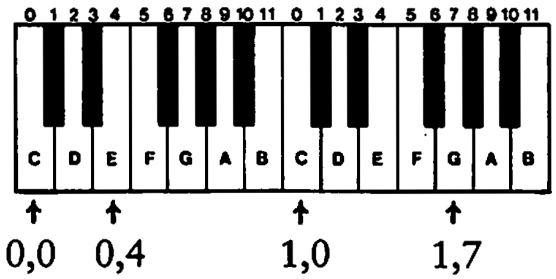
The following program demonstrates simple sound production with notes and an accompanying screen display:

```
10 SET MODE 40
20 SET SOUND TRUE
30 FOR I% = 1 TO 250
40    PLOT CHR$(14), RND(319), RND(249) BRUSH RND(15)
50    NOTE RND(340), 10
60 NEXT
```

To make it easy to specify the note that you want in terms of musical notation, use the PITCH function:

```
NOTE PITCH(0, 0), 100
```

The first number can be in the range –3 to 4 and it specifies one of eight octaves (–3 indicates the lowest and 4 indicates the highest); the note, middle C, lies in octave 0. The second number, in the range 0 to 11, indicates the note:



Example PITCH numbers

The following program uses PITCH to play the scale of C starting at middle C:

```
NEW
10 SET SOUND TRUE
20 FOR I% = 1 TO 8
30   READ Octave%, Note_Number%
40   NOTE PITCH (Octave%, Note_Number%), 40
50   DATA 0, 0, 0, 2, 0, 4, 0, 5, 0, 7, 0, 9, 0, 11, 1, 0
60 NEXT
```

You can change the data in line 50 to play other scales.

Glissandos

You can produce a note of sliding pitch (a musical glissando effect) with the NOTE command. Instead of one pitch number, you specify the starting pitch and the finishing pitch:

```
NOTE 148 TO 160, 100
```

This plays a note that starts at middle C and slides up one octave in one second.

You can slide the pitch upwards or downwards, as shown by this program:

```
NEW
10 REM : Fireworks
20 SET SOUND TRUE
30 FOR I% = 1 TO 5
40   NOTE 350 TO 75 * I%, 200
50 NEXT
```

Forte To Piano

You can control the volume of notes with a third number in the NOTE command. This number can be in the range 0 (no sound) to 15 (loudest). For example:

```
NOTE 148, 50, 5
```

plays with low volume.

In previous examples, where a volume number was not specified, the default value of 15 is used.

The no sound (0) option is useful when you want to place a silence (a rest) in music. For example, the next program plays this bar:

The rest is specified by using 0 for volume while other notes use 15:

```
10 FOR I% = 1 TO 6
20    READ Octave%, Note_Number%, Duration%, Volume%
30    NOTE PITCH(Octave%, Note_Number%), Duration%,
Volume%
40    DATA 1, 0, 60, 15, 0, 0, 20, 0, 0, 9, 20, 15
50    DATA 1, 0, 20, 15, 1, 2, 20, 15, 1, 4, 20, 15
60 NEXT
```

Sound Plumbing

When notes are played, RM Basic puts them in a queue and they are played one after the other when they reach the front of the queue.

A queue can hold up to 6 notes. If RM Basic has more notes to add to a full queue, it waits until there is space. This is provided by earlier notes that leave the queue, producing the sound that we hear.

The QUEUE function reports how many spaces are in the queue — it returns 6 when the queue is empty and 0 when it is full.

The following program shows the way the queue works. It plays fifteen notes of one second duration and displays a screen message as each note is released into the queue.

```
10 SET SOUND TRUE
20 SET HOLD FALSE
30 For N% = 1 TO 5
40    PRINT ! "Another note"
50    NOTE 50 + RND(200), 100
60 NEXT N%
```

Notice how the first six screen messages are displayed very quickly as the queue is filled up. The remaining messages are displayed just after a note has sounded, since RM Basic has to wait for a space in the queue before it can enter more notes.

Line 20 contains the SET HOLD command. When this is set FALSE (the default setting), each note flows out of the queue as soon as the previous note has been issued.

You can use SET HOLD TRUE to hold notes until you want them to be issued. In effect, you are putting a tap on the system.

As you will see later, the tap is opened and notes are released with the NOISE command. This is useful for synchronization of notes and for putting rests in music.

There is a danger that your program will hang up if you have not opened the tap for RM Basic to release all the notes. If this does happen, press <BREAK>.

Tempo

The duration of each note, or rest, depends on the speed of the music (the tempo). The table below shows the standard musical tempos with metronome settings (crochets per minute). Also shown are the equivalent values, in centiseconds, to be used in the NOTE command to give a note of one crochet in length.

Tempo	*Crochets per minute*	*Centiseconds per crochet in NOTE*
Very slow (largo)	40–60	150–100
Slow (adagio)	66–76	90–79
Medium (andante)	76–108	78–56
Quite Fast (moderato)	108–120	55–50
Fast (allegro)	120–168	49–36
Very fast (presto)	168–208	35–29

For example, bars played with a crochet length of 70 centiseconds are using a medium tempo. The time values of other notes, with respect to the crochet, can be calculated from the following table:

Note	*Symbol*	*Duration*
Breve	𝅜	8
Semibreve	𝅝	4
Dotted minim	𝅗𝅥.	3
Minim	𝅗𝅥	2
Dotted crochet	♩.	1 1/2
Crochet	♩	1
Dotted quaver	♪.	3/4
Quaver	♪	1/2
Triplet	♪3	1/3
Semiquaver	𝅘𝅥𝅯	1/4
Demisemiquaver	𝅘𝅥𝅰	1/8

Other Sound Options

The following sound features are available:

- three voices
- envelopes
- quarter semitones
- non-musical sounds
- synchronization of held sound

Three Voices

It was mentioned earlier that notes are put in a queue before they are played. There are three voice queues, so you can play up to three notes at the same time (chords).

This is done using SET VOICE command to select voice queue 1, 2, or 3. For example, to select voice queue 2:

```
SET VOICE 2
```

The next program uses two voices to play music. Two arrays are used to handle the data for the two voices. All the notes played by voice 1 are in octave 1, and those of voice 2 are in octave 0, so the octave number is not necessary in the data.

Although the total length of notes played by the two voices is the same, the length of individual notes is different. This causes loss of synchronization that is corrected by insertion of rests between bars in the voice 2 data.

```
 10 SET SOUND TRUE
20 DIM First%(53), Second%(53)
30 FOR I% = 0 TO 53
40    READ First%(I%)
50    DATA 0,100,15,4,50,15,2,100,15,7,50,15
60    DATA 4,50,15,2,50,15,0,50,15,2,100,15,7,50,15
70    DATA 4,50,15,2,50,15,0,50,15,2,50,15
80    DATA 4,50,15,5,50,15,4,100,15,2,50,15
90    DATA 0,150,15
100 NEXT I%
110 FOR J% = 0 TO 53
120    READ Second%(J%)
130    DATA 4,100,15,0,50,15,7,150,15,0,100,15
140    DATA 4,50,15,0,2,0,7,150,15,0,100,15,4,50,15
150    DATA 5,100,15,2,50,15,7,100,15,0,2,0,5,50,15
160    DATA 0,2,0,4,150,15,0,0,0,0,0,0
170 NEXT J%
```

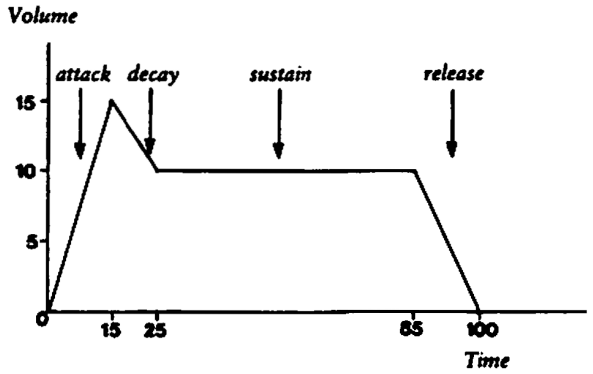
```
180 FOR K% = 0 TO 51 STEP 3
190    SET VOICE 1
200    NOTE PITCH(1, First%(K%)), First%(K% + 1),
First%(K% + 2)
210    SET VOICE 2
220    NOTE PITCH (0, Second%(K%)), Second%(K% + 1),
Second%(K% + 2)
230 NEXT K%
```

With three voices, the harmonic possibilities are considerable, as shown by the Play Bach program on the Welcome disk.

Envelopes

Notes played on different instruments sound different. This is partly due to the variation in volume as the note is played. This pattern of variation is called the envelope.

There are four stages of a sound envelope called attack, decay, sustain, and release:



Like SET DRAWING and SET WRITING, you use SET ENVELOPE both to define an envelope and to select an envelope.

You can define nine envelopes with:

SET ENVELOPE *number* TO *time1*, *volume1*; *time2*, *volume2*; *time3*, *volume3*; *time4*

where: *number* is a number in the range 1 to 9; *time1* is the time in centiseconds to reach final attack volume; *volume1* is the attack volume number (0 to 15); *time2* is the time in centiseconds to reach the decay volume; *volume2* is the decay volume number (0 to 15); *time3* is the time in centiseconds to reach the sustain volume; *volume3* is the sustain volume number (0 to 15); *time4* is the time in centiseconds to reach volume 0 if no more notes are played.

The duration of the note is given by *time1* + *time2* + *time3*. When notes are played one after the other, *time4* is ignored, and there is no interval between the end of one note and the start of the next.

The default setting is envelope 0 with constant maximum volume and 5 centiseconds duration. Notes are played with these values when you have not specified any other values.

The envelope in the above diagram is defined by:

```
SET ENVELOPE 1 TO 15, 15; 10, 10; 60, 10; 15
```

and selected by:

```
SET ENVELOPE 1
```

This program demonstrates a xylophone-like envelope shape:

```
NEW
10 SET ENVELOPE 5 TO 10, 15; 0, 15; 0, 15; 10
20 SET ENVELOPE 5
30 FOR I% = 1 TO 30
40    NOTE PITCH (RND(3), RND(11))
50 NEXT
```

You can also use the ENVELOPE keyword in the NOTE command to specify the envelope for the duration of the command. For example:

```
NOTE PITCH(2,6) ENVELOPE 4
```

Using the same notes as in the previous two-voice program, the next program defines four envelopes, all with piano-like envelope shapes, for the four note lengths. Now, the data only contains the note number and the envelope number.

```
10 SET SOUND TRUE
20 DIM First%(35), Second%(35)
30 FOR I% = 0 TO 35
40   READ First%(I%)
50   DATA 0,3,4,2,2,3,7,2,4,2,2,2,0,2,2,3,7,2,4,2,2,
2,0,2,2,2
60   DATA 4,2,5,2,4,3,2,2,0,4
70 NEXT I%
80 FOR J% = 0 TO 35
90   READ Second%(J%)
100   DATA 4,3,0,2,7,4,0,3,4,0,1,7,4,0,3,4,2,5,3,2,2,
7,3
110   DATA 0,1,5,2,0,1,4,4,0,0,0,0,
120 NEXT J%
130 SET ENVELOPE 1 TO 0, 0; 2, 0; 0, 0; 0
140 SET ENVELOPE 2 TO 5, 15; 10, 15; 0, 15; 70
150 SET ENVELOPE 3 TO 10, 15; 20, 15; 0, 15; 70
160 SET ENVELOPE 4 TO 15, 15; 30, 15; 0, 15; 105
170 FOR K% = 0 TO 34 STEP 2
180   SET VOICE 1
190   NOTE PITCH(1, First%(K%)), ENVELOPE
First%(K% + 1)
200   SET VOICE 2
210   NOTE PITCH(0, Second%(K%)), ENVELOPE
Second%(K% + 1)
220 NEXT K%
```

Non-Musical Sounds

You can radically change the sounds that are produced by the NOTE command. There is a sound attribute called TONE that controls whether notes or randomly pitched noise (pink noise) are produced:

- SET TONE TRUE selects notes (the default)
- SET TONE FALSE selects noise.

For example:

```
SET TONE FALSE
NOTE 160, 100
```

produces a noise.

As you will see in the "Noise Command" section, you can change the pitch of the noise with the NOISE command.

Quarter Semitones

The PITCH function allows you to specify notes that lie between the notes of the chromatic scale. This is useful if you want to play bagpipe music, eastern music, or modern western music where intervals of less than a semitone are common.

PITCH can take a third number that indicates where the pitch is with reference to the chromatic note. For example:

```
PITCH(0, 8, 1)
```

indicates a pitch that is a quarter semitone above G£

Note	*Value in PITCH function*
G	0, 7, 0
	0, 8, –3
	0, 8, –2
	0, 8, –1
G#	0, 8, 0
	0, 8, 1
	0, 8, 2
	0, 8, 3
A	0, 9, 0

The Noise Command

The NOISE command performs two functions:

- it changes the pitch of noise
- it synchronizes the playing of sound that has been held

To change the pitch, use NOISE with one number in the range 1 to 96. For example:

```
SET TONE FALSE
NOTE 160, 100
NOTE 160, 100 : NOISE 70
```

This will produce two noises, the second higher pitched than the first.

As you saw in "Sound Plumbing", notes or noises in the voice queue can be held until they are released by execution of a NOISE command. The HOLD attribute controls whether or not sound will be played as soon as it reaches the top of the queue:

`SET HOLD TRUE` selects held sound

`SET HOLD FALSE` selects immediate release (the default)
The scale-playing program is used to demonstrate how note-holding works. It is shown below with the addition of attribute settings in line 20, and a NOISE command in line 40:

```
10 SET SOUND TRUE
20 SET TONE TRUE : SET HOLD FALSE
30 FOR I% = 1 TO 8
40    READ Octave%, Note_Number%
50    NOTE PITCH(Octave%, Note_Number%), 40
60    NOISE I% * 12, 50
70    DATA 0,0,0,2,0,4,0,5,0,7,0,9,0,11,1,0
80 NEXT
```

As line 20 sets TONE TRUE and HOLD FALSE, the NOISE command will have no effect when you run the program, and the scale will be played as it was before.

If you change SET HOLD FALSE to SET HOLD TRUE and run the program, you will hear spaces between the notes. This is because the NOISE command is now operative and notes are released at 50 centisecond intervals — indicated by the second number in the command (the first number is ignored).

If you change SET TONE TRUE to SET TONE FALSE and run the program again, the first number of the NOISE command changes the noise pitch each time a noise is produced.

The following program plays a well known tune. It demonstrates the use of the QUEUE function to ensure that a blockage does not occur.

The loop structure of lines 120 to 150 ensures that the queues are kept full, and lines 160 to 180 are necessary to play notes that would otherwise be left in the note queue:

```
 10 DIM Play%(22), Duration%(22)
20 SET SOUND TRUE : SET HOLD TRUE : SET TONE TRUE
30 FOR I% = 1 TO 22
40    READ Octave%, Note_Number%, Duration%(I%)
50    Play%(I%) := PITCH(Octave%, Note_Number%)
60 NEXT
70 DATA 0,7,100,0,6,25,0,7,25,0,9,50,0,4,100,0,2,100,
0,0,100
80 DATA -1,11,25,0,0,25,0,2,50,-1,9,200,-1,11,100,0,
1,25,0,2,50,0,4,25
90 DATA 0,9,100,0,2,100,0,7,100,0,7,25,0,6,50,0,4,25,
0,2,200
100 :
```

```
110 I% := 1
120 REPEAT
130   IF QUEUE(0) > 1 THEN NOISE 1, 26
140   IF QUEUE(1) > 1 THEN NOTE Play%(I%),
Duration%(I%) := I% + 1
150 UNTIL I% > 22
160 REPEAT
170   NOISE 1, 26
180 UNTIL QUEUE(1) = 6
```

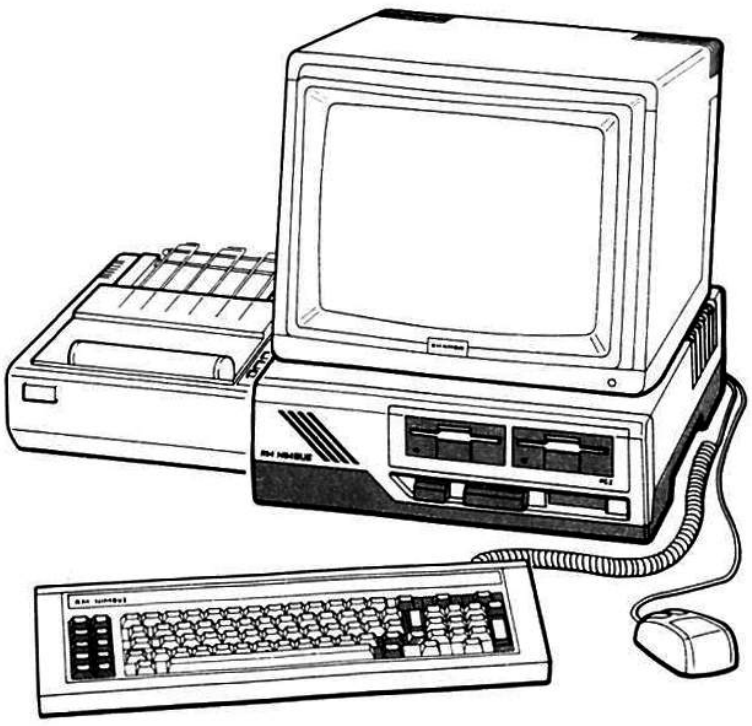
Chapter 10
The Outside World

This chapter gives details of the use of *channels* to communicate with the printers, files, and other devices. The RM Basic sound facilities are also described.

Channels

Normally, RM Basic receives information from the keyboard and sends information to the screen. This is done using a *channel* — a link between RM Basic and an input or output device.

A channel is identified by its number. Channel 0 is the keyboard input and screen output link that is automatically set up when you enter RM Basic. Channel 2 is used for output to a printer.



10.1

Channels 11 to 127 are unspecified channels that you can open and use with any device defined by an operating system utility — see "Operating System Concepts" in the Nimbus Owners Handbook. These are the channels that you use to handle disk files.

Using a Printer

To set up your printer, refer to "Printers: How to use them" in the Nimbus Owners Handbook.

When you enter an instruction such as:

```
PRINT "Hello"
```

RM Basic interprets this as:

```
PRINT #0, "Hello"
```

Where #0 represents channel 0. Consequently, the greeting is output on the screen.

If you want "Hello" to be printed on your printer, all you have to do is to specify channel 2:

```
PRINT #2, "Hello"
```

This rule applies to all commands that normally write to the screen. For example, if you want a printed listing of the current program in memory, enter:

```
LIST #2
```

Often, printers have features, such as bold typing, that are invoked when either a character or a sequence of characters is sent to the printer.

Details of these features will be found in your printer manual. You can use the PUT command to output control codes.

For example, if you have an Epson printer:

```
PUT #2, 15
```

sets the condensed print feature, and:

```
PUT #2, 27, "S", 0
```

sets the superscript print feature.

Files

You saw in chapter 1 how you can store a program on disk with the SAVE command. Data can also be stored on disk in files.

Data files, in microcomputer terms, are information recorded on to a storage device — a list of names and addresses or exam results, for example.

Using channel numbers, RM Basic allows you to write programs that store and retrieve information in files. As you will see from the following section, this makes it easy for you to manipulate data.

Storing Information

To store data, you have to open a channel and name the file that you are going to create. All this is done by the CREATE command. For example:

```
CREATE #11, "People"
```

opens channel 11 and creates a file called "People".

Now, you can write to "People" using PRINT #11. Here is an example program that takes input from the keyboard and writes it to the disk file:

```
10 CREATE #11, "People"
20 FOR I% = 1 TO 4
30 INPUT "Enter name:", Name$:REM Get name from
keyboard
40 PRINT #11, Name$ : REM Write name to file
50 NEXT I%
60 CLOSE #11
70 END
```

The CLOSE# 11 instruction of line 60 closes the file. You should always close files when you have finished writing to them.

Here is what happens when you run the program (with some example inputs):

```
Enter name : Keats
Enter name : Burns
Enter name : Thomas
Enter name : Joyce
```

The poets are now stored in a file called "People". You can use the DIR command to check that the file has been recorded on your disk. Hopefully, you will find a file called PEOPLE.BAS.

The BAS associated with PEOPLE is called a file extension. When you create a file, .BAS is added to your filename unless you specify otherwise when you create your file.

File extensions (three characters in length) can be chosen to remind you of what a file contains.For example, the "People" file consists of names, so you could use TXT — short for text:

```
CREATE #11, "People.TXT"
```

If you store numerical data, a good file extension would be .DAT — short for data.

When you have created a lot of files, you will appreciate the advantages of file extensions that label the contents. Also, you can produce a family of files with the same name, but with different extensions:

```
FRED TX1
FRED TX2
FRED DA1
FRED DA2
```

Reading From a Stored File

To read information from a disk file, you open a channel and name the file using the OPEN command:

```
OPEN #12, "People"
```

This allows you to read from "People" using INPUT #12. The following program will read the contents of the "People" file:

```
NEW
10 OPEN #12, "People"
20 FOR I% = 1 TO 4
30 INPUT #12, Name$ : REM Read name from file
40 PRINT Name$
50 NEXT I%
60 CLOSE #12
70 END
```

When you run this program, the contents of the file will be displayed on the screen.

```
Keats
Burns
Thomas
Joyce
```

You need to close the file after you have read the data from it, to let you use another file on that channel.

Modifying Files

Values written to a disk file are stored in sequence on the disk. When you read values from a file, they are read in the same sequence as they were written to the file. Consequently, if you are only interested in reading the last value of a file, you must read all the other values before you get to it.

Once you have created a file, written to it, and closed it, you can neither write more values to it nor remove values from it.

File modification can be done in the following manner:

- open the old file for reading
- create a new file for writing
- read all the values from the old file
- write the values that you want to keep, and/or any new values to the new file.
- close both files
- erase the old file
- rename the new file with the old file's name.

Here is a file modification program that allows you to add a name to a file. The contents of the file are organised in alphabetical order and the new name is written in the correct position:

```
300 OPEN #11, "Names"
310 CREATE #12, "Names.$$$"
320 INPUT "Enter name", Y$
330 End_File% := FALSE
340 ON EOF #11, End_File% := TRUE : GOTO 390
350 REPEAT
360   INPUT #11, X$
370   IF Y$ > X$ THEN PRINT #12, Y$ : Y$ := ""
380   PRINT #12, X$
390 UNTIL End_File% = TRUE
400 ERASE "Names"
410 RENAME "Names.$$$" TO "Names"
420 CLOSE
```

This program uses one channel (11) to read from Names, and another channel (12) to write to Names.$$$. (The extension .$$$ is often used by programs to denote a temporary file.)

Line 340 contains the ON EOF command. This is very useful in file-handling programs because it checks when the end of a file is reached during reading operations. It also controls what happens next.

Lines 400 and 410 erase the Names file and rename Names.$$$ as Names. The Names.$$$ name is, therefore, a temporary one.

Several Channels

You can use several channels at the same time for multiple file-handling. For example, you can transfer the contents of four files into one.

The following program segment calls a procedure four times to do this:

```
400 CREATE #40, "Newfile"
410 Read_File 31, 40
420 Read_File 32, 40
430 Read_File 33, 40
440 Read_File 34, 40
450 CLOSE : END
560 PROCEDURE Read_File X%, Y%
570 All_data% := FALSE
580 ON EOF #X% All_data% := TRUE : GOTO 620
590 REPEAT
600   INPUT #X%, InFo$
610   PRINT #Y%, InFo$
620 UNTIL All_data% = TRUE
640 ENDPROC
```

File-Handling Commands

RM Basic contains several commands that can be used to report on, delete, rename, and reorganise your disk files. Most are very similar to MS-DOS commands as shown below:

MS-DOS	*RM Basic*	*Action*
DIR	DIR command (some differences)	List files
	FSPACE function	Report free space
CHDIR *pathname*	CHDIR command	Change directory
CHDIR	PATH$ function	Report current directory
MKDIR	MKDIR command	Create directory
RMDIR	RMDIR command	Remove empty directory
DEL or ERASE	ERASE command	Erase file
RENAME	RENAME...TO command	Rename file
	LOOKUP	Search for file

If you are changing the directory on a different drive, the RM Basic CHDIR command selects the specified drive. A similar MS-DOS CHDIR command does not select the drive. It only changes the directory so that it will be the current one when you next access that drive.

The syntax is similar to that of the MS-DOS commands, details of which can be found in "Operating System Commands" in the Owners Handbook. The main difference is that RM Basic requires a string expression (quotes) — see the Keywords section.

Communicating With Other External Devices

You can write to and read from devices attached to the Nimbus Piconet. For details of the Piconet system and device names see "Piconet : How to use it" in the Owners Handbook.

As with files, you use CREATE to assign a channel to a device and open the channel for writing operations. For example:

```
CREATE #20, "COM1"
```

Now, you can use PRINT #20 or PUT # 20 to write to COM1.

If you want to read fom the device, close the channel and use OPEN to open it for reading activities with INPUT, GET, or GET$:

```
CLOSE #20
OPEN #20, "COM1"
```

Keywords

Introduction

This section contains descriptions of all the RM Basic *keywords*, arranged in alphabetical order. A keyword is a word that RM Basic understands.

A standard format is used for each description. Normally, there is one keyword per page, but some descriptions, such as PRINT, use more than one page.

Where keywords work in combination with others, such as REPEAT and UNTIL, they are combined with a / sign. This indicates that the keywords are separated by one or more lines of instructions.

When ... is between the keywords, it indicates a separation of one or more instructions on the same line.

At the start of each description is the keyword (or keyword combination) in capital letters. On the same line is a classification of its type:

- command
- function
- attribute
- construct
- logical operator

This is followed by a brief and simple description of what the keyword does, then sections on syntax, remarks, examples, options, and associated keywords.

Syntax

The format, punctuation, and options are shown using the following symbols:

...
indicates that more items can be used

[]
encloses optional items

/
indicates alternatives

exp, *exp1*
numeric or string expression

e, *e1* ...
numeric expressions

e$*, *e1$
string expressions

t
a numeric expression that is evaluated; a non-zero value returns TRUE and 0 returns FALSE.

var, *var1*...
numeric or string variable or array element

v, *v1* ...
numeric variable or array element

v$*, *v1$
string variable or array element

ov
object variable or array element

array, *array1*
array name with empty brackets

attrib
an attribute keyword (see SET, ASK)

name
procedure, function, or subroutine name

line
a program line number

range
a range of program line numbers

~e
screen writing area number (default if absent)

#*e*
channel number (the screen if absent)
coordinate list
either a list of pairs of expressions separated by semicolons. Each pair, separated by a comma represents the x coordinate and the y coordinate (from the current graphics axes) of a point on the screen

or a single dimension array containing a list of coordinate pairs. Element 0 contains the number of pairs, element 1 contains the first x coordinate,element 2 contains the first y coordinate, element 3 contains the second x coordinate, and so on.

All punctuation symbols (except those in the above list) must be included as shown in the syntax. Words in capital letters are keywords that must have spaces before and after them. They can be entered in upper or lowercase letters. RM Basic converts them to uppercase.

Keywords can be abbreviated by typing only the first few letters followed by a full stop (.). Priority is given to some of the the frequently used keywords so that only the first letter and a full stop is necessary. For example:

```
E. EDIT
L. LIST
I. INPUT
P. PRINT
```

Items in italics are supplied by you according to the relevant rules. A summary of rules is given below.

Graphics Syntax

When using the graphics commands AREA, CIRCLE, FLOOD, LINE, PLOT, POINTS, and SLICE, you can specify AT *e1*, *e2* after *coordinate list*. The value of *e1* is the x coordinate, and that of *e2* is the y coordinate of the point to which the first coordinate pair is moved. All other coordinates move to keep the specified shape.

You can also use SCALE *e1*[, *e2*]. This applies a scaling factor to the coordinates in the command (including the AT coordinates).

If only *e1* is specified, both x and y coordinates are scaled by the value of *e1*. If *e1* and *e2* are used, *e1* scales the x coordinates and *e2* scales the y coordinates.

The SCALE option does not scale characters drawn with PLOT (use the SIZE option in the PLOT command).

Remarks

This section describes in detail how the keyword (or keyword combination) works.

Any special symbols in the syntax are explained.

Examples

Examples show the use of the keyword in either direct or program mode. In some cases, the resultant screen output is given.

Options

Some graphics and sound commands can use options to temporarily select an attribute for the duration of the command. The relevant attribute keywords are shown.

Associated Keywords

This section indicates other keywords that have similar functions to the described keyword, or that are used in conjunction with the keyword.

Summary of Rules

Numbers

Integer
Whole number without a decimal point.
Range: –32760 to +32760

Floating-point
Number made up of an integer part and a fractional part after the decimal point. Numbers larger than 9999999 or smaller than 0.1 are output in scientific notation. Range: 1.2e–38 to 3.4e+38 and 0.

Hexadecimal
Number using the base 16 (figures 0 to 9 and letters A to F). On input, hexadecimal numbers must be preceded by an ampersand character (&). Maximum: 7FFFFFFF

Strings

A string is a sequence of characters (up to 32000) with a quotation mark (") at the start and end for input. To output a quotation mark within a string, place two quotation marks together.

Variable Names

Every variable must have a name:

- it must begin with a letter, with up to 31 characters made up of letters, numbers, and underscore characters (_).
- it cannot be a keyword, but it can contain an embedded keyword.
- it is followed by a type identifier to indicate the variable type:

$	string variable
%	integer number variable
no identifier	floating-point number variable
?	object variable

Array Names

Naming an array follows the same rules as variable names and the same type identifiers are used. An array name is followed by empty brackets. Each value in an array is an array element, identified by the number(s) in brackets. For example Demo_Data(20,5) is an array element of the array called Demo_Data(). Array elements are variables and can be used wherever variables are used.

Assignment Rules

String variables can only accept strings, but you can assign integer numbers to floating point variables and floating-point numbers to integer variables if they are in the correct range.

The number is stored in the form decreed by the variable. When a floating point number is assigned to an integer variable, the number is truncated.

In functions and procedures, you can assign a whole array using the array reference system. The assigned array must be exactly the same type as that of the array name in the function or procedure definition.

ABS Function

Calculate the absolute value of a number.

Syntax

ABS(*e*)

Remarks

When the value of *e* is either positive or zero, the function returns the value unchanged. When the value of *e* is negative, the (positive) absolute value is returned.

Examples

```
PRINT ABS(4)

 4


PRINT ABS(-27.19)

 27.19


PRINT ABS(-9 * 4)

 36
```

Associated Keywords

INT, SGN

AND Logical Operator

Logical operation on two expressions.

Syntax

e1 AND *e2*

Remarks

The AND operator gives the result of bit by bit combination (see appendix B) of two comparisons, expressions, or integers.

A common use is in control constructs such as IF...THEN...ELSE:

```
IF A = B AND C > D THEN STOP
```

AND combines the results of the A=B and C>D comparisons. The instruction following THEN is only executed when both comparisons are TRUE.

The AND operator can also be used in numerical expressions to logically combine two integer operands:

```
LET A% := 5 AND 6
```

This assigns 4 to A%.

Examples

```
Result% := Number% AND 95
```

Associated Keywords

NOT, OR, XOR

AREA Graphics Command

Draw area on the screen.

Syntax

AREA *coordinate list* [*option list*]

Remarks

The AREA command draws a shape by connecting each point from the coordinate list to the next, starting and finishing at the first one, and filling the enclosed area with colour.

Unless you specify the relevant option in the AREA command, the current settings of brush colour, filling style, and drawing mode are used (either the default settings or those set with the SET *attrib* command).

The following options, giving temporary change, can be specified after *coordinate list*:

Option	*Action*	*Syntax*
BRUSH	selects the brush colour	see BRUSH graphics attribute
STYLE	selects the filling style	see FILL STYLE graphics attribute
OVER	selects the drawing style	see OVER graphics attribute

Examples

The following instruction:

```
AREA 0, 0; 100, 0; 100, 100 BRUSH 2 OVER TRUE
```

draws a triangle in brush colour 2, using over plotting (non-XOR) drawing mode (see chapter 6). As no STYLE option is specified, the default or current setting of filling style applies.

An array is used in the next example:

```
AREA Octagon%() AT 50, 70 SCALE 2 STYLE 3 BRUSH 3
```

Options

BRUSH, OVER, [FILL] STYLE

Associated Keywords

CIRCLE, FLOOD, LINE, PLOT, POINTS, SLICE

ASC Function

Find the character code of the first character of a string.

Syntax

ASC(*e$*)

Remarks

This function returns an integer that is the ASCII code of the first character of *e$* (see "Technical Data" in the Nimbus Owners Handbook for character codes).

The CHR$ function has the opposite effect; it converts a numeric code to a character.

Examples

```
PRINT ASC("A")
 65

PRINT ASC("FRED")
 70
```

Associated Keywords

CHR$, HEX$, STR$, VAL

ASK Command

Report the current setting(s) of an attribute.

Syntax

ASK *attrib v1*, *v2* ...

Remarks

ASK returns the current setting(s) of the attribute specified by *attrib* in the variable(s) *v1*(, *v2* ...) that you supply.

attrib can be one of the following attribute keywords:

BORDER	ENVELOPE	MOUSE	POINTS STYLE
BRUSH	FILL STYLE	ORIGIN	RAD
CHAROVER	FKEY	OVER	SOUND
CHARSET	FLOOD EDGE	PATTERN	TIME
COLOUR	HOLD	PAPER	TONE
CURPOS	JOYSTICK	PEN	UNDERLINE
CURSOR	JOYSTICK2	PLOT DIRECTION	VOICE
DATE	KEYREP	PLOT FONT	WARN
DEG	LINE STYLE	PLOT SIZE	WRITING
DRAWING	MODE	POINTS SIZE	

With ASK COLOUR, ASK FKEY, and ASK PATTERN you must specify *e* before the variable(s).

With ASK COLOUR, the value of *e* indicates the colour number that you are interested in; with ASK FKEY, it indicates the function key number; with ASK PATTERN it indicates the dithered pattern.

Examples

```
ASK MODE Screen_Width%
PRINT Screen_Width%

 80
```

```
ASK WRITING W_Area%, Left%, Top%, Right%, Bottom%
PRINT W_Area%; Left%; Top%; Right%; Bottom%

 2  10  35  20
```

Associated Keywords

SET

ATN Function

Calculate the angle with the given tangent.

Syntax

ATN(*e*)

Remarks

This function returns the angle, in degrees or radians, that has the tangent value, *e*. The angle measurement unit is set with SET DEG or SET RAD.

Examples

```
10 SET DEG TRUE
20 PRINT ATN(1.557)
30 SET RAD TRUE
40 PRINT ATN(1.557)

 90
 1
```

Associated Keywords

COS, DEG, RAD, SIN, TAN

AUTO Command

Start automatic line numbering.

Syntax

AUTO [*line*][, *increment*]

Remarks

The AUTO command is used to generate line numbers automatically. If you enter AUTO on its own, line number 10 will be displayed. After you type in the instruction(s) for that line and <ENTER>, 20 will be displayed at the beginning of the next line.

The generation of line numbers (with an increment of 10) continues until you press <ENTER> with no instruction.

Use *line* when you want a different starting line number. Use *increment* to specify the increment between lines.

Before a line number is generated, any existing lines with numbers between the new line and the last auto-generated line are listed. Consequently, you can see what you are skipping over or what you are about to replace.

If you do not want to overwrite existing lines, you must break out of automatic generation.

Examples

```
AUTO 200, 5
```

Associated Keywords

EDIT, LIST

BLOCK COPY Graphics Command

Copy an area of the graphics screen.

Syntax

BLOCK COPY *e1*, *e2*; *e3*, *e4*; *coordinate list* [OVER *t*]

Remarks

This command is used to copy the area of graphics screen (indicated by the *e1*, *e2*, *e3*, and *e4* coordinates) to the destination area(s) specified in *coordinate list* (bottom left-hand corner coordinates).

Unless you specify the drawing mode with the OVER *t* option, the current setting will be used (either the default setting or that set with the SET OVER *t* command).

Examples

```
BLOCK COPY 0, 0; 100, 50; 150, 150 OVER FALSE
```

Associated Keywords

BLOCK READ, BLOCK WRITE

BLOCK READ Graphics Command

Read from an area of the graphics screen into memory.

Syntax

ov = BLOCK READ (*e1*, *e2*; *e3*, *e4*)

Remarks

This command is used to assign the graphics screen data, in the area indicated by the coordinates, into *ov*.

Examples

```
Square? = BLOCK READ (50, 50; 100, 100)
```

Associated Keywords

BLOCK COPY, BLOCK WRITE

BLOCK WRITE Graphics Command

Write to an area of graphics screen from memory.

Syntax

BLOCK WRITE *ov*, *coordinate list* [OVER *t*]

Remarks

This command is used to draw the graphics screen data assigned to *ov* (with BLOCK READ) at each of the destination area(s) specified in *coordinate list* (bottom left-hand corner coordinates).

Unless you define the drawing mode with the OVER *t* option, the current setting of drawing mode will be used.

Examples

```
BLOCK WRITE Square?, 20, 20; 100, 20; 200, 20 AT 50, 50
```

Associated Keywords

BLOCK COPY, BLOCK READ

BORDER Attribute

Select the screen border colour, or report its current setting.

Syntax

SET BORDER *e*

ASK BORDER *v*

Remarks

SET BORDER is used to specify the colour of the border area, using the colour numbers in the COLOUR keyword definition. The border colour is independent of the current settings of brush, paper, and pen colours.

The value of *e*, in the range 0 to 15, indicates what colour (colour monitors) or what shade of grey (monochrome monitors) is selected. You can use the MIX function to generate the colour number.

The default border colours are black in 40-character mode and blue in 80-character mode.

ASK BORDER assigns the current border colour to *v*.

Example

```
SET BORDER 14 : REM select yellow border
```

Associated Keywords

COLOUR, MIX, MODE

BOUNDS Function

Provide information about array dimensions.

Syntax

BOUNDS(*array*[, *e*])

Remarks

When *array* alone is present, the BOUNDS function returns the number of dimensions in the specified array.

To find the size of a dimension, the value of *e* indicates the particular one. BOUNDS will give the maximum subscript value of that dimension (the upper bound).

Examples

```
DIM A$(20, 40, 50) : PRINT BOUNDS(A$())

 3

DIM Records (37) : PRINT BOUNDS(Records, 1)

 37

10 DIM Records(7, 5)
20 FOR Dimension% = 1 TO BOUNDS(Records())
30    PRINT BOUNDS(Records(), Dimension%);
40 NEXT
RUN

 7   5
```

Associated Keywords

DIM, DEFINED, LVAR

BRUSH Graphics Attribute

Select the graphics colour for drawing and plotting on the screen, or report its current setting.

Syntax

SET BRUSH *e*

AREA/FLOOD/LINE/POINTS *coordinate list* BRUSH *e*

CIRCLE *e1*, *coordinate list* BRUSH *e*

SLICE *e1*, *e2* TO *e3*, *coordinate list* BRUSH *e*

PLOT *e$*, *coordinate list* BRUSH *e*

ASK BRUSH *v*

Remarks

SET BRUSH *e* selects the colour or grey level (brush colour) for graphics output to the screen. The BRUSH *e* option, in a graphics command, temporarily selects the brush colour for the duration of the command.

The value of *e* specifies whether the colour will be pure or dithered:

- in high resolution graphics mode, the values 0 to 3 represent the pure colour from the 80-character mode colour table; the initial settings of the table are shown below
- in low resolution graphics mode, the values 0 to 15 represent the pure colour from the 40-character mode colour table; the initial settings of the table are shown below
- 128 to 143 indicate a dithered pattern; see the PATTERN graphics attribute

Colour Table (80-character mode) *Initial settings*

Number	*Colour*	*Monochrome*	*Flashing*
0	dark blue	grey 1	No
1	dark green	grey 4	No
2	light red	grey 10	No
3	white	white	No

Colour Table (40-character mode) *Initial settings*

Number	*Colour*	*Monochrome*	*Flashing*
0	black	black	No
1	dark blue	grey 1	No
2	dark red	grey 2	No
3	purple	grey 3	No
4	dark green	grey 4	No
5	dark cyan	grey 5	No
6	brown	grey 6	No
7	light grey	grey 7	No
8	dark grey	grey 8	No
	light blue	grey 9	No
10	light red	grey 10	No
11	magenta	grey 11	No
12	light green	grey 12	No
13	cyan	grey 13	No
14	yellow	grey 14	No
15	white	white	No

The default brush colour is 15 in 40-character mode and 3 in 80-character mode.

You can change one element, several elements, or the whole of the colour table with the SET COLOUR command.

Dithered patterns do not work on all graphics commands. Where a brush number of 128 to 143 is not acceptable, 128 is subtracted giving a brush number in the range 0 to 15 (40-character mode) or 0 to 3 (80-character mode).

ASK BRUSH returns the current brush colour in *v*.

Examples

```
SET BRUSH 130

SET MODE 40
CIRCLE 20, 50, 50 BRUSH 12
```

Associated Keywords

AREA, CIRCLE, COLOUR, FLOOD, LINE, PAPER, PATTERN, PEN, PLOT, POINTS, SLICE.

BUTTONS Graphics Function

Report the current mouse or joystick button position (pressed or not pressed).

Syntax

BUTTONS(*e*)

Remarks

BUTTONS returns the current state of either the mouse button or a joystick button. The value of *e* specifies which device:

0 indicates the mouse
1 indicates joystick1
2 indicates joystick2

The value returned by BUTTONS indicates whether or not the button is pressed:

- for a mouse:
  0 = neither button is pressed
  1 = the right-hand button is pressed
  2 = the left-hand button is pressed
  3 = both buttons are pressed

- for a joystick:
  0 = the button is not pressed
  1 = the button is pressed

Examples

```
IF BUTTONS(2) = 2 THEN Menu_Display
```

Associated Keywords

JOYSTICK, JOYSTICK2, JOYX, JOYY, MOUSE, POSX, POSY

BYE Command

Close all files and leave RM Basic.

Syntax

BYE

Remarks

This command closes all files that have been in use, clears the screen, resets the colour tables, leaves RM Basic, and returns control to the operating system.

CHAROVER Attribute

Set the writing mode for text output, or report its current setting.

Syntax

SET CHAROVER *t*

ASK CHAROVER *v*

Remarks

SET CHAROVER is used to select whether or not characters will be destructively overwritten.

When the value of *t* is FALSE (0), characters issued by PRINT and PUT do not destructively overwrite (XOR mode). When the value of *t* is TRUE (–1), characters destructively overwrite. The default setting is TRUE.

In XOR mode you can write new characters over what is already written. When you repeat the writing instruction, the new characters are removed and the previous contents of the screen at that position are restored.

When you are using XOR mode, if two points intersect, the colours are mixed to form a third colour.

ASK CHAROVER returns the current writing mode setting in *v*.

Examples

```
10 SET CHAROVER FALSE
20 HOME
30 LIST
40 GOTO 10


ASK CHAROVER Which%
PRINT Which%

 0
```

Associated Keywords

OVER, PRINT, PUT

CHARSET Attribute

Select the character set for text output, or report the current setting.

Syntax

SET CHARSET *e*

ASK CHARSET *v*

Remarks

SET CHARSET is used to select one of two inbuilt character sets (see "Technical Data" in the Nimbus Owners Handbook), each one defining 256 letters, numbers, and other characters.

When the value of *e* is 1, the alternative set (RM 8-bit like) is selected. When the value of *e* is 0, the standard character set (IBM like) is selected. The default setting is 0.

ASK CHARSET returns the current setting in *v*.

Examples

```
SET CHARSET 1 : REM Select the alternative set
```

Associated Keywords

UNDERLINE

CHDIR Command

Change the current disk and file directory.

Syntax

CHDIR *e$*

Remarks

e$ is a string expression; its value represents a pathname (see "Operating System Concepts" in the Nimbus Owners Handbook).

CHDIR changes to the directory specified by *e$*. If a disk drive letter is also specified, the current drive is changed to it.

Unlike the MS-DOS command of this name, CHDIR cannot be used on its own to display the current directory. This can be found using the PATH$ function.

Examples

```
CHDIR "Steve"
```

```
A$ := ".."
CHDIR A$
```

```
CHDIR "A:\"
```

Associated Keywords

DIR, ERASE, FSPACE, LOOKUP, MKDIR, PATH$, RENAME...TO, RMDIR

CHR$ Function

Convert an ASCII code to its character equivalent.

Syntax

CHR$(*e*)

Remarks

e is a numeric expression with a value in the range 0 to 255.

This function returns a string containing the character with the code given by the integer part of *e* (see "Technical Data" in the Nimbus Owners Handbook for character codes).

The PRINT CHR$(*e*); instruction is equivalent to the PUT *e* instruction.

The ASC Function converts a character back to its decimal code.

Examples

```
PRINT CHR$(100)

d

10 REM Program to print characters with codes 32 to 255
20 WIDTH 25
30 FOR I% = 32 TO 255
40   PRINT CHR$(I%);
50 NEXT I%
```

Associated Keywords

ASC, STR$

CIRCLE Graphics Command

Draw circle(s), on the screen.

Syntax

CIRCLE *e*, *coordinate list* [*option list*]

Remarks

CIRCLE is used to draw one or more filled circles with a radius given by the value of *e* (y axis units). A circle is drawn around each coordinate in *coordinate list.*

Unless you specify the relevant option in the CIRCLE command, the current settings of brush colour, filling style, and drawing mode are used (either the default settings or those set with the SET *attrib* command).

The following options, giving temporary change, can be specified after *coordinate list*:

Option	*Action*	*Syntax*
BRUSH	selects the brush colour	see BRUSH graphics attribute
STYLE	selects the filling style	see FILL STYLE graphics attribute
OVER	selects the drawing style	see OVER graphics attribute

Examples

```
CIRCLE 50, 75, 100; 150, 130 BRUSH 2 STYLE 2, 5, -1
```

Options

BRUSH, OVER, [FILL] STYLE

Associated Keywords

AREA, FLOOD, LINE, PLOT, POINTS, SLICE

CLEAR Command

Set up string space or specify the number of file channels.

Syntax

CLEAR [*e1*]

Remarks

The CLEAR command, without *e1*, clears memory of all variables, arrays, and data files. Space is reserved to store 200 characters in string variables (200 in total, not 200 in each variable).

CLEAR *e1* produces additional action. Space is reserved to store at least *e1* characters in string variables. RM Basic automatically enlarges the space if you exceed it, but programs are more efficient if this does not need to happen.

CLEAR is an initializing command and it is usually put at the start of a program. You can use the FREE function to find how much unused space is left for strings.

Examples

```
CLEAR 500
```

Associated Keywords

FREE, MEM, NEW, RUN

CLG Graphics Command

Clear the current graphics drawing area of the screen.

Syntax

CLG

Remarks

The CLG command is used to clear the current drawing area. The area is filled with paper colour 0 which, if you have not changed it, is black in 40 character mode and dark blue in 80 character mode.

Associated Keywords

CLS, DRAWING, MODE, PAPER

CLL Command

Delete part or all the contents of a line of text.

Syntax

CLL [~*e1*][, *e2*]

Remarks

When the CLL command is executed, some or all of the contents of the line (of the specified writing area) containing the cursor are deleted. The value of *e2* specifies what characters will be cleared:

- 0, or no value, indicates all characters of the row
- greater than 0 indicates all characters to the right of the cursor position (including the character at the cursor position)
- less than 0 indicates all characters to the left of the cursor position (including the character at the cursor position)

The cursor position is not changed.

Examples

```
10 CLS : SET CURPOS 30, 25
20 PRINT "Bottom of the screen"
30 FOR I = 1 TO 2000 : NEXT
40 CLL 0 : PRINT
```

Associated Keywords

CLS, [CUR]MOVE, CURPOS, HOME, WRITING

CLOSE Command

Close one or all files.

Syntax

CLOSE [*#e*]

Remarks

e is a numeric expression with a value in the range 11 to 127. This value is the channel number that was specified either in the OPEN command or the CREATE command.

The CLOSE command closes the specified channel. When the command is used with no channel number, all open channels are closed.

Once a channel has been closed, you cannot perform input or output operations on a file on that channel.

Unless you use CLOSE, you cannot be sure that the contents of files on open channels will not be lost, and you may reach the limit imposed by the operating system.

Examples

```
CLOSE #12
```

Associated Keywords

CHDIR, CREATE, ERASE, FSPACE, LOOKUP, MKDIR, OPEN, PATH$, RENAME...TO, RMDIR

CLS Command

Delete part or all of the contents of a writing area.

Syntax

CLS [~*e1*][, *e2*]

Remarks

The CLS command is used to clear part or all of the specified writing area. The value of *e2* specifies what characters will be cleared:

- 0, or no value, indicates the whole of the writing area
- greater than 0 indicates all characters to the right of the cursor (including the character at the cursor position) on the cursor row, and all characters on the row(s) below the cursor row
- less than 0 indicates all characters to the left of the cursor (including the character at the cursor position) on the cursor row, and all characters on the row(s) above the cursor row

The cursor position is not changed.

Examples

```
10 FOR I = 1 TO 20
20 PRINT "Clearing the screen behind me"
30 CLS -1
40 NEXT
```

Associated Keywords

CLL, [CUR]MOVE, CURPOS, HOME, WRITING

COLOUR Attribute

Set an element of the colour table, or report its current setting.

Syntax

SET COLOUR *e1* TO *e2*[, *e3*, *e4*]

ASK COLOUR *e1*, *v1*[, *v2*, *v3*]

Remarks

SET COLOUR is used to redefine an element of the colour table used with SET BRUSH, the BRUSH option, SET PAPER, and SET PEN.

The value of *e1* indicates the number in the colour table that you want to redefine. The new colour (or shade of grey) to be assigned to the number is chosen from the list below and indicated by the value of *e2*.

In the list on the next page, G stands for grey. On a monochrome monitor, greyness lightens as the number increases.

If you want two flashing (alternating) colours rather than one colour, then the value of *e3* specifies the flashing rate:

0 indicates no flash
1 indicates slow flash
2 indicates fast flash

The value of *e4* specifies the second colour from the list below.

Instead of *e2* and *e4*, you can specify the colour in terms of gun settings with the MIX function.

Value of e2	*Resultant Colour* *Colour monitor*	*Mono monitor*
0	Black	Black
1	Dark blue	G1
2	Dark red	G2
3	Purple	G3
4	Dark green	G4
5	Dark cyan	G5
6	Brown	G6
7	Light grey	G7
8	Dark grey	G8
9	Light blue	G9
10	Light red	G10
11	Magenta	G11
12	Light green	G12
13	Cyan	G13
14	Yellow	G14
15	White	White

ASK COLOUR returns the current colour of the specified number of the colour table in *v1*, *v2* and *v3*.

Examples

The following program redefines the 80-character mode colour table:

```
10 SET MODE 80
20 SET COLOUR 0, 14
30 SET COLOUR 1, 11
40 SET COLOUR 2, MIX (0, 0, 1, 1)
50 SET COLOUR 3, 5, 1, 12
```

The 80-character mode table is now:

Number	*Colour Colour monitor*	*Monochrome monitor*	*Flashing*
0	Yellow	Grey 14	No
1	Magenta	Grey 11	No
2	Purple	Grey 3	No
3	Dark green	Grey 5	Yes. Second colour is light green (grey 12)

Associated Keywords

BORDER, BRUSH, MIX, PAPER, PEN

CONTINUE Command

Resume program execution at the next instruction.

Syntax

CONTINUE

Remarks

When a program has been interrupted, for example by typing <BREAK> or by execution of the STOP command, you can use the CONTINUE command to continue execution. The program starts running again from the next instruction.

CONTINUE can be used with STOP to help find errors in your program. STOP commands are placed at intervals throughout the program and each time execution is stopped, you can examine or change the value of variables before you continue execution with CONTINUE.

Associated Keywords

RESUME, STOP

COS Function

Calculate the cosine of an angle.

Syntax

COS(*e*)

Remarks

This function returns the cosine of the angle with value, *e*, expressed in degrees or radians depending on the current angle measurement unit. This is set with SET DEG or SET RAD.

Examples

```
10 SET RAD TRUE
20 REM Draw a cosine curve
30 SET MODE 40, 200, 200
40 FOR X% = 0 To 200 STEP 5
50   Angle := X% / 200 * 5
60   Y := 100 * (COS(Angle) + 1)
70   POINTS X%, Y STYLE 2
80 NEXT X%
```

Associated Keywords

ATN, DEG, RAD, SIN, TAN

CREATE Command

Create a file to receive output.

Syntax

CREATE *#e, e$*

Remarks

e is a numeric expression with a value in the range 11 to 127. This value is the channel number that you must subsequently specify in commands that write to *e$*.

e$ is a string expression that represents the file specification.

The CREATE command must be used to open a file before you can write to it with PRINT or PUT.

Examples

```
CREATE #12, "DATA2"
```

Associated Keywords

CHDIR, CLOSE, DIR, ERASE, FSPACE, LOOKUP, MKDIR, OPEN, PATH$, PRINT, PUT, RENAME...TO, RMDIR

[CUR]MOVE Command

Move the screen cursor with respect to its current position.

Syntax

[CUR]MOVE *e1*, *e2*

Remarks

The cursor is moved from its current position by *e1* columns and *e2* rows within the bounds of the current writing area.

When the value of *e1* is positive, the cursor is moved *e1* columns to the right. A negative value of *e1* causes the cursor to move *e1* columns to the left.

When the value of *e2* is positive, the cursor is moved *e2* rows down. A negative value causes upward movement of *e2* rows.

Examples

```
MOVE -10, 6 : REM Move the cursor 10 left and 6 down.
```

Associated Keywords

CLL, CLS, CURPOS, HOME, WRITING

CURPOS Attribute

Move the screen cursor, or report its current position.

Syntax

SET CURPOS *e1*, *e2*

ASK CURPOS *v1*, *v2*

Remarks

SET CURPOS moves the cursor to the column indicated by the value of *e1* and to the row indicated by the value of *e2*.

When the values of both *e1* and *e2* are outside the current writing area, the cursor moves to the home position of the writing area.

If the value of *e1* is within the area, but the value of *e2* is outside, the cursor moves to column *e1* and to the top row of the writing area.

If the value of *e2* is within the area, but the value of *e1* is outside, the cursor moves to row *e2* and to the left-most column of the writing area.

ASK CURPOS assigns the current cursor position to *v1* and *v2*.

Examples

```
SET CURPOS 1, 25 : REM Move to the bottom left of screen.
```

Associated Keywords

CLL, CLS, [CUR]MOVE, HOME, WRITING

CURSOR Attribute

Define screen cursor characteristics when using GET or GET$.

Syntax

SET CURSOR *e1*[, *e2*[, *e3*[, *e4*]]]

ASK CURSOR *v1*[, *v2*[, *v3*[, *v4*]]]

Remarks

RM Basic executes programs without displaying a cursor on the screen. You only see a cursor when RM Basic is expecting input from the keyboard. When you use normal line entry, the EDIT command, or the INPUT command, the editor is in operation, so underline and block cursors are displayed (see chapter 2).

A cursor is also displayed when the GET or GET$ functions are used to obtain keyboard input. The default cursor is the one currently in use with the operating system, and you can use SET CURSOR to change it:

- the value of *e1* indicates whether or not the cursor is displayed: 0 indicates not displayed; greater than 0 indicates displayed and not flashing; less than 0 indicates displayed and flashing.

- the value of *e2* represents the ASCII code of the character to be used as cursor, chosen from one of the character sets (see "Technical Data" in the Nimbus Owners Handbook).

- the value of *e3* indicates which character set: 0 indicates the standard set; 1 indicates the alternative character set.

- The value of *e4* indicates if the cursor character is underlined (0) or not (1).

ASK CURSOR assigns the current GET and GET$ cursor characteristics to *v1*, *v2*, *v3*, and *v4*.

Examples

```
SET CURSOR 1, 84, 0, 1
```

This selects the T character, underlined, from the standard character set as cursor for GET or GET$ use. The cursor is not flashing.

Associated Keywords

GET, GET$

DATA Command

Specify numeric or string constants that will be assigned to variables with the READ statement.

Syntax

DATA *c1*[, *c2*...]

Remarks

c1, *c2* ... are numeric or string constants.

DATA instructions can be placed anywhere in your program. There can be several instructions, but items are read in order.

Take care if string constants contain spaces, commas, or colons. Such data is best enclosed by quotation marks. For example, the instruction:

```
DATA "Tinker, Tailor"
```

is taken as a single string with a space before Tailor. However, the instruction:

```
DATA Tinker, Tailor
```

is taken as two strings. The first is Tinker and the second is Tailor with no space before it. If you put spaces after Tinker or Tailor, they will be included:

```
DATA Tinker ,Tailor
```

Tinker will have a space following it.

Examples

```
120 DATA 5, 4, 3
130 DATA Blue, Red, "YELLOW, GREEN"
```

Associated Keywords

READ, RESTORE

DATE Attribute

Update or provide the operating system date.

Syntax

SET DATE *e1*, *e2*[, *e3*]

ASK DATE *v1*, *v2*[, *v3*]

Remarks

SET DATE updates the operating system date. The values *e1*, *e2* and *e3* specify day, month, and year. If the third argument is omitted, the year is not changed.

ASK DATE returns the current operating system date in *v1*, *v2*, and *v3*.

Examples

```
SET DATE 13, 3, 86

ASK DATE Day%, Month%, Year%
PRINT Day%; Month%; Year%

 13  3  1986
```

Associated Keywords

DATE$, TIME, TIME$

DATE$ Function

Report the operating system date.

Syntax

DATE$

Remarks

The date is returned in the form day, date, month, year.

Examples

```
PRINT DATE$

Fri 22 Feb 1985
```

Associated Keywords

DATE, TIME, TIME$

DEFINED Function

Determine whether a variable has an assigned value, or whether an array has been dimensioned.

Syntax

DEFINED(*var*)

Remarks

The DEFINED function returns TRUE if *var* has been assigned or dimensioned, and FALSE if not. DEFINED is useful for finding how many parameters have been passed to a procedure.

Examples

```
10 REM Program using a procedure to plot information
on the screen.
20 Plot_Strings "Hello", 100, 50, 100, 70
30 Plot_Strings "Goodbye", 110, 60
40 Plot_Strings
50 PROCEDURE Plot_Strings Chars$, X1, Y1, X2, Y2
60 IF DEFINED(X1) AND DEFINED(Y1) THEN PLOT X1, Y1; Chars$
70 IF DEFINED(X2) AND DEFINED(Y2) THEN PLOT X2, Y2; Chars$
80 ENDPROC
```

Associated Keywords

BOUNDS, LVAR, PROCS, TRACE

DEG Attribute

Set the angle measurement unit, or report its current setting.

Syntax

SET DEG *t*
ASK DEG *v*

Remarks

Unless the value of *t* is FALSE (0), SET DEG sets angle measurement in trigonometrical functions to degrees. If the value of *t* is FALSE, the setting is radians.

ASK DEG assigns to *v* either TRUE when the angle measurement unit is degrees, or FALSE for radians.

Examples

```
10 PROCEDURE Sine_Curve
20 REM Change to degrees for curve, then return to
previous setting.
30 ASK DEG Setting%
40 SET DEG TRUE
50 SET MODE 80, 360, 200
60 FOR Angle% = 0 TO 360
70   Y := 100 * (SIN(Angle%) + 1)
80   POINTS Angle%, Y
90 NEXT Angle%
100 SET DEG Setting%
110 ENDPROC
```

Associated Keywords

ATN, COS, RAD, SIN, TAN

DELETE Command

Delete program lines.

Syntax

DELETE *range*

Remarks

This command deletes the specified line or range of lines from the program:

- DELETE *line1*
  Deletes the line with that line number
- DELETE *line1* TO
  Deletes all lines from the line with that number to the end of the program.
- DELETE *line1* TO *line2*
  Deletes these line numbers and all between them.
- DELETE TO *line2*
  Deletes all lines up to and including that line number.

The DELETE command on its own has no effect.

Examples

```
DELETE 30 TO 80
```

Associated Keywords

EDIT, LIST, NEW, RENUMBER

DIM Command

Declare array(s), define array size(s), and initialize storage space.

Syntax

DIM *array1*(*e1*[, *e2*...])[, *array2*...]

Remarks

Arrays (numeric and string) must be dimensioned with the DIM command before RM Basic refers to them. More than one array can be declared in the instruction.

The number of demensions is defined by the number of numeric expressions (*e1*, *e2* ...). In each dimension, the maximum subscript value is given by the value of the corresponding expression, giving elements 0 to the value of *e*.

The DIM command assigns an initial value to each element of an array:

- zero in numeric arrays
- an empty string in string arrays.

Examples

```
DIM Numbers(10, 20, 20), A$(25)
```

Associated Keywords

BOUNDS, CLEAR, DEFINED, GLOBAL

DIR Command

List the files in a directory.

Syntax

DIR [~*e*/#*e*,] [*e$*]

Remarks

e$ is a string expression; its value represents the pathname (see "Operating System Concepts" in the Nimbus Owners Handbook).

DIR lists the files in the specified directory to the specified writing area or channel. The command is similar to the MS-DOS command of the same name. The differences are:

- DIR on its own lists all *.BAS files in the current directory on the correct drive.
- DIR "filename" lists all filename.BAS files
- DIR "filename." lists all specified files with no extension

Examples

```
DIR "B:\Fred"
```

Associated Keywords

CHDIR, ERASE, FSPACE, LOOKUP, MKDIR, PATH$, RENAME...TO, RMDIR

DRAWING Graphics Attribute

Select or define a screen drawing area, or report the currently selected drawing area.

Syntax

SET DRAWING *e1*[TO *e2*, *e3*; *e4*, *e5*]

ASK DRAWING *v1*[, *v2*, *v3*, *v4*, *v5*]

Remarks

A drawing area is a square or rectangular area of the screen in which graphical output is displayed.

You can select up to 9 drawing areas as well as full screen with SET DRAWING and a single argument, *e1*. Its value indicates which area: 0 indicates full screen; 1 to 9 indicate a defined drawing area. The default setting is 0.

You use SET DRAWING *e1* with TO *e2*, *e3*, *e4*, and *e5* to define a drawing area:

e1	area number (1 to 9)
e2	x coordinate of the bottom left corner
e3	y coordinate of the bottom left corner
e4	x coordinate of the top right corner
e5	y coordinate of the top right corner

When undefined, drawing screen areas 1 to 9 are full screen areas.

ASK DRAWING assigns the current drawing area to *v1* and when you specify *v2* to *v5*, the coordinates of the bottom, left and top, right corners are assigned to these variables.

Examples

```
SET DRAWING 4 : REM Select drawing area 4.

SET DRAWING 7 TO 0, 0; 100 100 : REM Define drawing
area 7.
```

Associated Keywords

CLG, MODE, ORIGIN, WRITING

EDIT Command

Recall a program line for editing.

Syntax

EDIT [*line*]

Remarks

The EDIT command displays the specified program line on the screen, with the cursor positioned at the start of the line. If a line number is not specified, the most recent line containing an error (if there is one) is displayed.

A displayed line can be modified as described in chapter 2, using the numeric keypad for cursor movement (insert mode) and for deletion (delete mode).

Examples

```
EDIT 120
```

Associated Keywords

DELETE, LIST, RENUMBER

END Command

End program execution.

Syntax

END

Remarks

When an END command is executed, the program stops running and the screen prompt is displayed, allowing entry of further commands or program instructions. It completes the running of the program, and you cannot use CONTINUE to start it again.

You can have more than one END command in a program.

Associated Keywords

STOP

ENVELOPE Sound Attribute

Select or define a sound envelope, or report the currently selected envelope.

Syntax

SET ENVELOPE *e1*[TO *e2*, *e3*; *e4*, *e5*; *e6 e7*; *e8*]

NOTE *e9*[TO *e10*], ENVELOPE *e*

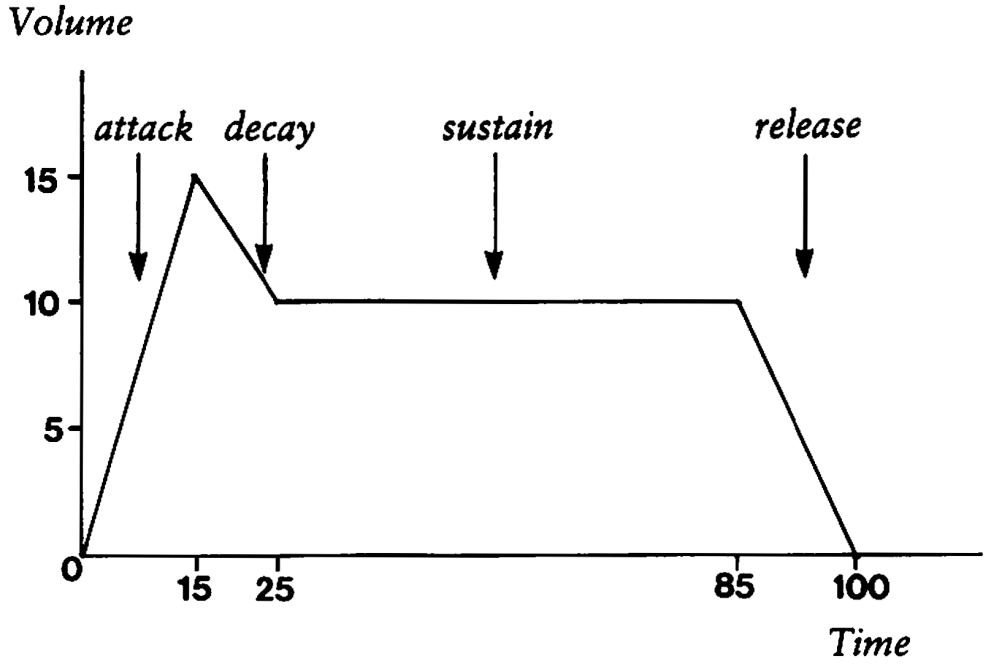
ASK ENVELOPE *v1*[*v2*, *v3*, *v4*, *v5*, *v6*, *v7*, *v8*]

Remarks

The SET ENVELOPE *e1* command selects one of ten volume envelopes for note production. The use of ENVELOPE *e*, in a NOTE command, temporarily selects an envelope for the duration of the command. The value of *e1* can be in the range 0 to 9. The default setting is envelope 0.

An envelope defines how the volume of a note varies as the note is sounding. All envelopes are initialized with constant maximum volume and with durations that range in 5 cetisecond increments from 5 (envelope 0) to 50 (envelope 9).

You can define envelopes 1 to 9 with the SET ENVELOPE *e1* TO command. This is done by specifying the four sections of the volume-time curve.



In the attack section, the volume rises in attack time, *e2*, to attack volume, *e3*. Then, the volume drops in the decay time, *e4*, to the decay volume, *e5*.

During the sustain time, *e6*, the volume changes to (or remains at) the sustain volume, *e7*. In the release section, the volume falls to zero in release time, *e8*.

The time arguments are in centiseconds, and the volume arguments can range from 0 (no sound) to 15 (maximum sound).

The release time is ignored if further notes are waiting in the voice queue.

ASK ENVELOPE returns the currently selected envelope in *v1* and details of the envelope definitions in the variables, *v2* to *v8*, when specified.

Examples

```
NOTE 148 TO 160, ENVELOPE 4 VOICE 2
```

```
SET ENVELOPE 2 TO 20, 15; 10, 12; 20, 12; 50
SET ENVELOPE 2
NOTE 200
```

Associated Keywords

HOLD, NOISE, NOTE, PITCH,
QUEUE, SOUND, TONE, VOICE

ERASE Command

Erase a single file stored on disk.

Syntax

ERASE *e$*

Remarks

e$ is a string expression; its value represents the file specification.

This command erases the specified file. Asterisk (*) or question mark (?) characters are not allowed in the specification to erase a group of files.

Examples

```
ERASE "Program1"
ERASE "\Myprogs\Game"
```

Associated Keywords

CHDIR, DIR, FSPACE, LOOKUP, MKDIR, PATH$, RENAME...TO, RMDIR

ERL Function

Provide an indication of where an error has occurred.

Syntax

ERL

Remarks

Whenever an error is detected, the current line number is transferred into ERL.

Examples

```
IF ERL = 200 THEN LOADGO "Menu"
```

Associated Keywords

CONTINUE, ERR, ERR$, ON ERROR, RESUME

ERR Function

Provide the error code of the last error.

Syntax

ERR

Remarks

Whenever an error occurs, its error code is transferred into ERR.

Examples

```
IF ERR = 20 THEN X := 0
```

Associated Keywords

CONTINUE, ERL, ERR$, ON ERROR, RESUME

ERR$ Function

Provide the error message of the last error.

Syntax

ERR$

Remarks

Whenever an error occurs, its error message is transferred into ERR$.

Examples

```
IF ERR <> 10 THEN PRINT ERR$; " At line"; ERL
```

Associated Keywords

CONTINUE, ERL, ERR, ON ERROR, RESUME

EXP Function

Calculate the exponential function, e^x

Syntax

EXP(*e1*)

Remarks

EXP returns the constant, e (about 2.718282), raised to the power of *e1*; EXP gives the inverse of LN.

Examples

```
PRINT EXP(1)

 2.718282


PRINT EXP(3.4)

 29.964103
```

Associated Keywords

LOG, LN

FALSE Function

It has a value of zero.

Syntax

FALSE

Remarks

Comparison operations return either FALSE or TRUE

Examples

```
IF X > 10 THEN Flag := FALSE
```

Associated Keywords

TRUE

FKEY Attribute

Define the function of the keys F1 to F10, or report their current function.

Syntax

SET FKEY *e1* TO *string list*

ASK FKEY *e2*, *v$*

Remarks

SET FKEY is used to define the function of the function key (on the left side of the keyboard) indicated by the value of *e1*, in the range 1 to 10.

string list is a list of numeric expressions (with values in the range 0 to 255), or string expressions, or both to be assigned to the specified key. Each expression must be separated by a comma. This is the same syntax as that of the PUT command.

ASK FKEY returns the string that would be given by pressing the specified key. If the key has not been redefined, the default string value is returned (see "Technical Data" in the Owners Handbook).

Examples

```
REM Assign PROCEDURE to F1
SET FKEY 1 TO "PROCEDURE"

REM ASSIGN RUN <ENTER> TO F2
SET FKEY 2 TO "RUN", 13
```

FLOOD Graphics Command

Fill an area of the graphics screen.

Syntax

FLOOD *coordinate list* [*option list*]

Remarks

This command is used to fill the area (or areas) of the screen containing the specified point (or points) in *coordinate list*, continuing in all directions until a boundary or drawing area edge is reached.

Unless you specify the relevant option in the FLOOD command, the current settings of brush colour, filling style, drawing mode, and boundary colour are used (either the default settings or those set with the SET *attrib* command).

The following options, giving temporary change, can be specified after *coordinate list*:

Option	*Action*	*Syntax*
BRUSH	selects the brush colour	see BRUSH graphics attribute
STYLE	selects the filling style	see FILL STYLE graphics attribute
OVER	selects the drawing style	see OVER graphics attribute
EDGE	selects the boundary colour	see FLOOD EDGE graphics attribute

A hollow filling style cannot be selected.

Examples

```
FLOOD 25, 50 BRUSH 4 EDGE 0
FLOOD 100, 100; 200, 100
```

Options

BRUSH, [FLOOD] EDGE, OVER, [FILL] STYLE

Associated Keywords

AREA, CIRCLE, SLICE

[FLOOD] EDGE Graphics Attribute

Set the boundary condition for flood filling an area of the graphics screen, or report its current setting.

Syntax

SET FLOOD EDGE *e1*[, *e2*]

FLOOD *coordinate list* EDGE *e1*[, *e2*]

ASK FLOOD EDGE *v1*[, *v2*]

Remarks

The SET FLOOD EDGE command specifies what the boundary colour is for flood filling. The EDGE option, in a FLOOD command, temporarily selects the boundary colour for the duration of the command.

When the value of *e1* is 0 (FALSE), any colour other than the filling colour will be the boundary. When the value of *e1* is 1, *e2* must be specified. Its value indicates the boundary colour.

ASK FLOOD EDGE assigns the current boundary condition to *v1* and *v2*.

Examples

```
SET FLOOD EDGE 0


FLOOD 10, 10; 200, 120 EDGE 1, 3
```

Associated Keywords

FLOOD

FLUSH Command

Clears the keyboard buffer.

Syntax

FLUSH

Remarks

When you type at the keyboard, the codes generated are stored in a buffer until RM Basic is ready for them.

When the FLUSH command is executed, the buffer is emptied so that the next key typed is immediately read.

Associated Keywords

GET, GET$, KEYREP, STOP

FOR/NEXT Loop Construct

Repeat a series of instructions, altering a control variable each time through the loop.

Syntax

FOR *v* = *e1* TO *e2* [STEP *e3*]
:
Instructions
:
NEXT [*v*]

Remarks

The FOR/NEXT construct operates in the following sequence:

1. The FOR instruction calculates values of *e1*, *e2*, and *e3*. If omitted, *e3* is assumed to be 1. The value of *e3* cannot be zero.
2. The value of *e1* is given to *v*.
3. The instructions following the FOR instruction are executed.
4. When the NEXT instruction is reached, the calculated value of *e3* is added to *v*.
5. Steps 3 and 4 are repeated until the value of *v* is greater than that of *e2* or, if the value of *e3* is negative, until the value of *v* is less than that of *e2*.

Loops are executed more quickly if the control variable is an integer.

The NEXT instruction may be labelled with the name of a control variable. If so, step 5 passes control back to the instruction following the most recent FOR instruction using that variable.

One FOR/NEXT construct can be placed inside another. If the outside construct loops m times and the inside one loops n times, the enclosed instructions are repeated m*n times.

To stop looping prematurely, assign to *v*, within the loop, a value that is outside the specified range.

Examples

```
10 FOR Greeting := 1 TO 3
20   PRINT "Hello"
30 NEXT Greeting

Hello
Hello
Hello
```

```
200 FOR I% := 1 TO 10
210   FOR J% := 1 TO 10
220     READ Info(I%, J%)
230   NEXT J%
240 NEXT I%
```

```
400 FOR I% := 1 TO 10
410   PRINT Message$(I%)
420   IF Message$(I%) = "—1" THEN I% = 99
430 NEXT I%
```

Associated Keywords

REPEAT/UNTIL

FREE Function

Find the space currently available for strings.

Syntax

FREE

Remarks

The FREE function returns the number of characters still available for strings, having compressed the strings already in memory into one end of string space.

Examples

```
PRINT FREE

 220
```

Associated Keywords

CLEAR, MEM, NEW, RUN

FSPACE Function

Provide the amount of free disk space.

Syntax

FSPACE(*e$*)

Remarks

This function returns the amount, in bytes, of free space on the disk in the drive specified by *e$*.

Examples

```
PRINT FSPACE("B:")

 30550
```

Associated Keywords

CHDIR, DIR, ERASE, LOOKUP, MKDIR, PATH%, RENAME...TO, RMDIR

FUNCTION/ENDFUN Construct

Define a function.

Defining Syntax

FUNCTION *name*([*var1/array1*[, *var2/array2*...]])
:
:
Instruction(s)
:
:
ENDFUN

Calling Syntax

name([*exp1/array1*[, *exp2/array2*...]])

Remarks

To declare a function, start with the FUNCTION header, defining the name of the function and its parameters.

The action of the function is defined by the instruction(s) on the following line(s), down to the next ENDFUN command. At least one of these instructions must contain the RESULT command, indicating what value will be returned.

To call the function, simply use *name* in the same way that you use a standard RM Basic function. The function will be executed and its value will be returned to the expression from which it was called when a RESULT command is executed.

You can define function parameters (*var1*, *var2*...) in the FUNCTION header. These are variables or array elements, each one accepting a value from the program to be used in the function.

You can specify array names (*array1*, *array2*...) in the header. Using the array reference system (see chapter 5), arrays from the main program can be assigned to the function.

Function declarations are usually placed at the end of a program.

Examples

```
200 REM A function that accepts four numbers and returns
their sum
210 FUNCTION Add(A, B, C, D)
220 Sum := A + B + C + D
230 RESULT Sum
240 ENDFUN
```

Associated Keywords

GLOBAL, PROCEDURE/ENDPROC, PROCS, RESULT

GET Command

Read the code of a character from the specified input channel.

Syntax

GET(*#e1*)

GET([*e2*])

Remarks

GET(*#e1*) returns the numeric code of a character (see "Technical Data" in the Nimbus Owners Handbook) from the channel specified by *e1*.

GET([*e2*]) returns the numeric code of a character entered at the keyboard.

In the case of function keys, negative values are returned.

When *e2* is specified, its value indicates the waiting time:

- 0 defines that a code is returned, if present, with no wait. When there is no character, 0 is returned.
- a positive number defines the time, in centiseconds (hundredths of a second), to wait for a character to be entered.
- –1 defines a wait of whatever waiting time was not used by the last GET instruction.
- –2 returns the waiting time that was not used by the last GET instruction.

When *e2* is not specified, GET waits indefinitely. While GET is waiting for a character from the keyboard, a cursor is displayed on the screen. The form of this cursor can be altered using SET CURSOR.

Examples

```
10 DIM X%(4), Y%(4)
20 SET MODE 40, 100, 1000
30 REPEAT : REM Draw areas until a key is pressed.
40   FOR I% = 0 TO 4
50     X%(I%) = RND(99)
60     Y%(I%) = RND(999)
70   NEXT I%
80   AREA X%(0), Y%(0); X%(1), Y%(1); X%(2), Y%(2);
X%(3), Y%(3); X%(4), Y%(4) STYLE 3 BRUSH 3
90 UNTIL GET(1)
```

Associated Keywords

CURSOR, GET$, KEYREP

GET$ Command

Read a character from a specified input channel.

Syntax

GET$(*#e1*)

GET$([*e2*])

Remarks

GET$(*#e1*) reads a character from the channel specified by the value of *e1*. GET$([*e2*]) reads a character entered at the keyboard. In the case of keys with extended keycodes, such as the function keys, two characters are returned.

The value of *e2*, if specified, indicates the waiting time:

- 0 defines that a character is returned, if present, with no wait. When there is no character, 0 is returned
- a positive number defines the time, in centiseconds (hundredths of a second), to wait for a character

When *e2* is not specified, GET$ waits indefinitely.

As with GET, you can alter the displayed cursor with SET CURSOR.

Examples

```
200 PRINT "Do you want another go? Enter Y or N :"
210 Answer$ := GET$()
220 IF Answer$ == "Y" THEN Game2 ELSE END
```

Associated Keywords

CURSOR, GET, KEYREP

GLOBAL Command

Create, and make accessible, global variables and arrays.

Syntax

GLOBAL *var1*[, *var2* ...]

Remarks

The GLOBAL command makes the specified global variable(s) and array(s) accessible to the program block (a procedure declaration, function declaration, or the main program) in which the command is placed.

When the declared variable does not exist, the GLOBAL command creates it and assigns either zero or an empty string to it. When an array is declared, it is not created until it is dimensioned with the DIM command.

A GLOBAL declaration should be placed at the start of the block in which the specified variables, and/or arrays, are to be used.

Examples

```
GLOBAL Total, Title$, A%, Jones()
```

Associated Keywords

FUNCTION/ENDFUN, LVAR GLOBAL, PROCEDURE/ENDPROC

GOSUB/SUBROUTINE Command/Label

Jump to a specified subroutine.

Syntax

GOSUB *line*
or
GOSUB *name*

Remarks

The GOSUB command causes program execution to jump to the line specified by *line* or *name*. Instructions are executed until a RETURN command is encountered. Then execution is transferred back to the instruction following GOSUB.

SUBROUTINE is used to label the start of a subroutine with *name*.

Examples

```
GOSUB 510


100 GOSUB Printout
:
:
500 SUBROUTINE Printout
510 FOR I% = 1 To 100
520   PRINT Name$(I%), Address$(I%)
530 NEXT I%
540 RETURN
```

Associated Keywords

GOTO, ON...GOSUB, RETURN

GOTO Command

Jump out of the normal sequence of program instructions.

Syntax:

GOTO *line*

Remarks

The GOTO command causes program execution to jump to the line specified by *line*. This allows you to jump forward, over lines, to a later part of the program, or you can jump back to an earlier line and repeat instructions.

Be very careful that you do not jump into or out of control constructs such as REPEAT/UNTIL and FOR/NEXT loops, functions, and procedures. RM Basic will not be aware that you have jumped, so it cannot warn you.

Examples

```
GOTO 400
```

Associated Keywords

GOSUB, ON...GOSUB, ON...GOTO

HEX$ Function

Produce a string that is the hexadecimal representation of a number.

Syntax

HEX$(*e*)

Remarks

This function returns a string of digits that represent the unsigned hexadecimal form of the integer part of *e*.

Examples

```
PRINT HEX$(45567)

B1FF
```

Associated Keywords

ASC, CHR$, STR$, VAL

HOLD — Sound Attribute

Select whether or not a sound will wait to be released, or report the current setting.

Syntax

SET HOLD *t*

NOTE *e1*[TO *e2*][, *e3*[, *e4*]/ENVELOPE *e5*] HOLD *t*

ASK HOLD *v*

Remarks

The SET HOLD *t* command selects whether or not a note from a NOTE command is held until released by a NOISE command. The HOLD *t* option, in a NOTE command, selects held or released sound for that note.

When *t* is FALSE, sounds are played as soon as a NOTE command is executed or as soon as they reach the start of the queue for their voice. The default setting is FALSE. When *t* is TRUE, you can use the NOISE command to control when notes are played, and to synchronize sound from more than one voice.

ASK HOLD returns the current setting in *v*.

Examples

```
SET HOLD TRUE
NOTE 100, 50: NOTE 200, 50: NOTE 300, 50
NOISE 1, 200, 200
```

Associated Keywords

ENVELOPE, NOISE, NOTE, PITCH, QUEUE, SOUND, TONE, VOICE

HOME Command

Move the screen cursor to the top left corner of the current writing area of the screen.

Syntax

HOME

Remarks

This command moves the cursor to the home position, the top left corner, of the current writing area.

Associated Keywords

CLL, CLS, [CUR]MOVE, CURPOS, WRITING

IF...THEN...ELSE Conditional Construct

Conditionally execute instruction(s) on a single line.

Syntax

IF *t* THEN Instruction(s) [ELSE Instruction(s)]

Remarks

When *t* is TRUE, the instruction(s) following THEN are executed and those following ELSE (if present) are skipped.

When *t* is FALSE, the instruction(s) following THEN are skipped and those following ELSE (if present) are executed.

You can put several instructions (separated by colons) after the THEN and ELSE keywords, or you can put a line number (implied GOTO).

Examples

```
IF Month = 9 THEN PRINT "September"

IF X < 3 OR X > 5 THEN 95

IF A > 10 THEN B := 5 : C := 6 ELSE C := X * Y
```

INPUT Command

Read data from a specified input channel.

Syntax

INPUT [LINE] [*#e*,][*prompt*] *var1*[, *var2*...]

Remarks

prompt is a string in quotes. Before you read data from the keyboard, this string is displayed on the screen. If you use a semicolon after *prompt*, the string will be followed by a question mark.

The INPUT command reads data from the keyboard (or specified channel) and assigns, appropriately, a number or a string to each variable. The following rules apply:

- string data should be separated by commas, and numeric data by spaces or commas
- the number of entered values must be the same as the number of variables
- if you do not enter enough data at the keyboard, a double question mark prompt is displayed on the screen
- if you press <ENTER> without entering data at the keyboard, either an empty string or zero will be assigned to the relevant variable
- if there is not enough entered data from an input channel, an end of file error will occur
- if you enter too much data, a warning is displayed on the screen

As with the DATA command, take care when using INPUT to read strings that may contain spaces or commas. Such data is best handled by the INPUT LINE (or LINE INPUT) command.

INPUT LINE reads an entire line of data items, including commas and loading spaces, into one string variable.

Examples

```
10 INPUT "Enter 3 numbers :", A, B, C
20 PRINT
30 PRINT "Their sum is"; A + B + C
RUN

Enter 3 numbers :10, 45.8, 4

Their sum is 59.8

NEW

10 REM Read data from a phone directory File
15 OPEN #12, "Phone.lst"
20 FOR I% = 1 To 100
30    INPUT LINE #12, Phone_Number$
40    PRINT Phone_Number$
50 NEXT I%
60 CLOSE #12
RUN

Mr T. Smith           63487
Miss M. Jones         48849
Mr and Mrs A. Hart    53160
:
:
```

Associated Keywords

DATA, GET, GET$, READ

INSTR Function

Search one string for the first occurrence of another string.

Syntax

INSTR(*e1$*, *e2$*[, *e1*[, *e2*]])

Remarks

This function searches *e1$* (from left to right) for a match with *e2$* and returns a number indicating the position of the first character of the matching sub-string. If no match is found, 0 (FALSE) is returned.

The value of *e1* specifies the character position in *e1$* from which the search commences.

The value of *e2* specifies the number of characters to be searched.

Examples

```
PRINT INSTR("BASIC", "SIC")

 3


IF INSTR("aeiou", Letter$) THEN PRINT "Letter$ is a vowel"

PRINT INSTR("THE CAT SAT ON THE MAT", "THE", 9)

 16
```

Associated Keywords

LEFT$, LEN, MID$, RIGHT$, STRING$

INT Function

Calculate the largest whole number that is less than or equal to a given value.

Syntax

INT(*e*)

Remarks

This function returns a whole number that is lower than (or equal to) the value of *e*.

Examples

```
PRINT INT(34.53)
 34

PRINT INT(-5.63)
-6
```

Associated Keywords

ABS, SGN

JOYSTICK[2] Graphics Attribute

Initiate the use of joysticks and set up the joystick cursor coordinates, or report the current cursor coordinates, stick direction, and button state.

Syntax

SET JOYSTICK[2] [*e1*, *e2*[, *e3*, *e4*, *e5*]][OFF]

ASK JOYSTICK[2] [*v1*, *v2*[, *v3*[, *v4*, *v5*]]]

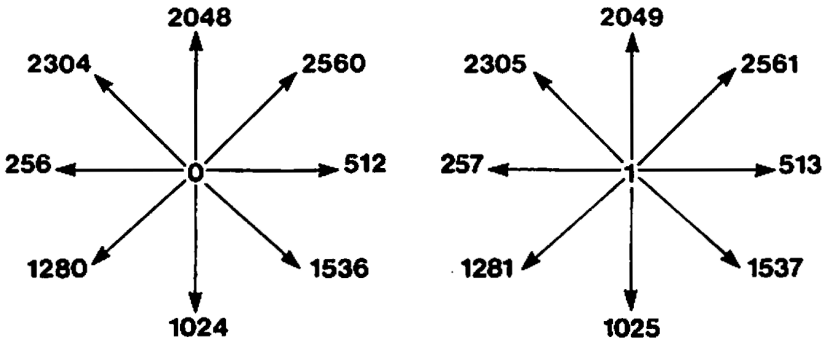
Remarks

You can use either one or two joysticks with Nimbus. When there is only one, it is referred to as JOYSTICK. When there are two, the second is referred to as JOYSTICK2:

- SET JOYSTICK[2] initializes the use of a joystick and sets up the coordinates of the joystick cursor (using the current graphics coordinate system). They are either at the centre of the screen or at the last position with previous use (when the screen mode has not been reset since previous use)
- SET JOYSTICK[2] *e1*, *e2* sets up the joystick cursor coordinates. The values of *e1* and *e2* indicate the initial x and y coordinates
- SET JOYSTICK[2] *e1*, *e2*, *e3*, *e4*, *e5* sets up the joystick coordinates as above. The values of *e4* and *e5* (in the range 0 to 255, with a default value of 1) specify the gain in the x and y directions — the distance that the joystick can be moved before the value returned by ASK JOYSTICK changes. *e3* can have any value; it is ignored
- SET JOYSTICK[2] OFF stops the use of joysticks. If you try to find details of the state of the joystick, an error will occur

The joysticks are automatically turned off when you change screen widths.

ASK JOYSTICK[2] returns the current joystick cursor coordinates in *v1* and *v2*, and the current gain in *v4* and *v5*. The value returned in *v3* indicates the direction of the stick and the state of the joystick button:



Use ASK JOYSTICK[2] to guarantee accurate values for x and y coordinates at any instant. With POSX and POSY, you cannot obtain simultaneous values of the coordinates.

Examples

```
200 SET JOYSTICK : REM Draw circles
210 REPEAT
220   ASK JOYSTICK X%, Y%, Butt%
230   IF Butt% AND 1 THEN CIRCLE 20, X%, Y% BRUSH 3
240 UNTIL FALSE
```

Associated Keywords

BUTTONS, JOYX, JOYY, MODE, MOUSE, POSX, POSY, RPOINT

JOYX Graphics Function

Provide the current joystick direction in the x axis.

Syntax

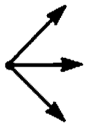
JOYX(*e*)

Remarks

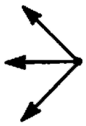
JOYX returns the current x-axis direction of a joystick. The value of *e* specifies which joystick:

1 indicates joystick1
2 indicates joystick2

The value returned by JOYX indicates the direction in which the joystick is pointing:



these directions give +1



these directions give –1

these directions give 0

If the value of *e* is 0, this specifies a mouse and the value returned by JOYX is always 0.

Associated Keywords

BUTTONS, JOYSTICK, JOYSTICK2, JOYY, POSX, POSY, RPOINT

JOYY Graphics Function

Provide the current joystick direction in the y-axis.

Syntax

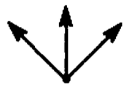
JOYY(*e*)

Remarks

JOYY returns the current y-axis direction of a joystick. The value of *e* specifies which joystick:

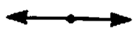
1 indicates joystick1
2 indicates joystick2

The value returned by JOYY indicates the direction in which the joystick is pointing:



these directions give +1

these directions give –1



these directions give 0

If the value of *e* is 0, this specifies a mouse and the value returned by JOYY is always 0.

Associated Keywords

BUTTONS, JOYSTICK, JOYSTICK2, JOYX, POSX, RPOINT

KEYREP Attribute

Set the auto-repeat key option on or off, or report its current setting.

Syntax

SET KEYREP *t*

ASK KEYREP *v*

Remarks

When the value of *t* is TRUE, SET KEYREP sets the auto-repeat option on. When the value of *t* is FALSE, auto-repeat is off.

When auto-repeat is on, the action of a key is repeated when you hold the key down; this is the default setting.

ASK KEYREP returns the current setting in *v*.

Examples

```
ASK KEYREP A%
PRINT A%
```

```
-1
```

Associated Keywords

FLUSH

LEAVE Command

Leave a procedure before reaching the end.

Syntax

LEAVE

Remarks

The LEAVE command is used to return control from anywhere within a procedure as if the ENDPROC command had been reached.

Example

```
400 PROCEDURE Test(A)
410 IF A = 0 THEN LEAVE
420 PRINT "Entered value is O.K."
430 ENDPROC
```

Associated Keywords

PROCEDURE/ENDPROC

LEFT$ Function

Create a string of characters copied from another string, starting at its left end.

Syntax

LEFT$(*e$*, *e*)

Remarks

The number of characters to be copied from *e$* is specified by the value of *e*.

If the value of *e* is greater than the length of *e$*, the entire string is returned.

Examples

```
10 Best$ := "RESEARCH MACHINES"
20 B$ := LEFT$(Best$, 8)
30 PRINT B$

RESEARCH
```

Associated Keywords

INSTR, LEN, MID$, RIGHT$, STRING$

LEN Function

Return the number of characters in a string.

Syntax

LEN(*e$*)

Remarks

LEN returns zero when *e$* is an empty (null) string. Otherwise, LEN returns the number of characters in the string.

The maximum allowable length of a string depends on the internal memory size of your computer, up to a maximum of 32000.

Examples

```
10 Hotel$ := "White Lion"
20 PRINT LEN(Hotel$)

 10
```

Associated Keywords

INSTR, LEFT$, MID$, RIGHT$, STRING$

LET Command

Assign the value of an expression to a variable.

Syntax

[LET] *v* [:]= *e*

or

[LET] *v$* [:]= *e$*

Remarks

The value of the expression is stored in the variable.

Strings cannot be assigned to numeric variables and numbers cannot be assigned to string variables.

Examples

```
LET A = A + 1

Author$ := "Charles Dickens"

LET Shade$(6) = "beige"
```

LINE Graphics Command

Draw line(s) on the screen.

Syntax

LINE *coordinate list* [*option list*]

Remarks

This command is used to draw lines connecting each coordinate pair to the next, starting at the first and ending at the last. When only one coordinate pair is specified, a point is drawn.

Unless you specify the relevant option in the LINE command, the current settings of brush colour, line-drawing style, and drawing mode are used (either the default settings or those set with the SET *attrib* command).

The following options, giving temporary change, can be specified after *coordinate list*:

Option	*Action*	*Syntax*
BRUSH	Selects the brush colour	see BRUSH graphics attribute
STYLE	selects the line-drawing style	see LINE STYLE graphics attribute
OVER	selects the drawing style	see OVER graphics attribute

Examples

```
LINE 0, 0; 100, 50 BRUSH 4 STYLE 2
```

Options

BRUSH, OVER, [LINE] STYLE

Associated Keywords

AREA, CIRCLE, FLOOD, PLOT, POINTS, SLICE

LIST Command

List all or part of the program currently in memory.

Syntax

LIST [~*e*/#*e*] [*range*]

Remarks

This command lists the specified line or range of lines, from the program currently in memory to either the specified writing area or to the specified channel (0 for screen, 2 for printer, 11–127 for file).

- LIST
  Lists the whole program.
- LIST *line1*
  Lists the line with that line number.
- LIST *line1* TO
  Lists all lines from *line1* to the end of the program.
- LIST *line1* TO *line2*
  Lists these lines and all between them.
- LIST TO *line2*
  Lists all lines up to and including that line number.

Examples

```
LIST TO 100
```

Associated Keywords

AUTO, EDIT, RENUMBER, DELETE

LN Function

Calculate the natural logarithm of a number.

Syntax

LN(*e*)

Remarks

e is a numeric expression; its value must be greater than zero.

LN returns the natural logarithm (log to the base e) of a numeric expression.

Examples

```
PRINT LN(1.32)

 0.277632


PRINT LN(24 / 5)

 1.568616
```

Associated Keywords

EXP, LOG

LOAD Command

Load a program file from disk into memory.

Syntax

LOAD *e$*

Remarks

e$ is a string expression; its value represents the file specification.

A search is made for a file specified by *e$*:

- If *e$* is found, the current contents of memory are cleared and the file is loaded into memory.
- If *e$* is not found, an error message is displayed and the current contents of memory are not cleared.

RM Basic will automatically detect the format (SAVE or PSAVE) that was used when saving the file, so that it is loaded correctly.

Examples

```
LOAD "Fredprog"

LOAD "B:Ballgame"
```

Associated Keywords

LOADGO, MERGE, MERGEGO, PSAVE, SAVE

LOADGO Command

Load a program file from disk into memory, and execute the program.

Syntax

LOADGO *e$*

Remarks

LOADGO is similar to the LOAD command, but if *e$* is found the program is executed immediately, starting at the first line of the program.

LOADGO can be useful within a program to run another program in its place.

Examples

```
 10 REM Menu program that displays available
 20 REM programs and runs the selected one.
 30 DIM Prog_Name$(5)
 40 FOR Option% = 1 TO 5
 50   READ Name$ : PRINT Option%, Name$
 60   LET Prog_Name$(Option%) = Name$
 70 NEXT Option%
 80 Input "which program"; Choice
 90 LOADGO Prog_Name$(Choice)
100 DATA "Fredprog", "Ballgame", "PROG1", "\Master", "Edu"
```

Associated Keywords

LOAD, MERGE, MERGEGO, PSAVE, SAVE

LOG Function

Calculate the logarithm, to the base 10, of a number.

Syntax

LOG(*e*)

Remarks

e is a numeric expression; its value must be greater than zero.

LOG returns the logarithm to the base 10 of a numeric expression.

Examples

```
PRINT LOG(5.2)

 0.716003


PRINT LOG(16.4 -3.7)

 1.103804
```

Associated Keywords

EXP, LN

LOOKUP Function

Search for a specified disk file.

Syntax

LOOKUP(*e$*)

Remarks

e$ is a string expression; its value represents a filename.

The LOOKUP function returns either TRUE (–1) when the specified file is found, or FALSE (0) when it is not found.

Examples

```
IF NOT LOOKUP("DataFile\Mydate") THEN
CREATE #12 "DataFile\Mydata"
```

Associated Keywords

CHDIR, CREATE, DIR, ERASE, FSPACE, MKDIR, OPEN, PATH$, RENAME...TO, RMDIR

LVAR [LEVEL] Command

List the names of the current arrays and the values of all current variables, or list those in a specified level.

Syntax

LVAR [~*e1*/#*e1*][, LEVEL *e2*]

Remarks

LVAR writes the name and value of currently accessible variables to the specified writing area or channel. The names of arrays and their upper bound (maximum subscript value) are also listed. Global variables and arrays are preceded by an asterisk.

LVAR LEVEL supplies only those variables and arrays in the program level specified by the value of *e2*:

0	the current level (default setting)
–1	the next level of procedure nesting
–2	two levels of nesting

and so on.

Examples

```
LVAR ~2

A = 4
B$ = "India"
*X% = 6
Y(10)
```

Associated Keywords

BOUNDS, DEFINED, LVAR GLOBAL, PROCS, TRACE

LVAR GLOBAL Command

List the names of all global variables and arrays and the values of the variables.

Syntax

LVAR [~*e*/#*e*,] GLOBAL

Remarks

LVAR GLOBAL writes the names and values of all global variables to the specified writing area or channel. The names of all global arrays and their upper bounds are also listed.

Associated Keywords

BOUNDS, DEFINED, LVAR [LEVEL], PROCS, TRACE

MEM Function

Find the amount of free memory available for variables and arrays.

Syntax

MEM

Remarks

The MEM function returns the number of bytes that are available for variables and arrays. Memory requirements are:

- a string or integer variable uses 4 bytes
- a floating-point variable uses 6 bytes
- within an array, a string or integer variable uses an extra 2 bytes for each element. A floating-point variable uses an extra 4 bytes

Examples

```
PRINT MEM

 21225
```

Associated Keywords

CLEAR, FREE, NEW, RUN

MERGE Command

Combine the lines from a program file with the lines of the program currently in memory.

Syntax

MERGE *e$*

Remarks

e$ is a string expression that represents the file specification.

A search is made for the named file. If found, the program lines in the file are added to the program in memory as if entered from the keyboard.

If any line in the file program has the same line number as a line of the program in memory, the line from the file replaces the one in memory.

Program files that have been saved with the PSAVE command cannot be merged.

Examples

```
MERGE "B:Jill2"
```

Associated Keywords

LOAD, LOADGO, MERGEGO, SAVE

MERGEGO Command

Combine file program lines with memory program lines and run all or part of the modified program in memory.

Syntax

MERGEGO *e$*

Remarks

e$ is a string expression that represents the file specification.

MERGEGO is similar to the MERGE command, but after program combination, the modified program in memory is executed immediately.

Examples

```
MERGEGO "Addition"
```

Associated Keywords

LOAD, LOADGO, MERGE, SAVE

MID$ Function

Create a copy of part of a string.

Syntax

MID$(*e$*, *e1*[, *e2*])

Remarks

The value of *e1* gives the position of the first character to be copied (counting from the left end of *e$*). The value of *e2* specifies the number of characters to be copied.

All characters to the right of the *e1* position will be copied either if *e2* is omitted or if the value of *e2* is greater than the length of *e$*.

Examples

```
10 Bells$ := "CAMPANOLOGY"
20 PRINT MID$(Bells$, 4, 3)

PAN
```

Associated Keywords

INSTR, LEFT$, LEN, RIGHT$, STRING$

MIX Graphics Function

Generate a colour in terms of colour gun settings.

Syntax

MIX(*e1*, *e2*, *e3*, *e4*)

Remarks

Using the MIX function, you can specify colour in terms of the colour guns of the monitor. The value returned by MIX can be used in the COLOUR command for redefinition of the colour table.

The relationship of the gun settings to the resultant colour and returned value are shown below (1 is gun on, 0 is gun off):

Gun settings				*Value*	*Resultant Colour*	
Brightness	*Green*	*Red*	*Blue*	*Returned*	*Colour monitor*	*Mono monitor*
0	0	0	0	0	Black	Black
0	0	0	1	1	Dark blue	G1
0	0	1	0	2	Dark red	G2
0	0	1	1	3	Purple	G3
0	1	0	0	4	Dark green	G4
0	1	0	1	5	Dark cyan	G5
0	1	1	0	6	Brown	G6
0	1	1	1	7	Light grey	G7
1	0	0	0	8	Dark grey	G8
1	0	0	0	9	Light blue	G9
1	0	1	0	10	Light red	G10
1	0	1	1	11	Magenta	G11
1	1	0	0	12	Light green	G12
1	1	0	1	13	Cyan	G13
1	1	1	0	14	Yellow	G14
1	1	1	1	15	White	White

G stands for grey. Greyness lightens as the number increases on a monochrome monitor.

The four arguments of the MIX function specify the state of each gun:

- *e1* indicates the brightness gun
- *e2* indicates the green gun
- *e3* indicates the red gun
- *e4* indicates the blue gun

Each argument can be zero (gun off) or non-zero (gun on).

Examples

```
SET COLOUR 2 TO MIX(0, 1, 1, 0)
```

Associated Keywords

COLOUR

MKDIR Command

Create a disk file directory.

Syntax

MKDIR *e$*

Remarks

e$ is a string expression; its value represents the pathname (see "Operating System Concepts" in the Nimbus Owners Handbook).

MKDIR creates a new directory. This command works in the same way as the MS-DOS command, MKDIR (see "Operating System Concepts" in the Nimbus Owners Handbook).

Examples

```
MKDIR "Amanda"

MKDIR "\Students\Anthony"
```

Associated Keywords

CHDIR, DIR, ERASE, FSPACE, LOOKUP, PATH$, RENAME...TO, RMDIR

MOD Arithmetical Operator

Provide the remainder of integer division.

Syntax

e1 MOD *e2*

Remarks

MOD gives the integer remainder from integer division of *e1* by *e2*.

If X is the result of integer division:

```
X := e1 \ e2
```

Integer division operator

then the integer remainder given by MOD is:

```
Remainder := e1 - e2 * X
```

For example, 8 MOD 3 returns 2

Examples

```
PRINT 24 MOD 5
 4
```

MODE Attribute

Clear and reinitialize the screen display, or report the current number of characters per line and the current graphics coordinate range.

Syntax

SET MODE [*e1*[, *e2*, *e3*]]

ASK MODE *v1*[, *v2*, *v3*]

Remarks

On its own, SET MODE clears the screen and reinitializes text and graphics output to the screen. Graphics input (mouse and joystick(s)) is reinitialized.

When *e1* is specified its value should be 40 or 80:

- 40 indicates forty text characters per screen row and graphics coordinates of 0 to 319 pixels in the x direction, and 0 to 249 pixels in the y direction (low resolution graphics).
- 80 indicates eighty text characters per screen row and graphics coordinates of 0 to 639 pixels in the x direction, and 0 to 249 pixels in the y direction (high resolution graphics).

When *e2* and *e3* are specified, their values indicate the range of pixel coordinates in the x direction (0 to *e2*) and the range in the y direction (0 to *e3*).

ASK MODE assigns the current screen mode in *v1* and the current graphics coordinates in *v2* and *v3*.

Examples

```
SET MODE 80, 500, 200
```

Associated Keywords

BRUSH, [CUR]MOVE, CURPOS,
DRAWING, JOYSTICK, JOYSTICK2,
MOUSE, ORIGIN, PAPER, PEN, WRITING

MOUSE Graphics Attribute

Initiate the use of a mouse and set up the mouse cursor coordinates, or report the current mouse cursor coordinates, and button state.

Syntax

SET MOUSE [*e1*, *e2*[, *e3*, *e4*, *e5*]] [OFF]

ASK MOUSE [*v1*, *v2*[, *v3*[, *v4*, *v5*]]]

Remarks

SET MOUSE is used to inform RM Basic that a mouse is going to be used:

- SET MOUSE initializes the use of a mouse and sets up the x and y coordinates (using the current coordinate system) of the mouse cursor. The coordinates are either at the centre of the screen or at the last position with previous use (when screen mode has not been changed since previous use).
- SET MOUSE *e1*, *e2* sets up the mouse cursor coordinates. The value of *e1* indicates the x-coordinate, and that of *e2* indicates the y-coordinate.
- As above, SET MOUSE *e1*, *e2*, *e3*, *e4*, *e5* sets up the mouse cursor coordinates. In addition, the values of *e4* and *e5* (in the range 0 to 255, with a default value of 1) specify the gain in the x and y directions — the distance that the mouse can be moved before the value returned by ACT MOUSE changes. *e3* can have any value; it is ignored.
- SET MOUSE OFF stops the use of a mouse. If you try to find details of the state of the mouse, an error occurs.

The mouse is automatically turned off when you leave RM Basic.

ASK MOUSE returns the current mouse cursor coordinates in *v1* and *v2*, and the current gain in *v4* and *v5*. The value returned in *v3* indicates the state of the mouse buttons:

0 neither buttons pressed
1 the right-hand button pressed
2 the left-hand button pressed
3 both buttons pressed

Use ASK MOUSE to guarantee accurate values of x and y coordinates at any instant. With POSX, and POSY, you may not obtain simultaneous values of the coordinates.

Examples

```
500 SET MOUSE
510 REPEAT
520   ASK MOUSE X%, Y%, Butt%
530   POINTS X%, Y% STYLE 3
540 UNTIL Butt% <> 0
550 REM Predefined area of the screen.
560 REM Such as a menu square.
570 IF X% > 10 AND X% < 100 AND y% > 10 AND
Y% < 100 THEN Game 3
```

Associated Keywords

BUTTONS, JOYSTICK, JOYSTICK2, MODE, POSX, POSY, RPOINT

NEW Command

Delete the program currently in memory and clear all variables.

Syntax

NEW

Remarks

This command is used before you enter a new program to ensure that all previous instructions are forgotten.

Associated Keywords

CLEAR, FREE, MEM, RUN

NOISE Sound Command

Synchronize the playing of held notes and change the pitch of noise produced.

Syntax

NOISE *e1*[, *e2*...]

Remarks

When the HOLD attribute is set to TRUE, the execution of a NOTE command does not produce sound until the note or noise is released by a NOISE command. The sound is stored at the end of the voice queue until released.

If TONE is set FALSE, the value of *e1*, in the range 1 to 96, specifies the pitch around which pink noise is produced. If TONE is set TRUE, *e1* is ignored.

The value of *e2* indicates the time in centiseconds that will elapse between the playing of the first sound in the queue and the next. Subsequent arguments (maximum 5) indicate the elapse time for subsequent sounds in the queue.

You can use this facility to synchronize notes when playing music. But, be very careful that you do not fill the queue; the maximum is six stored sounds per voice. If the queue overflows, you cannot release the sounds in the queue. You will have to reset your Nimbus.

You can use NOISE 1 to clear voice channels of stored sounds, the QUEUE function returns the number of free spaces in a voice queue.

In a similar way that sounds are stored in a queue, the holding times from NOISE commands are stored in the voice 0 queue, as before, the maximum number is five.

Examples

```
10 SET HOLD TRUE
20 FOR I% = 1 TO 5
30   NOTE I% * 50, 50
40   NOISE 1, 200
50 NEXT I%
```

Associated Keywords

ENVELOPE, HOLD, NOTE, PITCH, QUEUE, SOUND, TONE, VOICE

NOT Logical Operator

Logical complement of an expression.

Syntax

NOT *e*

Remarks

NOT gives the result of bit by bit inversion (see appendix B) of an integer numeric expression.

A common use is in control structures, such as IF...THEN...ELSE:

```
IF NOT (A == B) THEN STOP
```

When the A=B comparison returns 0 (FALSE), NOT gives –1 (TRUE), the complement of 0, and the instruction following THEN is executed.

NOT can also be used in numerical expressions to invert an integer operand:

```
A% := NOT 5
```

This assigns –6 to A%.

Examples

```
FLAG := NOT FLAG
```

Associated Keywords

AND, OR, XOR

NOTE Sound Command

Produce sound from the internal speaker.

Syntax

NOTE *e1*[TO *e2*][, *e3*[, *e4*]/ENVELOPE *e5*] [option]

Remarks

The NOTE command is used to produce sound.

When the TONE attribute is set TRUE (the default setting), sound of a specific pitch is played. In the simplest case, NOTE *e1* produces one note with a pitch given by the value of *e1* in the range 1 to 388. The note length and volume is given by the current envelope setting (default 0).

NOTE *e1* TO *e2* can be used for a sliding-pitch note (glissando effect). The value of *e1* is the starting pitch and the value of *e2* is the finishing pitch.

You can use the PITCH function to generate pitch values.

The duration of single note or sliding-pitch notes is given by the value of *e3* in centiseconds, and the volume is given by the value of *e4* in the range 0 to 15. Alternately, you can use ENVELOPE *e5* to specify duration and volume.

Unless you specify the relevant option in the NOTE command, the current settings of released or held sound, notes or noise, and voice channel are used (either the default settings or those set with the SET *attrib* command).

The following options, giving temporary change, can be specified in the NOTE command:

Option	*Action*	*Syntax*
HOLD	select released or held sound	see HOLD sound attribute
TONE	select note or noise production	see TONE sound attribute
VOICE	select the voice channel	see VOICE sound attribute

Examples

```
NOTE 148, 100, 10

NOTE PITCH(0,0) TO PITCH(1,0), ENVELOPE 4 VOICE 1
```

Options

HOLD, TONE, VOICE

Associated Keywords

ENVELOPE, NOISE, PITCH, QUEUE, SOUND

ON BREAK Command

Control what happens when keyboard interruption of a running program occurs.

Syntax

ON BREAK [OFF]

Remarks

ON BREAK traps an attempt to interrupt a running program at the keyboard using <BREAK>.

When <BREAK> is pressed, execution jumps to the instruction(s) following ON BREAK.

Interrupts are only trapped after an ON BREAK command has been executed, and once control has been transferred, the trap is no longer active.

You can switch off an interrupt trap with ON BREAK OFF.

Examples

```
10 REM Ignore keyboard interruption.
20 ON BREAK RESUME

10 REM Return to program menu by calling procedure.
20 ON BREAK Menu
```

Associated Keywords

ON ERROR, ON EOF

ON EOF Command

Control what happens when an attempt is made to read past the end of a disk file.

Syntax

ON EOF *#e* [OFF]

Remarks

The ON EOF command controls what happens when RM Basic tries to read past the end of a disk file with INPUT [LINE] or GET.

When the end of a file is reached, program execution jumps to the instruction(s) following ON EOF.

The end of file trap only works after an ON EOF command has been executed. Once control has been transferred, the trap is no longer active.

You can switch the trap off with ON EOF OFF.

Examples

```
10 OPEN #11, "Jimsdata"
20 ON EOF #11 End_File = TRUE : CONTINUE
30 End_File = FALSE
40 REPEAT
50    INPUT #11, Number
60    PRINT Number
70 UNTIL End_File
80 CLOSE #11
90 END
```

Associated Keywords

INPUT [LINE], ON ERROR

ON ERROR Command

Control what happens when an error occurs.

Syntax

ON ERROR [OFF]

Remarks

Normally, RM Basic displays an error message when an error occurs during program execution. However, you can use the ON ERROR command to change what happens — this is known as setting an error trap.

If an error occurs, execution jumps to the instruction(s) following ON ERROR. This could be transferred to a procedure or subroutine to handle the problem, or re-entry of data, or whatever you require.

Errors will only be trapped after the ON ERROR command has been executed, and once control has been transferred, the error trap is no longer active.

You can switch off a trap with ON ERROR OFF.

Examples

```
60 ON ERROR GOSUB 400

400 ON ERROR PRINT "AN error has occurred. Please
re-enter all the data."
410 FOR I% = 1 To 10
420    INPUT "Enter data :", Value(I%)
430 NEXT I%
```

Associated Keywords

ON BREAK, ON EOF

ON...GOSUB ON...GOTO Commands

Jump to one of a selection of line numbers or line labels, depending on the value of an expression.

Syntax

ON *e* GOSUB *line/name*[, *line/name...*]

ON *e* GOTO *line*[, *line...*]

Remarks

These commands are used to alter the order of program execution by jumping to one of a list of line numbers and/or labelled lines; the value of *e* determines which one of the list is selected.

For example, if the value of *e* is 2, the second line number in the list will be the next line to be executed.

When the value of *e* is either greater than or less than the number of items in the list, a line number is not selected and the instruction following the ON ... GOSUB or GOTO instruction is executed.

Examples

```
ON Result% GOSUB 500, 700, 1000

ON RND(4) GOSUB Shape1, Shape2, Shape3, Shape4
```

Associated Keywords

GOSUB, GOTO, RETURN

OPEN Command

Open a file for input from it.

Syntax

OPEN *#e, e$*

Remarks

e is a numeric expression with a value in the range 11 to 127. This value is the channel number that you must subsequently specify in commands that read from *e$*. *e$* is a string expression that represents the file specification.

The OPEN command must be used to open a file before you can read from it with GET, GET$, or INPUT [LINE]. Files should be closed after use.

Examples

```
OPEN #11, "DATA1"

OPEN #20, "FILEB.TXT"
```

Associated Keywords

CHDIR, CLOSE, CREATE, DIR, ERASE, FSPACE, LOOKUP, MKDIR, PATH$, RENAME...TO, RMDIR

OR Logical Operator

Logical operation of two expressions.

Syntax

e1 OR *e2*

Remarks

OR gives the result of bit by bit combination (see appendix B) of two comparisons or integers.

A common use is in control constructs such as IF...THEN...ELSE:

```
IF A = B OR C > D THEN STOP
```

OR combines the results of the A=B and C>D comparisons. The instruction following THEN is executed when one or both comparisons return TRUE.

OR can be used in numerical expressions to logically combine two integer operands:

```
A% := 5 OR 6
```

This assigns 7 to A% (see appendix B).

Examples

```
Fred2 := Fred1 OR 36
```

Associated Keywords

AND, NOT, XOR

ORIGIN Graphics Attribute

Change the origin of the current graphics coordinate system, or report its current setting.

Syntax

SET ORIGIN *e1*, *e2*

ASK ORIGIN *v1*, *v2*

Remarks

SET ORIGIN is used to change the position of the origin within the current coordinate system. The value of *e1* indicates the x-coordinate, and that of *e2* indicates the y-coordinate.

Future coordinate specifications in graphics commands are with respect to this origin.

The default setting of the origin is 0, 0 — the bottom left-hand corner of the graphics screen.

ASK ORIGIN returns the current origin coordinates to *v1* and *v2*.

Examples

```
SET ORIGIN 100, 0
```

Associated Keywords

DRAWING, MODE

OVER Graphics Attribute

Set drawing mode, or report its current setting.

Syntax

SET OVER *t*

AREA/FLOOD/LINE/POINTS *coordinate list* OVER *t*

CIRCLE *e1*, *coordinate list* OVER *t*

SLICE *e1*, *e2* TO *e3*, *coordinate list* OVER *t*

PLOT *e$*, *coordinate list* OVER *t*

BLOCK COPY *e1*, *e2*; *e3*, *e4*; *coordinate list* OVER *t*

BLOCK WRITE *ov*, *coordinate list* OVER *t*

ASK OVER *v*

Remarks

The SET OVER *t* command selects the graphics drawing mode. The OVER *t* option, in a graphics command, temporarily selects the drawing mode for the duration of the command.

When the value of *t* is TRUE (non-zero) characters (drawn with PLOT), points, lines, and areas are drawn in destructive overwrite mode. When the value of *t* is FALSE, non-destructive overwriting (XOR mode) is used. The default setting is TRUE.

In XOR mode you can draw new graphics over what is already drawn. When you repeat the drawing instruction, the new drawing is removed and the previous one re-appears.

ASK OVER returns the current setting of the drawing mode in *v*.

Examples

```
PLOT "Happy Birthday", 120, 100 OVER FALSE
```

Associated Keywords

AREA, BLOCK COPY, BLOCK WRITE, BRUSH, CHAROVER, CIRCLE, FLOOD, LINE, PLOT, POINTS, SLICE

PAPER Attribute

Select the screen text background colour, or report its current setting.

Syntax

SET PAPER *e*

ASK PAPER *v*

Remarks

SET PAPER is used to set the colour or grey level of the background on which screen text characters are displayed. The value of *e* specifies the colour from the appropriate colour table — 40-character mode or 80-character mode.

The default settings of the colour tables are shown in the BRUSH definition. In 40-character mode, the value of *e* can be in the range 0 to 15. In 80-character mode, the value of *e* can be in the range 0 to 3; if you specify a higher value, it will be folded into the range 0 to 3. The default background colour in both modes is 0.

You can change a single element, several elements, or the whole of the colour table with SET COLOUR.

ASK PAPER returns the currently selected background colour in *v*.

Examples

```
SET PAPER 3
```

Associated Keywords

BRUSH, CLS, COLOUR, MODE, PEN

PATH$ Function

Report the current disk file directory.

Syntax

PATH$

Remarks

This function returns the full pathname from the root directory to your current directory on the default drive (see "Operating System Concepts" in the Nimbus Owners Handbook).

Examples

If your working directory is \NAMES\RACHEL\TEXT and the drive is A:, this is what happens:

```
PRINT PATH$
A:\NAMES\RACHEL\TEXT
```

Associated Keywords

CHDIR, DIR, ERASE, FSPACE, LOOKUP, MKDIR, RENAME...TO, RMDIR

PATTERN Graphics Attribute

Define a graphics dithered pattern, or report the details of a defined dithered pattern.

Syntax

SET PATTERN *e1*, *pattern*

ASK PATTERN *e1*, *pattern*

Remarks

A dithered pattern is a pattern of dots of different colours. When used to fill areas, such patterns have the effect of producing a new colour that is a combination of the pure colours in the pattern.

A dithered pattern can be specified for area-filling with the SET BRUSH *e* command or with the BRUSH *e* option. The value of *e* can be in the range 128 to 143.

There are eight pre-defined dithered patterns (brush numbers 128 to 135) for each graphics resolution, shown below. You can define a further eight (brush numbers 136 to 143) and, if you wish, you can re-define 128 to 135.

The SET PATTERN command defines a pattern. The value of *e1*, in the range 128 to 143, indicates the brush number to which the pattern will be assigned.

There are two methods of using the command:

In method 1, *pattern* has the following syntax:

e2 TO *e3*, *e4*, *e5*, *e6*

The value of *e2*, in the range 0 to 3, represents the row number of the pattern. The values of *e3* to *e6*, in the range 0 to 3 (high resolution graphics) or 0 to 15 (low resolution graphics) represent the colours of the four dots in that row.

An even distribution of the dot colours gives a uniform colour when areas are filled.

In method 2, *pattern* is a numeric array containing sixteen pure colour numbers. The first four make up the first row, the second four make up row 2, the third four make up row 3, and the last four make up row 4.

ASK PATTERN can be used to return the details of a dithered pattern into four numeric variables (method 1) or into *e1* and a numeric array (method 2).

A dithered pattern is defined by four of these SET PATTERN instructions. For example:

```
SET PATTERN 136, 1 TO 1, 2, 1, 2
SET PATTERN 136, 2 TO 2, 3, 2, 3
SET PATTERN 136, 3 TO 1, 2, 1, 2
SET PATTERN 136, 4 TO 2, 3, 2, 3
```

40-character mode (low resolution)
pre-defined patterns

Pattern number	*Pattern of dot colours*	*Pattern number*	*Pattern of dot colours*
128	0 1 0 1 1 0 1 0 0 1 0 1 1 0 1 0	123	1 3 1 3 3 1 3 1 1 3 1 3 3 1 3 1
129	0 2 0 2 2 0 2 0 0 2 0 2 2 0 2 0	133	2 3 2 3 3 2 3 2 2 3 2 3 3 2 3 2
130	0 3 0 3 3 0 3 0 0 3 0 3 3 0 3 0	134	15 0 15 0 0 15 0 15 15 0 15 0 0 15 0 15
131	1 2 1 2 2 1 2 1 1 2 1 2 2 1 2 1	135	15 1 15 1 1 15 1 15 15 1 15 1 1 15 1 15

80-character mode (high resolution) pre-defined patterns

Pattern number	*Pattern of dot colours*	*Pattern number*	*Pattern of dot colours*
128	1 0 1 0 0 1 0 1 1 0 1 0 0 1 0 1	132	3 2 3 2 2 3 2 3 3 2 3 2 2 3 2 3
129	2 0 2 0 0 2 0 2 2 0 2 0 0 2 0 2	133	3 1 3 1 1 3 1 3 3 1 3 1 1 3 1 3
130	1 2 1 2 2 1 2 1 1 2 1 2 2 1 2 1	134	3 1 3 2 1 3 2 3 2 3 1 3 3 2 3 1
131	3 0 3 0 0 3 0 3 3 0 3 0 0 3 0 3	135	1 3 1 2 3 1 2 1 2 1 3 1 1 2 1 3

Examples

```
ASK PATTERN 137, Dither()
```

Associated Keywords

BRUSH

PEN Attribute

Select the foreground (character) colour of screen text, or report its current setting.

Syntax

SET PEN *e*

ASK PEN *v*

Remarks

SET PEN is used to set the colour or grey level of the screen text characters produced by subsequent PRINT or PUT commands. The value of *e* specifies the colour from the appropriate colour table — 40-character mode or 80-character mode.

The initial settings of the colour tables are shown in the BRUSH definition. The default pen colour is 15 in 40-character mode and 3 in 80-character mode. In 40-character mode, the value of *e* can be in the range of 0 to 15. In 80-character mode, the value of *e* can be in the range 0 to 3; if you specify a higher value, it will be folded into the range 0 to 3.

You can change a single element, several elements, or the whole of the colour table with SET COLOUR.

ASK PEN returns the current pen colour in *v*.

Examples

```
SET PEN 1
```

Associated Keywords

BRUSH, COLOUR, MODE, PAPER

PI Function

Provide the value of the constant, pi.

Syntax

PI

Remarks

The PI function returns the value, 3.141593.

Examples

```
PRINT 2 * PI

 6.283185
```

PITCH Function

Generate a number giving the specified pitch of a note.

Syntax

PITCH(*e1*, *e2*[, *e3*])

Remarks

Using the PITCH function, you can specify the pitch of a note to be played by the NOTE command.

The value of *e1*, ranging from –3 to 4, indicates the octave. Octave –3 is lowest, and middle C lies in octave 0.

The value of *e2*, ranging from 0 to 11, indicates the note:

0	C
1	C#, D♭
2	D
3	D#, E♭
4	E
5	F
6	F#, G♭
7	G
8	G#, A♭
9	A
10	A#, B♭
11	B

When specified, the value of *e3* (in the range –3 to 3) indicates a quarter semitone pitch by offsetting from one of the chromatic notes of the above table.

For example, if you have selected note D, you can select a quarter semitone above or below it:

Note	*Offset*
C#	–4
	–3
	–2
	–1
D	0
	1
	2
	3
D#	4

Examples

```
NOTE PITCH (–1, 0) TO PITCH (0, 0), 100, 15
```

Associated Keywords

ENVELOPE, HOLD, NOISE, NOTE,
QUEUE, SOUND, TONE, VOICE

PLOT Graphics Command

Draw graphics characters on the screen.

Syntax

PLOT *e$*, *coordinate list* [*option list*]

Remarks

This command is used to draw *e$* at each of the points specified in *coordinate list*.

Unless you specify the relevant option in the PLOT command, the current settings of brush colour, drawing mode, direction of drawing, character size, and character font are used (either the default settings or those set with the SET *attrib* command).

The following options, giving temporary change, can be specified after *coordinate list*:

Option	*Action*	*Syntax*
BRUSH	selects the brush colour	see BRUSH graphics attribute
OVER	selects the drawing style	see OVER graphics attribute
DIRECTION	selects the drawing direction	see PLOT/CHAR DIRECTION graphics attribute
SIZE	selects the size of characters	see PLOT/CHAR SIZE graphics attribute
FONT	selects the font of characters	see PLOT/CHAR FONT graphics attribute

Examples

```
PLOT "Hello", 100, 50 SIZE 4
```

Options

BRUSH, [PLOT/CHAR] DIRECTION,
[PLOT/CHAR] FONT, [PLOT/CHAR] SIZE,
OVER

Associated Keywords

AREA, CIRCLE, FLOOD, LINE,
POINTS, SLICE

[PLOT/CHAR] DIRECTION — Graphics Attribute

Set the graphics character drawing angle, or report its current setting.

Syntax

SET PLOT DIRECTION *e*
PLOT *e$ coordinate list* DIRECTION *e*
ASK PLOT DIRECTION *v*

Remarks

The SET PLOT DIRECTION command selects the drawing direction for the PLOT command. The DIRECTION option, in a PLOT command, temporarily selects the drawing direction.

The value of *e* indicates the direction:

- 0 specifies normal, horizontal drawing (0 degrees). This is the default setting
- 1 specifies upwards drawing at an angle of 90 degrees from horizontal
- 2 specifies upside-down drawing at an angle of 180 degrees from horizontal
- 3 specifies downwards drawing at an angle of 270 degrees from horizontal

ASK PLOT DIRECTION gives the current direction value in *v*.

Examples

```
SET PLOT DIRECTION 3 : PLOT "Downwards", 60, 40
```

Associated Keywords

PLOT, [PLOT/CHAR] FONT, [PLOT/CHAR] SIZE

[PLOT/CHAR] FONT

Graphics Attribute

Select the graphics character font, or report the current setting.

Syntax

SET PLOT FONT *e*[, *ov*]

PLOT *e$ coordinate list* FONT *e*[, *ov*]

ASK PLOT FONT *v*[, *ov*]

Remarks

The SET PLOT FONT command selects one of the predefined character sets (see "Technical Data" in the Nimbus Owners Handbook) or a defined set for drawing characters with the PLOT command. The FONT option temporarily selects a set for the duration of a PLOT command.

When the value of *e* is 0, the standard character set (IBM-like) is selected. A value of 1 specifies the alternative character set (RM 8-bit). A value of 2 or –1 indicates that you want to use a defined character set; *ov* contains its definition. The default value is 0.

ASK PLOT FONT returns the current character font in *v*.

Examples

```
SET PLOT FONT 1
PLOT "This is in the standard set", 20, 100 FONT 0
```

Associated Keywords

PLOT, [PLOT/CHAR] DIRECTION, [PLOT/CHAR] SIZE

[PLOT/CHAR] SIZE

Graphics Attribute

Set graphics character magnification, or report its current setting.

Syntax

SET PLOT SIZE *e1*[, *e2*]

PLOT *e$ coordinate list* SIZE *e1*[, *e2*]

ASK PLOT SIZE *v1*, *v2*

Remarks

The SET PLOT SIZE command is used to set the width and height magnification of characters drawn with the PLOT command. The SIZE option temporarily selects the magnification for the duration of a PLOT command.

The value of *e1* specifies the width magnification. When specified, the value of *e2* indicates the height magnification; when not specified, the height is set to the same as the width. The default values are 1 and 1.

ASK PLOT SIZE returns the current magnification values in *v1* and *v2*.

Examples

```
PLOT "Wait", 100, 100 SIZE 2, 1
```

Associated Keywords

PLOT, [PLOT/CHAR] DIRECTION, [PLOT/CHAR] FONT

POINTS Graphics Command

Draw point(s) on the screen.

Syntax

POINTS *coordinate list* [*option list*]

Remarks

This command is used to draw a marker at each of the points specified in *coordinate list.*

Unless you specify the relevant option in the POINTS command, the current settings of brush colour, point style and size, and drawing mode are used (either the default settings or those set with the SET *attrib* command).

The following options, giving temporary change, can be specified after *coordinate list*:

Option	*Action*	*Syntax*
BRUSH	selects the brush colour	see BRUSH graphics attribute
SIZE	selects the point size	see POINTS SIZE graphics attribute
STYLE	selects the point style	see POINTS STYLE graphics attribute
OVER	selects the drawing style	see OVER graphics attribute

You cannot specify a dithered pattern for this command.

Examples

```
POINTS 10, 10; 40, 40; 60, 60 BRUSH 3 OVER 0
```

Options

BRUSH, OVER, [POINTS] SIZE, [POINTS] STYLE

Associated Keywords

AREA, CIRCLE, FLOOD, LINE,
PLOT, SLICE

[POINTS] SIZE Graphics Attribute

Set marker magnification for point drawing, or report its current setting.

Syntax

SET POINTS SIZE *e1*[, *e2*]

POINTS *coordinate list* SIZE *e1*[, *e2*]

ASK POINTS SIZE *v1*[, *v2*]

Remarks

The SET POINTS SIZE command selects the width and height magnification of markers drawn with the POINTS command. The SIZE option, in a POINTS command, temporarily selects the magnification for the duration of the command.

The value of *e1* specifies the width magnification. When specified, the value of *e2* indicates the height magnification; when not specified, the height is set to the same magnification as the width. The default values are 1 and 1.

When style 1 (a dot) is the current selection for point-drawing, the marker cannot be magnified.

ASK POINTS SIZE returns the current marker magnification values in *v1* and *v2*

Examples

```
SET POINTS SIZE 2, 4
```

Associated Keywords

POINTS, [POINTS] STYLE

POS Function

Provide the column position of the screen cursor or other device.

Syntax

POS([*e*])

Remarks

e is a numeric expression. Its value specifies a channel number:

- 0 indicates the screen
- 2 indicates a printer
- 11–127 indicates a file

POS returns the number of the column in which the next character will be output. This number will be greater than 0 and less than the current WIDTH setting.

In the case of channel 0, POS returns the current column position of the screen cursor.

Examples

```
IF POS(0) > 60 THEN PRINT
```

Associated Keywords

[CUR]MOVE, CURPOS

POSX Graphics Function

Provide the x-coordinate of either the mouse cursor or the joystick cursor.

Syntax

POSX(*e*)

Remarks

This function returns the x-coordinate of the graphics cursor under the control of the control unit indicated by the value of *e*:

- 0 specifies a mouse
- 1 specifies the left joystick
- 2 specifies the right joystick

Examples

```
PRINT POSX(1)

 55
```

Associated Keywords

JOYSTICK, JOYSTICK2, JOYX, JOYY, MOUSE, POSY, RPOINT

POSY Graphics Function

Provide the y-coordinate of either the mouse cursor or the joystick cursor.

Syntax

POSY(*e*)

Remarks

This function returns the y-coordinate of the graphics cursor under the control of the control unit indicated by the value of *e*:

- 0 specifies a mouse
- 1 specifies the left joystick
- 2 specifies the right joystick

Examples

```
PRINT POSY(0)

 425
```

Associated Keywords

JOYSTICK, JOYSTICK2, JOYX, JOYY, MOUSE, POSX, RPOINT

PRINT Command

Write data to a specified output channel.

Syntax

PRINT [*~e/#e,*][*print list*]

Remarks

The PRINT (or ? abbreviation) command is used to output data to the specified writing area, or to the specified channel (0 for screen, 2 for printer, 11–127 for an open file). When *#e* is omitted, output goes to the current writing area.

print list is a list of numeric or string expressions, or both. The expressions should be separated by a semicolon, comma, space, or exclamation mark.

A number is written according to the following rules:

- a positive number (and zero) has a preceding space
- a negative number has a preceding minus sign
- numbers larger than 9999999, or smaller than 0.001 are written in scientific notation, where a number is followed by the letter e and a signed integer (the exponent). For example, 2.2e+06 represents 2200000
- each number has a space following it

String expressions must be enclosed in quotation marks. To output a quotation mark in a string, you can place two quotation marks together. For example:

```
PRINT "The ""RM Nimbus"""
The "RM Nimbus"
```

The layout of values depends on the punctuation in *print list.*

If you separate two expressions with a semicolon or a space, the second value is written immediately after the first one.

If you use a comma, the second value is written at the next print zone. The print zones are set so that data can be written in columns 15 positions apart.

An exclamation mark causes the next value to be printed on a new line.

When you put a semicolon or comma at the end of a *print list*, the first value of the next PRINT instruction is written on the same line, and the above spacing rules apply. Otherwise, the next PRINT instruction begins writing on a new line.

You can use PRINT alone or with an exclamation mark to give blank lines.

As well as numeric or string expressions, the TAB and SPC functions can be included in *print list* to control the column position at which the following value will be written.

You can use the WIDTH command to define the number of characters written before starting a new line. Before RM Basic writes a number, it checks that there is enough room on the line as specified by WIDTH. When the number will not fit, it is written on the next line. Strings can be split over lines.

Examples

```
PRINT "Next"! "line"; TAB(20); "Today"; SPC(5); 20

Next
line                    Today       20

? 43 * 76; "Chop" + "sticks" ! 43 * 36; "Bowls"

 3268 Chopsticks
 1634 Bowls
```

Associated Keywords

CURPOS, POS, PUT, SPC, TAB, WIDTH

PROCEDURE/ENDPROC Structure

Define a procedure.

Defining Syntax

PROCEDURE *name* [*var1*[, *var2*...]] [RETURN *exp3*[, *exp4*...]]
or
PROCEDURE *name* [*array1*[, *array2*...]]
:
:
Instruction(s)
:
:
ENDPROC

Calling Syntax

name [*exp1*[, *exp2*...]] [RECEIVE *var3*[, *var4*...]]
or
name [*array1*[, *array2*...]]

Remarks

To declare a procedure, start with the PROCEDURE header, defining the name of the procedure and its parameters. If you are making a GLOBAL declaration, this should be on the next line. The action of the procedure is defined by the instruction(s) on the following line(s) down to the next ENDPROC command.

To call the procedure, simply use *name* in the same way that you use a standard RM Basic command. The procedure will be executed and control will be passed back to the instruction following the one that called the procedure, when either an ENDPROC or a LEAVE command is executed.

You can give parameters to a procedure (*var1*, *var2*...). These are variables or array elements, each one accepting a value from the program to be used in the procedure. The values from the program are given in the procedure call (*exp*1, *exp2*...).

Arrays can be assigned to procedures (*array1*, *array2*) using the array reference system (see chapter 5).

Unlike a function, a procedure does not normally return a value. You can make procedures return values by using the RETURN and RECEIVE feature (see chapter 5). This allows parameters to work in the opposite direction.

Values for the procedure (*exp3*, *exp4*...) are accepted by variables or array elements (*var3*, *var4*...) in the procedure call.

Procedure declarations are usually placed at the end of a program.

Examples

```
700 REM This procedure produces a copy of a file.
710 PROCEDURE Copy_File Original$, Copy$
720 OPEN #11, Original$
730 CREATE #12, Copy$
740 All_Done% = FALSE
750 ON EOF #11 All_Done% := TRUE : GOTO 790
760 REPEAT
770   INPUT #11, x$
780   PRINT #12, x$
790 UNTIL All_Done%
800 CLOSE
810 ENDPROC
```

Associated Keywords

DEFINED, FUNCTION/ENDFUN, GLOBAL, LEAVE, PROCS

PROCS Command

Lists the names of procedures, functions, and subroutines.

Syntax

PROCS [~*e*/#*e*]

Remarks

This command writes all procedure names, defined functions and subroutine labels in the program currently in memory to the specified writing area or channel.

PROCS lists the line containing the start of the declaration or subroutine:

- the line number
- the type of declaration — procedure, function, or subroutine
- the name or label
- parameters

Examples

```
PROCS

2000 PROCEDURE Overdue Title$, Author$, Fine
5000 PROCEDURE Draw_House
7500 FUNCTION(Number%)
```

Associated Keywords

FUNCTION/ENDFUN, GOSUB, LVAR, PROCEDURE/ENDPROC, TRACE

PSAVE Command

Write a program file to disk in encoded format.

Syntax

PSAVE *e$*

Remarks

e$ is a string expression that represents the file specification.

The PSAVE command writes the program stored in memory to disk in encoded format, storing it with the specified filename.

If you do not put a file extension in your filename, .BAS is added automatically. If you do not want a file extension, add a full stop to the end of your filename.

Programs that are saved with the PSAVE command cannot be listed on screen when re-loaded, and cannot be edited. So make sure that you have a copy made with SAVE as well.

Examples

```
PSAVE "B:Stats2"
```

Associated Keywords

LOAD, SAVE

PUT Command

Write character(s) to a specified writing area or channel.

Syntax

PUT [~*e*/#*e*,] *print, list*

Remarks

print list is a list of numeric expressions (with values in the range 0 to 255), or string expressions, or both. Each expression must be separated from the next by a comma.

The contents of the list are output to the specified writing area or channel one after another.

A numeric expression is evaluated and the equivalent character from the current character set, or a control code, is output. See "Technical Data" in the Nimbus Owners Handbook.

A string expression is evaluated and the resultant string is written character by character.

Unlike the PRINT command, carriage return is not generated after all items of the list have been output.

Example

```
PUT 65, 38, 66 : REM Display A&B on the screen.
A&B
PUT #2, "Hello" : REM   Output to the printer.
```

Associated Keywords

PRINT

QUEUE Function

Provide the number of free spaces in the queue of notes of a voice channel.

Syntax

QUEUE(*e*)

Remarks

The QUEUE sound function returns the number of free spaces (maximum 6) in the queue of the voice specified by the value of *e* (0 to 3).

Voice 0 store the NOISE command; voices 1, 2 and 3 store notes issued by the NOTE command.

Examples

```
PRINT QUEUE(3)

 3
```

Associated Keywords

ENVELOPE, HOLD, NOISE, NOTE, PITCH, SOUND, TONE, VOICE

RAD Attribute

Set the angle measurement unit, or report its current setting.

Syntax

SET RAD *t*

ASK RAD *v*

Remarks

Unless the value of *t* is FALSE (0), SET RAD sets angle measurement in trigonometrical functions to radians. If the value of *t* is FALSE, the setting is degrees.

ASK RAD assigns to *v* either –1 (TRUE) when the angle measurement unit is radians, or 0 (FALSE) for degrees.

Examples

```
SET RAD TRUE
ASK RAD Setting1%
SET DEG TRUE
ASK RAD Setting2%
PRINT Setting1% ; Setting2%

-1  0
```

Associated Keywords

ATN, COS, DEG, SIN, TAN

READ Command

Read values from a DATA instruction and assign them to variables.

Syntax

READ *var1*[, *var2*....]

Remarks

This command reads a number of values from one or more DATA instruction(s) into numeric or string variables in strict order. The number of values in the DATA instruction(s) must be at least equal to the number of variables specified in the READ instruction(s).

A READ instruction assigns data to its variables in the order that data appears in the program. Subsequent READ instructions read the data that follows on from that read by the first. You can read numbers into string variables but you cannot read strings into numeric variables. Use the RESTORE command to reread data from any line number.

Examples

```
10 RESTORE
20 FOR I% = 1 TO 5
30   READ A(I%)
40 NEXT I%
50 DATA 55, 61, 48.4, 92, 55
```

```
10 DATA 25, 37
20 READ St$, Num : REM Reads "25" into St$ and 37 into Num
```

Associated Keywords

DATA, INPUT, RESTORE

REM Command

Insert explanatory comments in a program.

Syntax

REM *remark*

Remarks

remark is any sequence of characters up to the end of the line.

The REM command introduces a comment into the program. The comment is ignored when the program is running.

You should put comments and explanations into your program, as they will be invaluable if you want to modify the program at a later date.

Examples

```
10 REM ************************
20 REM Program      : BUILD
30 REM Author       : Susan Smith
40 REM Version      : 2.0c
50 REM Date         : 11/4/86
60 REM ************************
```

RENAME...TO Command

Rename a file stored on disk.

Syntax

RENAME *e1$* TO *e2$*

Remarks

e1$* and *e2$ are string expressions; their values represent filenames.

This command changes the name of a file from *e1$* to *e2$*. If a file extension is not specified, the .BAS extension is assumed.

Examples

```
RENAME "Program1" TO "Fastsort"


RENAME "DATA.BAK" TO "OLDDATA.TXT"
```

Associated Keywords

CHDIR, DIR, ERASE, FSPACE, LOOKUP, MKDIR, PATH$, RMDIR

RENUMBER Command

Renumber some or all of the lines of the program currently in memory.

Syntax

RENUMBER [*line1*[, [*line inc*][, *range*]]]

Remarks

The RENUMBER command renumbers a group of program lines. You can specify which lines are to be renumbered with *range*.

The first line of the renumbered group becomes line number 10 unless you specify a starting line number with *line1*. Subsequent lines have an increment of 10 unless you select a different value with *line inc*.

- RENUMBERenumbers the whole program — first line is 10 ai. increment is 10

- RENUMBER *line1*
  Renumbers the whole program — first line is *line1* and increment is 10

- RENUMBER [*line1*], *line inc*
  Renumbers the whole program — first line is *line1* (or 10) and increment is *line inc*

- RENUMBER [*line1*], [*line inc*], *range*
  Renumber the specified range of lines — first line is *line1* (or 10) and increment is *line inc* (or 10)

The range of lines to be renumbered is given by:

- *line2* TO
  All lines from old *line2* to the end of the program

- *line2* TO *line3*
  These lines and all between them

- TO *line3*
  All lines up to and including old *line3*

  The RENUMBER command changes references to the renumbered lines. There will be a warning if reference is made to non-existent lines. You cannot use RENUMBER to change the order in which instructions are executed.

Examples

```
LIST

 10 INPUT "Enter a name :"; Name$
 20 PRINT Name$
 30 GOSUB 40
 40 SUBROUTINE Names
:
140 RETURN

RENUMBER 200, , 40 TO 140
LIST

 10 INPUT "Enter a name :"; Name$
 20 PRINT Name$
 30 GOSUB 200
200 SUBROUTINE Names
:
300 RETURN
```

Associated Keywords

AUTO, DELETE, EDIT, LIST

REPEAT/UNTIL Loop Construct

Repeat the execution of instructions until a condition is fulfilled.

Syntax

REPEAT
:
Instruction(s)
:
UNTIL *t*

Remarks

The instructions following the REPEAT command are executed. Each time the UNTIL instruction is reached, *t* is evaluated. Unless its value is TRUE (non-zero), control returns to the instruction following the REPEAT statement.

The body of the REPEAT loop is always executed at least once.

Examples

```
10 REPEAT
20   INPUT "Please answer Y or N :", A$
30 UNTIL A$ == "Y" OR A$ == "N"
```

Associated Keywords

FOR/NEXT

RESTORE Command

Prepare to reread DATA instruction(s).

Syntax

RESTORE [*line*]

Remarks

line is the line number of a DATA statement in your program.

This command allows data to be reread from the specified line number. When a line number is not specified, data will be read from the first DATA instruction in the current procedure.

At the beginning of your program and after a RESTORE command with no line number, the first READ instruction will take data from the start of the first DATA instruction.

Examples

```
10 READ A, B, C
20 RESTORE
30 READ D, E, F
40 DATA 10, 20, 30
50 PRINT A, B, C, D, E, F

 10  20  30  10  20  30
```

Associated Keywords

READ, DATA

RESULT Command

Return from a defined function with its result.

Syntax

RESULT *exp*

Remarks

The RESULT command is used to return control from anywhere within a defined function. RESULT indicates what value is returned, so there must be at least one RESULT in every function declaration.

Examples

```
300 FUNCTION Sq (X)
310 Square := X * X
320 RESULT Square
330 ENDFUN
```

Associated Keywords

FUNCTION/ENDFUN

RESUME Command

Resume program execution at the same instruction.

Syntax

RESUME

Remarks

When a program has been interrupted, for example by typing <BREAK> or by an error, you can use the RESUME command to resume execution.

The program starts running again from the instruction at which it was interrupted.

Example

```
10 LET Z = 32 / A
20 PRINT Z
RUN

Trying to divide by zero
10 LET Z = 32 / A

LET A = 16
RESUME

 2

440 ON BREAK RESUME : REM Disable program interruption.
```

Associated Keywords

CONTINUE, ON BREAK, STOP

RETURN Command

Bring program execution back from a subroutine.

Syntax

RETURN

Remarks

When program execution has been transferred to a subroutine with the GOSUB command, RM Basic remembers where it came from. When a RETURN command is executed, control returns to that point and program execution continues from the instruction following that GOSUB.

A subroutine can be called from within another subroutine. In this case, RETURN passes control back to the instruction following the corresponding GOSUB.

Examples

```
500 SUBROUTINE Data_Entry
510 Num% = 1
520 REPEAT
530   INPUT "Enter data :", Entry(Num%)
540 UNTIL Num% = 10
550 GOSUB Calc_Mean
560 PRINT "The mean value is "; Mean
570 RETURN
```

Associated Keywords

GOSUB/SUBROUTINE, ON...GOSUB

RIGHT$ Function

Create a string of characters copied from another string, counting from its right end.

Syntax

RIGHT$(*e$*, *e*)

Remarks

The value of *e* specifies the number of characters to be copied from *e$*. If the value of *e* is greater than the length of *e$*, the entire string is returned.

Examples

```
10 Word$ := "ASTRONOMICAL"
20 B$ := RIGHT$(Word$, 7)
30 PRINT B$

NOMICAL
```

Associated Keywords

INSTR, LEFT$, LEN, MID$, STRING$

RMDIR Function

Erase an empty disk file directory.

Syntax

RMDIR *e$*

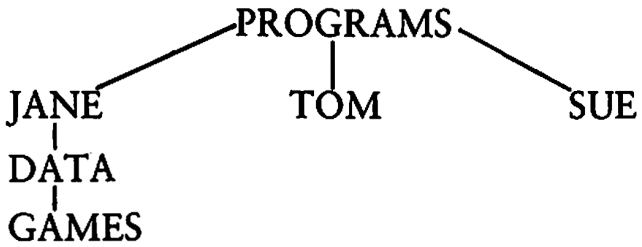
Remarks

e$ is a string expression; its value represents the pathname (see "Operating System Concepts" in the Nimbus Owners Handbook).

RMDIR is used to delete an empty directory from the MS-DOS directory structure. An empty directory does not contain any other directories or files. You cannot use RMDIR to remove the working directory — the one you are currently using.

Examples

Here is an example directory structure:



If the current working directory is PROGRAMS, and if you want to remove the empty GAMES directory, use:

```
RMDIR "JANE\DATA\GAMES"
```

Associated Keywords:

CHDIR, DIR, ERASE, FSPACE, LOOKUP, MKDIR, PATH$, RENAME...TO

RND Function

Generate a random number.

Syntax

RND (*e*)

Remarks

The RND function returns a random number. The value of *e* determines what will happen.

When the value is greater than 1, an integer in the range 1 to *e* is returned. When the value is between 0 and 1 a floating-point number is produced greater than or equal to 0 and less than 1. If the value of *e* is 0, the last number generated by RND is repeated.

The random number generator produces a list of numbers. When you use RND, random numbers are selected sequentially from this list. You can reseed the generator — start again from a particular point in the list — by making any of the above values (except 0) negative.

Examples

```
10 Random 10 : Random 1 : Random 0 : END
20 PROCEDURE Random X
30 FOR I% = 1 To 4
40   PRINT RND(X); SPC(2);
50 NEXT I%
60 PRINT
70 ENDPROC
```

```
 5     8     9     5
 0.512655     0.833125     0.140957     0.574636
 0.574636     0.574636     0.574636     0.574636
```

RPOINT Graphics Function

Provide the brush colour of a point on the graphics screen.

Syntax

RPOINT(*e1*, *e2*)

Remarks

The values of *e1* and *e2* specify the x and y coordinates of a point on the screen, based on the current coordinate system. The current brush colour of that point is returned.

Examples

```
A% := RPOINT(50, 100)
PRINT A%

 2
```

Associated Keywords

BRUSH, POINTS, POSX, POSY

RUN Command

Begin execution of a program.

Syntax

RUN [*line*]

Remarks

When this command is issued, the following events occur:

- All local and global variables are forgotten (see chapter 5)
- Space is reserved for string variables
- The disk system is reset
- Execution of the program currently in memory begins at the first line or, if a line number is specified, at that line number.

Associated Keywords

CLEAR, FREE, NEW

SAVE Command

Write a program file to disk.

Syntax

SAVE *e$*

Remarks

e$ is a string expression; its value represents the file specification.

The SAVE command writes the program stored in memory to disk, storing it with the specified filename. The program is listed to disk exactly as it is on screen.

If you do not put a file extension in your filename, .BAS is added automatically.

If you do not need a file extension, add a full stop to the end of your filename.

Examples

```
SAVE "Draw1"

SAVE "B:FRED."
```

Associated Keywords

LOAD, LOADGO, MERGE, MERGEGO, PSAVE

SET Command

Select an attribute and set up its parameters.

Syntax

SET *attrib e1, e2, ...*

Remarks

The attribute specified by *name* is selected using the parameter(s) indicated by the values of *e1*(, *e2* ...). *attrib* can be one of the following attributes:

BORDER	ENVELOPE	MOUSE	POINTS STYLE
BRUSH	FILL STYLE	ORIGIN	RAD
CHAROVER	FKEY	OVER	SOUND
CHARSET	FLOOD EDGE	PATTERN	TIME
COLOUR	HOLD	PAPER	TONE
CURPOS	JOYSTICK	PEN	UNDERLINE
CURSOR	JOYSTICK2	PLOT DIRECTION	VOICE
DATE	KEYREP	PLOT FONT	WARN
DEG	LINE STYLE	PLOT SIZE	WRITING
DRAWING	MODE	POINTS SIZE	

With SET FKEY the parameters following *e1* can be numeric expessions, string expressions, or both.

Examples

```
SET MODE 40
SET BRUSH 3
SET BORDER 6
```

Associated Keywords

ASK

SGN Function

Determine whether a number is positive, zero, or negative.

Syntax

SGN(*e*)

Remarks

This function returns:

- 1 when the value of *e* is positive
- 0 when the value of *e* is zero
- –1 when the value of *e* is negative.

Examples

```
ON SGN(x) + 2 GOSUB 500, 1000, 1500
```

Associated Keywords

ABS, INT

SIN Function

Calculate the sine of an angle.

Syntax

SIN(*e*)

Remarks

This function returns the sine of the angle, *e*, expressed in degrees or radians, depending on the current angle measurement unit. This is set with SET DEG or SET RAD.

Examples

```
SET DEG TRUE
PRINT SIN(90)

 1
```

Associated Keywords

ATN, COS, DEG, RAD, TAN

SLICE Graphics Command

Draw slices of a circle (pie-slices) on the screen.

Syntax

SLICE *e1*, *e2* TO *e3*, *coordinate list* [*option list*]

Remarks

SLICE is used to draw a part of a circle around each centre point specified in *coordinate list.*

e1 is a numeric expression; its value represents the radius (in y axis units) of the circle of the slice.

e2 and *e3* are numeric expressions; their values represent angles, in degrees or radians depending on the current setting of SET DEG.

The value of *e2* indicates the angle at which drawing starts; the value of *e3* specifies the angle at which to stop.

Straight lines are drawn from the centre to the perimeter at these points, and the enclosed area is filled. The 12 o'clock position is angle 0 and the slice is drawn in a clockwise direction.

Unless you specify the relevant option in the SLICE command, the current settings of brush colour, filling style, and drawing mode are used (either the default settings or those set with the SET *attrib* command).

The following options, giving temporary change, can be specified after *coordinate list*:

Option	*Action*	*Syntax*
BRUSH	selects the brush colour	see BRUSH graphics attribute
STYLE	selects the filling style	see FILL STYLE graphics attribute
OVER	selects the drawing style	see OVER graphics attribute

Examples

```
SLICE 100, 90 TO 360, 150, 150 STYLE 2, 3, 1
```

Options

BRUSH, OVER, [FILL] STYLE

Associated Keywords

AREA, CIRCLE, FLOOD, LINE, PLOT, POINTS

SOUND — Sound Attribute

Enable or disable the use of sound, or report whether or not it is initiated.

Syntax

SET SOUND *t*

ASK SOUND *v*

Remarks

SET SOUND is used to enable or disable sound production with the Nimbus speaker. When the value of *t* is TRUE, sound is enabled. When *t* is FALSE, sound is disenabled. The default setting is TRUE.

ASK SOUND returns the current setting in *v*.

Examples

```
SET SOUND TRUE : REM Enable sound production
```

Associated Keywords

ENVELOPE, HOLD, NOISE, NOTE, PITCH, QUEUE, TONE, VOICE

SPC Function

Space separator in a PRINT instruction.

Syntax

SPC(*e*)

Remarks

This function can only be used in PRINT instructions. SPC(*e*) causes *e* spaces to be displayed between the two items that it separates. Remember that numbers have leading and trailing spaces.

Examples

```
PRINT "HERE"; SPC(10); "THERE"

HERE          THERE


PRINT 20; SPC(5); 30

 20         30
```

Associated Keywords

CURPOS, PRINT, TAB

SQR Function

Calculate the square root of a number.

Syntax

SQR(*e*)

Remarks

e is a numeric expression that must be positive or zero.

This function returns the square root of the value of *e*.

Examples

```
PRINT SQR(23)
 4.795832
```

STOP Command

Stop program execution.

Syntax

STOP

Remarks

This command can be used anywhere in a program to stop execution.

Using the CONTINUE command, you can start running the program again at the instruction that follows the STOP command.

Associated Keywords

CONTINUE

STRING$ Function

Create a string of multiple copies of another string.

Syntax

STRING$(e$, *e*)

Remarks

The value of *e* specifies the number of copies of *e$* that will be returned by this function.

Examples

```
Letters$ := STRING$("ABC", 3)
PRINT Letters$

ABCABCABC

Star$ := STRING$("*", 15)
PRINT Star$

***************
```

Associated Keywords

INSTR, LEFT$, LEN, MID$, RIGHT$

STR$ Function

Convert a number into the equivalent string representation.

Syntax

STR$(*e*)

Remarks

This function returns a string representing the value of *e*. If the value is positive, the string contains a leading space.

Examples

```
10 LET A$ = STR$(4.5)
20 PRINT A$

 4.5
```

```
10 IF LEN(STR$(Value)) > 3 THEN PRINT "Too Big"
20 REM STR$ converts Value to a string.
30 REM Then LEN gives the number of digits.
```

Associated Keywords

ASC, CHR$, HEX$, VAL

[FILL] STYLE Graphics Attribute

Select a style for filling areas, circles, or slices on the graphics screen, or report the current setting.

Syntax

SET FILL STYLE *e1*[, *e2*[, *e3*]]

AREA/CIRCLE/FLOOD/SLICE *coordinate list* STYLE *e1*[, *e2*[, *e3*]]

ASK FILL STYLE *v1*[, *v2*[, *v3*]]

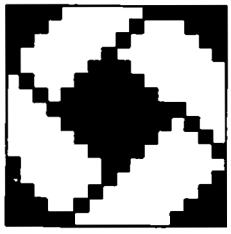
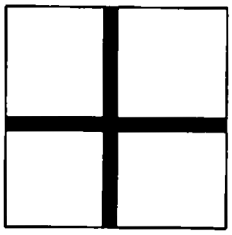
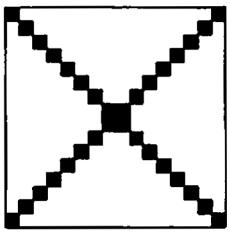
Remarks

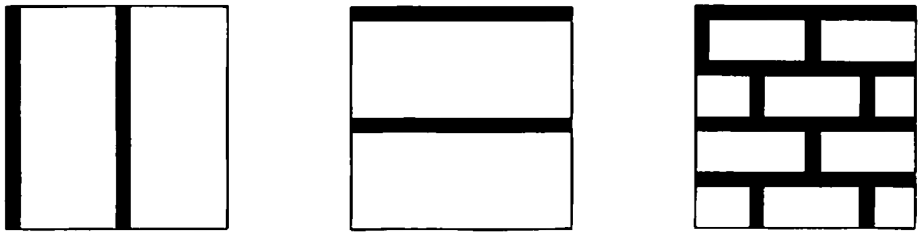
The SET FILL STYLE command selects the filling style for graphics areas. The STYLE option, in a graphics area-drawing command, temporarily selects the filling style for the duration of the command.

The value of *e1* indicates what style:

1. solid (default setting), pure or dithered colour
2. hatched pattern (see below)
3. hollow — only the area border is drawn

When 2 is selected, *e2* must be specified. Its value, in the range 0 to 5, indicates which of the six hatching patterns will be used:



These are distributions of two colours in a repeating 16 pixel by 16 pixel pattern.

The hatching pattern will be drawn with the current brush colour and a second brush colour, indicated by the value of *e3*. If you want to leave the colour beneath the second colour unchanged, then either specify –1 or do not indicate a second colour.

ASK FILL STYLE returns the current style setting in *v1*, *v2*, and *v3*.

Examples

```
SET FILL STYLE 2, 1, 4

10 REM Draw a grid.
20 AREA 0, 0; 200, 0; 200, 200; 0, 200 STYLE 2, 3
30 AREA 0, 0; 200, 0; 200, 200; 0, 200 STYLE 2, 4
```

Associated Keywords

AREA, BRUSH, CIRCLE, FLOOD, SLICE

[LINE] STYLE Graphics Attribute

Select a style for drawing lines on the graphics screen, or report the current setting.

Syntax

SET LINE STYLE *e*[, *exp*]

LINE *coordinate list* STYLE *e*[, *exp*]

ASK LINE STYLE *v*[, *var*]

Remarks

The SET LINE STYLE command selects the line style. The STYLE option, in a LINE command, temporarily selects the line style for the duration of the command.

The value of *e* indicates what style:

1	solid (default setting), pure or dithered colour	———
2	dashed	- - - - -
3	dotted	. . . . .
4	dashed-dotted	-.-.-.-.-
5	dashed (irregular)	– – – – –
6	defined style	see below

If you select 6, the value of *exp* should be a string containing dots and spaces that define your style. The string should be 16 characters long. If less, it is padded with spaces; if more, the excess is ignored.

If you select either 2, 3, 4, or 5, the value of *exp* should be a number representing the second colour for drawing between dots and dashes. If you do not want a second colour, do not specify one, or specify –1.

ASK LINE STYLE assigns the current style in *v* and *var*.

Examples

```
SET LINE STYLE 3, -1
```

```
SET LINE STYLE 6, ". . . .... . . ."
```

Associated Keywords

LINE, [FILL] STYLE, [POINTS] STYLE

[POINTS] STYLE Graphics Attribute

Select a style for drawing points on the graphics screen, or report the current setting.

Syntax

SET POINTS STYLE *e*[, *ov*]

POINTS *coordinate list* STYLE *e*[, *ov*]

ASK POINTS STYLE *v*

Remarks

The SET POINTS STYLE command selects the point-drawing style. The STYLE option, in the POINTS command, temporarily selects the point-drawing style for the duration of the command.

The value of *e* indicates the style:

1 . (default setting)
2 +
3 *
4 o
5 X
6 defined style

When 6 is selected, an object variable, *ov*, containing the style definition, must be specified.

ASK POINTS STYLE assigns the current points style in *v*.

Examples

```
SET POINTS STYLE 3
```

Associated Keywords

POINTS, [POINTS] SIZE

TAB Function

Tab separator in a PRINT instruction.

Syntax

TAB(*e*)

Remarks

This function may only be used in PRINT instructions.

TAB(*e*) causes the cursor to move to column *e* before the next item is displayed. Columns are numbered starting from one.

If the value given for the TAB function indicates a position to the left of the current cursor position, the cursor is moved to the next line.

Examples

```
PRINT "NAME"; TAB(20); "PHONE No."

NAME                    PHONE No.

10 REM Display second number in the 10th column
20 PRINT 5; TAB(9); 6

 5         6
```

Associated Keywords

CURPOS, PRINT, SPC

TAN Function

Calculate the tangent of an angle.

Syntax

TAN(*e*)

Remarks

This function returns the tangent of the angle, *e*, expressed in degrees or radians. The angle measurement unit is set with SET DEG or SET RAD.

Examples

```
SET RAD TRUE
PRINT TAN(0.5)

 0.546302
```

Associated Keywords

ATN, COS, DEG, RAD, SIN

TIME Attribute

Update or provide the operating system time.

Syntax

SET TIME *e1*[, *e2*[, *e3*]]

ASK TIME *v1*[, *v2*[, *v3*[, *v4*]]]

Remarks

SET TIME updates the computer's internal clock. The values of *e1*, *e2*, and *e3* specify hours, minutes, and seconds; zero is assumed for any omitted arguments.

ASK TIME returns the current time in *v1*, *v2*, *v3*, and *v4*. The value returned in *v4* is hundredths of a second.

Examples

```
SET TIME 11, 25, 30

ASK TIME Hour%, Min%, Sec%, Hun%
PRINT Hour%; Min%; Sec%, Hun%
 11  25  57  54
```

Associated Keywords

DATE, DATE$, TIME$

TIME$ Function

Report the operating system time.

Syntax

TIME$

Remarks

The time is returned in the form hh:mm:ss.

Examples

```
PRINT TIME$

14:34:27
```

Associated Keywords

DATE, DATE$, TIME

TONE Attribute

Select the control of sound pitch by either NOTE or NOISE, or report the current setting.

Syntax

SET TONE *t*

NOTE *e1*[to *e2*][, *e3*[, *e4*]] TONE *t*

ASK TONE *v*

Remarks

The SET TONE *t* command selects whether sound produced by the NOTE command is a specified pitch or a fuzzy sound. The TONE *t* option, in the NOTE command, temporarily selects the type of sound production for the duration of the command.

When *t* is FALSE (0), fuzzy sound is produced. The default setting is TRUE.

When TONE is set FALSE, the NOISE command can be used to change the pitch of the fuzzy sound.

ASK TONE returns the current TONE setting in *v*.

Examples

```
SET TONE FALSE
NOTE 148, 100
```

Associated Keywords

ENVELOPE, HOLD, NOISE, NOTE, PITCH, SOUND, QUEUE, VOICE

TRACE Command

Trace the execution of program instructions.

Syntax

TRACE [~*e*/#*e*,] *t*

Remarks

The TRACE command enables (when *t* is TRUE) or disables (when *t* is FALSE) the program trace feature.

When enabled, TRACE lists each line number (enclosed in angle brackets) to the specified writing area or channel before execution of the instruction(s) in that line.

Examples

```
10 A := 5
20 FOR I% = 1 TO 4
30    B% = A * I%
40    PRINT A; B%; I%
50 NEXT I%
TRACE TRUE

RUN

<10><20><30><40> 5   5   1
<50><30><40> 5   10   2
<50><30><40> 5   15   3
<50><30><40> 5   20   4
<50>:
```

Associated Keywords

BOUNDS, DEFINED, LVAR, PROCS

TRUE Function

It has a value of –1.

Syntax

TRUE

Remarks

Comparison operations return either TRUE or FALSE.

Examples

```
IF X < 10 THEN Flag := TRUE
```

Associated Keywords

FALSE

UNDERLINE Attribute

Set the character underlining option, or report its current setting.

Syntax

SET UNDERLINE *t*

ASK UNDERLINE *v*

Remarks

Unless the value of *t* is FALSE (0), SET UNDERLINE sets all displayed characters to be underlined. If the value of *t* is FALSE, characters are written normally. This is the default.

SET UNDERLINE TRUE causes all space characters to be underlined. Consequently, if you issue a CLS command, the screen will be filled with underline marks.

ASK UNDERLINE assigns to *v* either TRUE when the underline option is set, or FALSE for normal output.

Examples

```
10 CLS : SET UNDERLINE TRUE
20 FOR I% = 10 TO 50 STEP 10
30   SET CURPOS I%, 10 : PRINT "Hello"
40 NEXT I%
50 SET UNDERLINE FALSE
```

Hello Hello Hello Hello Hello

Associated Keywords

CHARSET, CHR$, PRINT, PUT

VAL Function

Return the numerical value of a string.

Syntax

VAL(*e$*)

Remarks

This function returns the numerical value of the string, *e$*. Any leading spaces, and any characters after the numeric part (as defined in chapter 3) of the string are ignored.

Zero is returned when the first character (ignoring spaces) is not the start of a number.

The READ and INPUT commands use this function when they are assigning data to numeric variables.

Examples

```
PRINT VAL("38 Church St")' VAL("-1.2")' VAL("&1a)

 38
-1.2
 26

PRINT VAL ("Richard is 21 today")

 0
```

Associated Keywords

ASC, CHR$, HEX$, STR$

VERSION Function

Provide RM Basic and operating system version numbers.

Syntax

VERSION$

Remarks

VERSION$ returns the RM Basic version number, the operating system name, and its version number.

Examples

```
PRINT VERSION$

1.1B  MS-DOS 3.1
```

VOICE — Sound Attribute

Select a voice for sound output, or report the currently selected voice.

Syntax

SET VOICE *e*

NOTE *e1*[TO *e2*][, *e3*[, *e4*]] VOICE *e*

ASK VOICE *v*

Remarks

The SET VOICE *e* command selects the voice channel for sound output. The VOICE *e* option, in a NOTE command, temporarily selects the voice channel for the duration of the command.

The voice queue into which NOTE will send its output is selected by the value of *e* in the range 1 to 3. The default is 1.

ASK VOICE returns the currently selected voice channel in *v*.

Examples

```
NOTE PITCH (0, 0), 100 VOICE 1
```

Associated Keywords

ENVELOPE, HOLD, NOISE, NOTE, PITCH, QUEUE, SOUND, TONE

WARN Attribute

Enable or disable warning message display, or report the current setting.

Syntax

SET WARN *t*

ASK WARN *v*

Remarks

SET WARN is used to select whether or not warning messages (see appendix A) will be displayed.

When the value of *t* is FALSE (0), warning messages are disabled; when the value of *t* is TRUE (–1), warning messages are displayed.

ASK WARN returns the current setting in *v*.

Examples

```
SET WARN FALSE : REM Stop warning display.
```

WIDTH Command

Set the output line length for a channel.

Syntax

WIDTH [#*e1*,] *e2*

Remarks

e1 is a numeric expression. Its value specifies a channel number. When *e1* is omitted, channel 0 is assumed.

The WIDTH command sets the line length to the value of *e2* (15 to 249). This is the number of characters that can be written to a line before a new line is started.

Before RM Basic writes a number with the PRINT or PUT commands, it checks that there is enough room on the current line.

A number that will not fit is written on the next line. However, no such check is made for strings before they are written, and they will wrap around.

When you start RM Basic, line length is not defined. WIDTH 0 returns you to this state.

Examples

```
WIDTH 20
PRINT "Saturday, sixth November"

Saturday, sixth
November
```

Associated Keywords

PRINT

WRITING Attribute

Select or define a screen writing area, or report the currently selected writing area.

Syntax

SET WRITING *e1*[TO *e2*, *e3*; *e4*, *e5*]

ASK WRITING *v1*[, *v2*, *v3*, *v4*, *v5*]

Remarks

A writing area (scrolling area) is an area of the screen in which characters are displayed. Cursor movement (with CURMOVE and CURPOS) and character deletion (with CLL and CLS) are limited to this area, when selected; scrolling takes place within the limits of the area.

You can select up to 9 writing areas with SET WRITING. A single argument, *e1*, is specified. Its value indicates which area:

0 specifies full screen (default setting)
1 to 9 specifies the writing area

You define a writing area with *e1*, *e2*, *e3*, *e4*, and *e5*. These values represent:

e1 area number (1 to 9)
e2 left-most column number
e3 top row number
e4 right-most column number
e5 bottom row number

When undefined, writing areas 1 to 9 are full screen areas.

In commands that write to the screen, such as PRINT, PROCS and LIST, you can specify the writing area. For example:

```
PRINT ~3, "This is writing area 3"
```

ASK WRITING assigns the selected writing area to *v1* and, when you specify *v2*, *v3*, *v4*, and *v5*, the limiting rows and columns of that area are assigned to them.

Examples

```
SET WRITING 6 : REM Selecting writing area 6.

SET WRITING 3 TO 10, 10, 20, 40 : REM Define writing area 3
```

Associated Keywords

CLL, CLS, [CUR]MOVE, CURPOS, HOME.

XOR Logical Operator

Logical operation on two expressions.

Syntax

e1 XOR *e2*

Remarks

XOR gives the result of bit by bit combination (see appendix B) of two integers or comparisons.

In control constructs, XOR combines the resuls of two comparisons. For example:

```
IF A = 3 XOR B = 4 THEN STOP
```

The instruction following THEN is executed if either A=3 or B=4 but not if both are true.

XOR is used in numerical expressions to logically combine two integer operands:

```
LET A% := 5 XOR 6
```

This assigns 3 to A%.

Associated Keywords

AND, NOT, OR

Appendix A
Error Messages and Warnings

Introduction

RM Basic has understandable and informative error messages that help you to correct program mistakes (this is known as *debugging*). Also, it makes error detection easy by displaying a line with an arrow symbol pointing to the position that was reached when the error was detected. The pointer does not always indicate exactly where the error occurred, but it gives you a very good idea.

For example, the following line:

```
10 FOR = 1 TO 10
```

will give the following error message when the program is run:

```
Numeric variable needed
10 FOR = 1 TO 10
```

If you enter this line in direct mode:

```
A := "Fred"
```

the following error message is displayed as soon as you press <ENTER>:

```
Numeric expression needed
A := "Fred"
```

When errors are detected, execution stops. You can use the EDIT command to recall the program line for correction, or you can re-type the line.

Error Commands

You can prevent the occurrence of an error from stopping your program by using the ON ERROR command. You can control what happens when an error occurs and, for instance, transfer execution to another part of the program to deal with the problem.

You can test errors with the ERL, ERR, and ERR$ functions (see the Keywords section).

Warning Messages

There are three errors that do not stop program execution. When this type of error is detected, a warning message is displayed on the screen and you can decide whether or not to let execution continue.

Error messages

All the RM Basic error messages and warning messages are listed below (including error numbers in brackets), with explanations where necessary. The messages are grouped into six categories:

Syntax
Commands
Files and Channels
Graphics
Size or Range Problems
Procedures
Warnings

Syntax

`Comma separator is needed` (1)

`Opening bracket is needed` (2)

`Closing bracket needed` (3)

`Semicolon separator needed` (4)

`Unknown value stack item` (5)

`Variable name is needed` (6)

`Array variable is wrong` (7)
An array variable is not allowed as a control variable in FOR/NEXT constructs.

`Numeric variable needed` (8)

`TRUE/FALSE value needed` (9)

`Number or string needed` (10)

`Invalid expression found` (11)

`Numeric expression needed` (12)

`String expression needed` (13)

`Expression too complicated` (14)
The expression is too long or complicated and there is insufficient memory space to handle it. Try to break it into smaller expressions.

`Wrong number of subscripts` (15)

`Array needed (no subscript)` (17)

`Specified line not found` (18)

`Line number/label needed` (19)

`Positive value required` (69)
SPC and TAB accept only a positive argument.

`End of instruction expected` (77)
An end of line code or a colon separator is expected at the end of the instruction. Instead, an illegal character has been found. This usually means that too many arguments have been supplied.

`Maximum arguments exceeded` (78)

`Object variable expected` (84)

`Arrays not of same type` (85)

`No such level` (96)

Commands

`Array already dimensioned` (16)

`Unknown command/procedure` (20)
The entered text is not recognised as an RM Basic command or defined procedure name.

`Function/array not found` (21)
The defined function name or array name is not recognised.

`Bad argument for function` (22)
With built-in RM Basic functions, such as COS and LEFT$, the specified argument is not allowed:

- out of range
- a number with the wrong sign
- a number instead of a string
- a string instead of a number

`WIDTH specification wrong` (23)
The specified line length is not allowed.

`Too many GOTOs to renumber` (24)

`GOTO or GOSUB is missing` (25)
A control instruction , such as ON EOF, or a jump instruction, such as ON *e*, does not have a GOTO or GOSUB command following it.

`TO is needed before value` (26)
The TO keyword is missing in either a FOR instruction or in a command that specifies a range of line numbers – for example, RENUMBER.

`THEN with no matching IF` (27)
The THEN instruction is not preceded by an IF instruction.

`RETURN without any GOSUB` (28)
A RETURN instruction needs a previous unmatched GOSUB instruction.

`No error to continue from` (29)

`UNTIL without any REPEAT` (30)
An UNTIL instruction needs a previous unmatched REPEAT instruction.

`NEXT without matching FOR` (31)
The NEXT instruction does not have a corresponding FOR instruction, or the variable in the NEXT instruction does not correspond with any variable in a for instruction.

`Too many FOR/REPEAT/GOSUB` (32)

`Wrong SET/ASK attribute` (33)
A SET, ASK, graphics, or sound instruction is not followed by a valid attribute name.

`Unknown SET/ASK attribute` (34)
The specified attribute is not recognised.

`Unable to read excess DATA` (49)

`No more DATA to be read` (50)

`Writing area inappropriate` (51)

`Envelope number is wrong` (52)

`Variable used as a local` (74)
A variable that has already been used as a local variable cannot be declared global.

`Interrupted by BREAK key` (76)
You can continue program execution with RESUME or CONT.

`Configuration command unknown` (97)

`Error in loading extension` (98)

`Not an extension file` (99)

Files and Channels

`Directory cannot be found` (35)

`Unable to remove directory` (36)

`Unable to create directory` (37)

`Exact filename is needed` (38)
Wildcard characters (* and ?) cannot be used.

`Unable to rename the file` (39)
The named file cannot be found, or the new name already exists.

`Unable to erase the file`	(40)
`Named channel is already in use`	(41)
`Unable to open named file` Either the file cannot be found or it does not exist.	(42)
`Channel not open for input`	(43)
`Channel not open for output`	(44)
`Wrong channel number used`	(45)
`Reading past end of file`	(47)
`Too many files open`	(89)
`File specified invalid or doesn't exist`	(90)
`No more space available on disk`	(91)
`File in use or name is a Dir/Vol`	(92)
`File operation failure`	(93)
`Directory full`	(94)
`Unable to close file`	(95)
`Filename is a directory`	(100)
`Disk is read only`	(101)
`Drive not ready`	(102)

Graphics

`Point coordinates needed` (53)
The x-coordinate has not been specified.

`A coordinate pair needed` (54)
The y-coordinate has not been specified.

`Coordinate is out of range` (55)

`Fill area too complicated` (56)

`Bad FLOOD start coordinates` (57)

`Mouse or joystick problem` (58)

`General graphics failure` (59)

`Parameter not understood` (60)

`Coordinate array is wrong` (61)

Size or Range Problems

`All available memory used` (62)

`Entered line is too long` (63)

`Number not in allowed range` (64)

`Line number out of range` (65)

`Length of string too great` (66)

`String offset out of range` (67)
The numeric argument, indicating the character count in a string function, is inappropriate.

`Step value not large enough` (68)

`Trying to divide by zero`	(70)
`Empty string not allowed`	(71)
`Array subscript is wrong`	(73)

Procedures

`Procedure exit without call`	(72)
`RECEIVE variables needed`	(79)
`Not enough parameters`	(80)
`Function exit without call`	(81)
`Need RESULT to exit function`	(82)
`Cannot execute definition`	(83)
`Name of definition required`	(86)
`End of definition expected`	(87)
`Function nesting too deep`	(88)

Warnings

`Named file already exists`
This is followed by the prompt:
`Abort command? (Y/N):`
	(46)

`Excess input data ignored`	(48)
`Variable without any value`	(75)

Appendix B
Logical Matters

This appendix explains logical, or bit-by-bit, operations that are used by the AND, OR, NOT, and XOR logical operators.

First bit	*Second bit*	*AND*	*OR*	*XOR*	*NOT*	
1	1	1	1	0	1	0
1	0	0	1	1	0	1
0	1	0	1	1		
0	0	0	0	1		

To understand bit-by-bit operation, you need to know about the binary system of numbers. The hexadecimal system is also explained.

The Binary System

We are used to the decimal system that has a base of 10. This indicates that ten symbols are used: 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9.

When we write a decimal number, we know that the position of each symbol is important. For example:

Number of thousands	*Number of hundreds*	*Number of tens*	*Number of ones*
1	9	8	5

Each position represents a power of 10.

Computers use the binary system with a base of 2. It uses only two symbols — 0 and 1. Once again, the position of each symbol is important:

Number of eights	*Number of fours*	*Number of twos*	*Number of ones*
1	0	1	1

In this system, each position signifies a power of 2. The example is the binary representation of 11 in decimal.

Each symbol of a binary number is known as a bit, so 1011 is a 4-bit number. RM Basic integers are stored in binary form as 16-bit numbers. Consequently, 1985 (decimal) is stored as 0000 0111 1100 0001 — 1024 + 512 + 256 + 128 + 64 + 1 by position.

The Hexadecimal System

You can split large binary numbers into groups of four bits. Each group has a volume of 0 to 15. A shorthand way of representing these groups is to use the hexadecimal system.

The hexadecimal number system uses sixteen digits — the figures 0 to 9 and the letters A to F. You count up to 15 before making a new column:

Hexadecimal	*Decimal*	*Hexadecimal*	*Decimal*
0	0	10	16
1	1	11	17
2	2	12	18
3	3	13	19
4	4	14	20
5	5	15	21
6	6	16	22
7	7	17	23
8	8	18	24
9	9	19	25
A	10	1A	26
B	11	1B	27
C	12	1C	28
D	13	1D	29
E	14	1E	30
F	15	1F	31

A sixteen digit binary number is represented by four hexadecimal digits:

Binary	0000	0111	1100	0001
Hexadecimal	0	7	C	1

Negative Numbers

We indicate a negative number by placing a minus symbol in front of it. The computer cannot do this, and another method has to be used.

The leftmost bit of the 16-bit binary number is used to represent the sign of that number:

- when the leftmost bit is 0, the number is positive
- when the leftmost bit is 1, the number is negative

For example:

0000 0000 0000 1011 is a positive number

1000 0000 0000 1011 is a negative number

The decimal value of the negative number is not –11 as you would expect.

A negative binary number is obtained by subtracting its absolute value from zero. For example, to obtain binary — 1 you subtract binary 1 from binary 0:

```
0000 0000 0000 0000    (in the binary system, 0
0000 0000 0000 0001    minus 1 is 1 and borrow 1)
---------------------
1111 1111 1111 1111
```

This means that 1111 1111 1111 1111 is a small negative number (–1) and 1000 0000 0000 1011 is a large negative number (–32757).

True and False

When reading this manual, you will have noticed that TRUE and FALSE are used in RM Basic to control what will happen.

For example, the instruction:

```
SET DEG TRUE
```

selects degree angle measurement. Radians are selected by:

```
SET DEG FALSE
```

When a comparison operator is used in an expression, the result is either TRUE or FALSE.

For example, in an instruction of this form:

```
IF A% = 10 THEN PRINT "Bingo" ELSE PRINT "Hard luck
```

the A% = 10 comparison will return TRUE when the value of A% is 10, and Bingo will be displayed. The comparison will return FALSE when the value of A% is not 10, and the instruction after ELSE will be executed.

The computer representations of TRUE and FALSE are the 16-bit binary forms of –1 and 0:

1111 1111 1111 1111 is TRUE
0000 0000 0000 0000 is FALSE

These values are chosen because they are the inverse of each other.

Bit-by-bit Operations

You saw in Chapter 3 how two expressions can be combined by a logical operator. For example:

```
IF A% = 10 AND B% = 20 THEN PRINT "Bingo"
```

First the two comparison expressions are evaluated and, in each case, the result is either TRUE or FALSE.

Then the AND logical operator performs a bit-by-bit comparison of the two results. This means that each of the 16 bits of the first result is compared with the corresponding bit of the second result to give a new 16-bit number.

The AND Rules

Each logical operator has different rules for comparing bits. The AND rules are:

AND logical operator

first bit	second bit	resultant bit
1	1	1
1	0	0
0	1	0
0	0	0

So, if A% is 10 (giving TRUE) and B% is 20 (giving TRUE), the numbers for bit-by-bit comparison are:

1111 1111 1111 1111

and

1111 1111 1111 1111

Each of the 16 bits of the first result (1) is compared with each corresponding bit of the second result (1). From the table of rules, when both bits are 1 the result is 1. Consequently the resultant number is:

1111 1111 1111 1111

This is TRUE and Bingo is displayed.

You can see from the AND bit-by-bit rules that if one or both of the results had been FALSE, the result of the AND operation would have been FALSE.

In an instruction such as:

```
B% := 4 AND 2
```

bit-by-bit comparison is used on the 16-bit binary representations of 4 and 2:

0000 0000 0000 0100

0000 0000 0000 0010

From the AND table of rules, the result is:

0000 0000 0000 0000

So 0 is assigned to B%

Other Rules

The OR logical operator has the following rules for comparing bits:

OR logical operator

first bit	second bit	resultant bit
1	1	1
1	0	1
0	1	1
0	0	0

In an instruction such as:

```
B% := 4 OR 2
```

the result of bit-by-bit comparison is:

0000 0000 0000 0110

So 6 is assigned to B%.

The NOT logical operator is a unary operator. Instead of comparing two numbers, it takes one number and inverts each bit.

NOT logical operator

number bit	resultant bit
1	0
0	1

For example:

```
A% := 0
B% := NOT A%
```

All the zeros of A% are inverted by the NOT operator to give –1, which is assigned to B%.

The XOR logical operator has the following rules for comparing bits:

XOR logical operator

first bit	second bit	resultant bit
1	1	0
1	0	1
0	1	1
0	0	0

In an instruction such as:

```
B% := 4 XOR 2
```

the result of bit-by-bit comparison is:

0000 0000 0000 0111

so 7 is assigned to B%

XOR Drawing and Writing

The XOR mode of drawing or writing on the screen works by the XOR operator's rules of bit-by-bit comparison.

For example, when one 4-bit number, 1011, is combined with a second 4-bit number, 1100, by XOR rules, the result is 0111.

When the result is combined with the second number, 1100, by XOR rules, the result is the first number, 1011.

This means that when you cover what is on the screen using XOR drawing or writing, you can uncover it by using XOR mode again with the same covering instruction.

Appendix C
Configuration Files

RM Basic is supplied with fixed settings of attributes (flags) and with no machine code extensions. If you want to configure RM Basic to your own requirements, you can produce a configuration file with a .BCF extension. This file automatically loads machine code extensions and it can be used to change the default settings of RM Basic.

When you type BASIC<Enter> to load RM Basic, the default configuration file (RMBASIC.BCF) is read. You can specify that your own configuration file is read by entering:

`BASIC` *file*`.BCF<Enter>`

where *file* is the name of your file. You can put *file* with no file extension. The .BCF extension will be assumed.

You create a configuration file (using a text editor, the MS-DOS COPY utility, or RM Basic itself) containing the following lines:

`!EXT` *extfile*
where *extfile* is a machine code extension filename. You can have up to 10 extension files.

`!BCF` *confile*
where *confile* is a configuration filename. The configuration file that you are creating will look for the named configuration files (maximum 10). The named files cannot look for other configuration files.

`!KEY`
This causes all the files to be scanned using the lines after it as keyboard lines.

For example, the following configuration file (saved with the name RMBASIC.BCF) sets up some attributes by loading instructions as if they were entered at the keyboard:

```
!KEY
SET COLOUR 2 TO 14
SET COLOUR 3 TO 10
SET PEN 2
SET PAPER 3
SET KEYREP FALSE
SET WARN FALSE
```

When you want to load machine code extensions, the configuration file would look like this:

```
!EXT GPIB.EXT
!KEY
SET WARN FALSE
LOAD "GPIB"
RUN
```

If you want the previous configuration file as well, use:

```
!EXT GPIB.EXT
!BCF RMBASIC.BCF
!KEY
LOAD "GPIB"
RUN
```

The RM Basic extension package is available from Research Machines.

Appendix D
RM Basic Reserved Words

This appendix contains a list of words used by RM Basic. These words cannot be used as names for:

- variables
- array names
- procedures
- functions

ABS
AND
AREA
ASC
ASK
AT
ATN
AUTO
BLOCK
BORDER
BOUNDS
BREAK
BRUSH
BUTTONS
BYE
CHAR
CHAROVER
CHARSET
CHDIR
CHR$
CIRCLE
CLEAR
CLG
CLL
CLOSE
CLS
COLOR
COLOUR
COPY
CONT
CONTINUE
COS
CREATE
CURMOVE
CURPOS
CURSOR
DATA
DATE
DATE$
DEFINED
DEG
DELETE
DIM
DIR
DIRECTION
DRAWING
EDGE
EDIT
ELSE
END
ENDFUN
ENDPROC
ENVELOPE
EOF
ERASE
ERL
ERR
ERROR
ERR$
EXP
FALSE
FILL
FKEY
FLOOD
FLUSH
FONT
FOR
FREE
FSPACE
FUNCTION
GET
GET$
GLOBAL
GO
GOSUB

GOTO
HEX$
HOLD
HOME
IF
INP
INPUT
INSTR
INT
JOYSTICK
JOYSTICK2
JOYX
JOYY
KEYREP
LEAVE
LEFT$
LEN
LET
LEVEL
LINE
LIST
LN
LOAD
LOADGO
LOG
LOOKUP
LVAR
MEM
MERGE
MERGEGO
MID$
MIX
MKDIR
MOD
MODE
MOUSE
MOVE
NEW
NEXT
NOISE
NOT
NOTE
OFF
ON
OPEN
OR
ORIGIN
OVER
PAPER
PATH$
PATTERN
PEN
PI
PITCH
PLOT
POINTS
POSX
POSY
PRINT
PROC
PROCEDURE
PROCS
PUT
QUEUE
RAD
READ
RECEIVE
REM
RENAME
RENUMBER
REPEAT
RESTORE
RESULT
RESUME
RETURN
RIGHT$
RMDIR
RND
RPOINT
RUN
SAVE
SCALE
SET
SGN
SIN
SIZE
SLICE
SOUND
SPC
SQR
SQRT
STEP
STOP
STRING$
STR$
STYLE
SUB
SUBROUTINE
TAB
TAN
THEN
TIME
TIME$
TO
TONE
TRACE
TRUE
UNDERLINE
UNTIL
VAL
VERSION$
VOICE
WARN
WIDTH
WRITING
XOR

Index

40-character screen mode 7.2

80-character screen mode 7.1

? K.163

Abandoning a line 2.10
Abbreviation of keywords K.3
ABS K.8
Absolute value of a number K.8
Addition 4.8
Addition operator 4.8
Alternative character set 7.10
Ampersand character 3.4
AND B.6, 4.5
Angle unit
  degrees K.54
  radians K.172
Angle with a given tangent K.15
AREA 6.4
Area drawing 6.4
Area filling styles 6.16
Arithmetic operators 4.7, 3.16
Arithmetical calculations 1.3
Arrays 3.8, 3.1
  assignment rules 5.7
  dimensioning global arrays 5.14
  elements 3.10
  in graphics 6.29
  in music production 9.7
  initial assignment K.56
  integer 3.10
  memory limits 3.11
  naming them 3.9
  passing by reference 5.6
  string 3.9
  subscript 3.10
  with several subscripts 3.11
ASC K.12
ASCII code K.12
ASK 1.9
Assigning
  an expression 4.1
  numbers to variables 3.7
  values to variables 3.5
Assignment
  character 3.6
  rules for arrays 5.7
  rules for parameters 5.6
  rules for variables 3.7
Associated Keywords K.4
AT K.3, 6.23
ATN K.15
Attack 9.9
Attribute K.13, 1.9
AUTO 1.11
Auto repeat mechanism 2.4
Automatic line number generation 1.11
Available free memory K.115
Axes 6.3
Axes origin 6.8

Backspace key 1.2, 2.1
Bagpipe music 9.12
Binary numbers B.1, 4.3
Bit B.2
Bit-by-bit operation B.5, 4.7
Black and white monitor 6.1
Blank lines 5.5
BLOCK COPY 6.22
Block cursor 2.6
BLOCK READ 6.22
BLOCK WRITE 6.22
Blockage in queues 9.14
Bold typing 10.2
BORDER 6.2
Border
  area of the screen 6.1
  colour 6.2
BOUNDS 5.6
Brackets
  for operation priority 4.9
  with defined functions 5.8
BREAK keys 1.13
Breaking programs into small parts 5.1
BRUSH 6.12
Brush colour 6.11
Buffer (keyboard) K.75
BUTTONS 8.8
Buttons (mouse and joystick) 8.4
BYE K.26

Calculating
  square root of a number K.196
  the angle with given tangent K.15
  the base 10 logarithm K.111
  the exponential function K.69
  the natural logarithm K.108
  the sine of an angle K.190
  the tangent of an angle K.207
Calling
  a procedure 5.2
  procedures from procedures 5.10
  program lines to edit 2.9
Changing
  character size 6.20
  colours with SET COLOUR 7.9
  directory 10.8
  line numbers 2.9
  mouse initial position 8.6
  noise pitch 9.13

program lines 2.7
the colour tables 6.27
the cursor 7.11
the drawing size 6.24
the origin 6.8
the plotting direction 6.20
Channel 10.1
errors A.7
number 10.1
Characters
10.1
. . . in keywords K.1
/ in keywords K.1
ampersand 3.4
ASCII code K.12
assignment 3.6
brackets 4.9
changing their size (graphics) 6.20
codes K.31, 4.3
comparison rules 4.3
control 7.10
conversion of ASCII codes K.31
displaying them 7.10
dollar 3.6
equals 3.6
exclamation mark 5.5
modes 7.1
percent 3.6
plotting in graphics 6.19
sets 7.1, 7.10
tilde 7.6
underlined 7.10
CHAROVER 7.10
CHARSET 7.10
CHDIR 10.8
Checking
data with loops 4.7
for end of file K.134
for the end of a file 10.7
Choosing
a colour 6.11
a drawing area 6.26
Chords 9.7
CHR$ K.31, 7.10
Chromatic scale 9.12
CIRCLE 6.6
Circle drawing 6.6
CLEAR K.34
Clearing
a line on the screen 7.4
a writing area 7.6
memory K.34, 1.6
part of the screen 7.3
the keyboard buffer K.75
the screen 1.5, 7.3
CLG 6.27
CLL 7.4
CLOSE 10.4
Closing files K.26, 10.4
CLS 1.5, 7.3
Codes of characters K.31, 4.3
Colon separator 1.9
COLOUR 6.27, 7.8
Colour 6.9
brush 6.11
changes with SET COLOUR 6.27
choice 6.11
default 6.11
flashing 6.28
monitor 6.1
of text 7.8
of text background 7.8
paper 7.8, 6.11
pen 7.8, 6.11
resetting to default 6.12
tables 6.10, 7.8
Column position of device K.160
Columns (screen text handling) 7.1
Combining
graphics and text 7.7
program lines K.116
Command errors A.3
Communicating with devices 10.9
Comparing
expressions or values 4.2
strings 4.3
with logical operators 4.5
Comparison operator B.5
Concatenation of strings 4.4
Constants
e K.69
integer 3.2
pi K.149
string 3.1
CONTINUE K.42
Control
characters 7.10
codes for printers 10.2
unit 8.1
Controlling
error occurrences K.135
keyboard interruption K.133
what happens after errors A.2
Converting
ASCII code to character K.31
number to equivalent string K.199
Coordinates 6.3
in arrays (graphics) 6.29
list K.2
repositioning 6.23
Copying
an area of graphics 6.22
characters from right of string K.183
part of a string K.118
Correcting
errors A.1, 2.1
program mistakes 2.7
synchronization loss 9.7
COS K.43
Cosine of an angle K.43
CREATE 10.2
Creating
a global variable 5.11
a multiple copy string K.198
a variable 3.6
directory 10.8

Crochet 9.6
CURPOS 7.2
Current
  joystick position 8.7
  mouse position 8.2
CURSOR 7.11
Cursor 1.2
  changing it for GET 7.11
  finding its position 7.3
  in delete mode of editor 2.6
  indicating joystick position 8.7
  indicating mouse position 8.2
  movement using the editor 2.3
  moving it with MOVE 7.3
  moving it with SET CURPOS 7.2
  moving to the screen top-left 1.9
  preventing flash when tracking 8.4

Dashed lines 6.15
DATA 1.9
Data
  files 10.2
  instructions 1.9
  rereading with RESTORE K.179
  storage 10.2
Date K.51
DATE K.51
DATE$ K.52
Debugging A.1
Decay 9.9
Default
  colours 6.27
  drawing, writing colours 6.11
DEFINED K.53
Defining
  a drawing area 6.26
  a function 5.7
  a procedure 5.2
  a writing area 7.4
  an envelope 9.9
  dithered patterns 6.13
  drawing areas 6.26
  function of keys F1 to F10 K.71
  line styles 6.15
  your own functions 4.8
DEG 6.6, K.54
Degrees K.54
Del key 2.6
DELETE 2.10
Deleting
  a line of characters 2.6
  a program line 2.8
  characters 2.1
  several program lines 2.10
  with the editor 2.6
Description of keywords K.1
Destructive
  overdrawing 6.20
  overwriting 7.10
Detection of errors A.1
Device column position K.160
Diagrams 6.1
DIM 3.9
Dimensioning global arrays 5.14
DIR 1.12, 10.4
Direct mode 1.3
Direction of joystick 8.7
Directional plotting 6.20
Disk file 10.2
Displaying
  a file directory 1.12
  characters 7.10
  files 10.6
  numbers 3.5
  program lines 1.5
  things on the screen 1.3
  warning messages K.217
Dithered patterns 6.13
  definition 6.13
  pre-defined 6.13
  selection 6.13
Dividing the screen into areas 7.4
Division 4.8
  operator 4.8
Dotted lines 6.15
Down key 2.5
DRAWING 6.26
Drawing 6.1
  arcs 6.7
  area 6.25
  areas with writing areas 7.7
  circles 6.6
  lines 6.3
  modes 6.20
  outlines 6.17
  over things 6.21
  points 6.9
Duration of notes 9.2

e K.69
Eastern music 9.12
EDIT 2.8
Editing 2.1
  abandoning a line 2.10
  changes to program lines 2.7
  moving by one character 2.3
  moving by one word 2.4
  moving to start or end of line 2.4
  moving up and down a line 2.5
  restoring a line 2.10
Elements 3.10
Enclosed quotes in strings 3.1
END K.61
End key 2.4
End of file 10.7, K.134
Enlarging characters 6.20
Enter key 1.2
Entering
  data during program running 1.6
  information (at keyboard) 1.3
  information (READ and DATA) 1.9
ENVELOPE 9.9
Envelopes in sound 9.9
Epson printer 10.3

Equals character 3.6
ERASE 10.8
Erasing a file 10.8
ERL A.2
ERR A.2
ERR$ A.2
ERROR A.2
Error
  commands A.2
  correction A.1
  messages A.1, 1.12
Escape sequence 7.10
Examples in Keywords section K.4
Exclamation mark in PRINT 5.5
Execution
  of instructions 1.3
  order in expressions 4.9
EXP K.69
Exponential function K.69
Exponentiation 4.8
  of negative numbers 4.10
  operator 4.8
Expressions 4.1
  assigning to a variable 4.1
  assigning to an array element 4.1
  with logical operators 4.7
Extension to filename 10.4
External devices 10.9

FALSE B.4, 4.2
File 1.12
  and channels errors A.7
  checking for end of K.134
  closing 10.4
  extension 1.12, 10.4
  handling commands 10.8
  modification 10.6
  names 1.12
FILL STYLE 6.17
Filling a graphics area with FLOOD 6.18
Finding
  character of an ASCII code K.31
  current attribute settings K.13
  if a variable is assigned K.53
  Number of characters (string) K.78
  numerical value of string K.214
  operating system version number K.215
  RM Basic version number K.215
  setting of features (ASK) 1.10
  the available free memory K.115
  the character code K.12
  the cosine of an angle K.43
  the current mouse position 8.2
  the cursor position 7.3
  the joystick button state 8.7
  the joystick position 8.7
  the mouse button state 8.5
FKEY K.71
Flashing
  colours 6.28
  text 7.9
Floating point
  inaccuracies 4.3
  numbers 3.3
  variable 3.6
FLOOD 6.18
Flood fills 6.18
FLUSH K.75
FOR/NEXT 1.7
Fractional numbers 3.3
FREE K.78
FSPACE 10.8
Function keys K.7.1
FUNCTION/ENDFUN 5.7, 4.8
Functions 4.8
  defining 5.7
  in expressions 4.9
  naming 5.7
  string-handling 5.15

Generating random numbers 4.4
GET 7.11
Getting
  data from a file 10.5
  out of delete mode (editor) 2.7
Glissandos 9.3
GLOBAL 5.11
Global arrays 5.14
  dimensioning 5.14
Global instructions
  position in program 5.13
Global variabes 5.11, 8.10
  creating 5.11
  identifying 5.11
  retaining a value 5.14
Graph drawing 6.9
Graphics 6.1
  area drawing 6.4
  area filling styles 6.17
  axes 6.3
  changing the origin 6.8
  choosing a colour 6.11
  circle drawing 6.6
  colour 6.12
  combining with text 7.7
  coordinate list K.2
  coordinates 6.3
  copying an area 6.22
  defining a drawing area 6.26
  defining drawing areas 6.26
  defining line styles 6.15
  dithered patterns 6.13
  drawing lines 6.4
  enlarging characters 6.20
  errors A.7
  filling an area with FLOOD 6.18
  flood fills 6.18
  hatching patterns 6.17
  high resolution 6.3
  high resolution colour table 6.10
  line drawing 6.3
  low resolution 6.4
  low resolution colour table 6.10
  modes 6.2

options in commands 6.13
origin of coordinates 6.3
plotting direction 6.20
plotting options 6.20
plotting strings 6.19
point drawing 6.9
re-dimensioning with SCALE 6.24
repositioning coordinates 6.23
resolution 6.5
scaling the drawing size 6.24
selecting a drawing area 6.26
selecting line styles 6.15
selecting resolution 6.4
slice drawing 6.7
storing an area 6.22
syntax in commands instructions K.3
uisng arrays 6.29
writing an area from memory 6.22

Hatching pattern 6.17
Hexadecimal
numbers 3.4
system B.2
High resolution
colour table 6.10
graphics 6.3
HOLD 9.13
Holding
notes in the queue 9.6
shapes in arrays 6.29
HOME 1.9, 7.2
Home
key 2.4
position of cursor 7.2
How notes are played 9.4
How numbers are written 3.5

Identifiers 3.6
Identifying a global variable 5.11
Inaccuracies
floating point 4.3
Indention of lines on listing 1.7
Information storage 10.2
INPUT 1.6, 2.1
Input or output device 10.1
Ins key 2.7
Instruction 1.3
Integer 3.2
arrays 3.10
division 4.8, K.122
division operator 4.8
numbers B.2
or floating point? 3.4
remainder 4.8
remainder operator 4.8
variable 3.6
Interestingly equal operator 4.3
Interrupting
from keyboard 8.11
programs 1.12
Intervals less than a semi-tone 9.12

Introductory message 1.2
Irregular area filling 6.18

Joining strings 4.4
JOYSTICK 8.7
Joystick 8.7
finding the current position 8.7
initial position 8.7
sensitivity 8.7
state of the button 8.7
stick direction 8.7
JOYX 8.8
JOYY 8.8

Keeping a variable alive 5.14
Keyboard
buffer K.75
input 10.1
interruption 8.11, K.133
KEYREP 2.4
Keys
Backspace 1.2, 2.1
BREAK 1.13
Del 2.6
Down 2.5
End 2.4
Enter 1.2
function K.71
Home 2.4
Ins 2.7
Left 2.3
Num lock 2.2
numeric 2.2
Pg dn 2.4
Pg up 2.4
Right 2.3
Up 2.5
Keywords K.1, 3.8
abbreviation K.3
associated Keywords K.4
description K.1
examples in Keywords section K.4
options in Keywords section K.4
remarks in Keywords section K.4
rules of use K.3
symbols used in Keywords section K.2
syntax K.1

Labelling file contents 10.4
LEAVE 5.9
Leaving
a function 5.8
a procedure 5.9
automatic line number generation 1.11
delete mode of editor 2.7
RM Basic K.26
Left key 2.3
LET 3.5, K.104
LINE 6.3

Line
  automatic number generation 1.11
  clearing 7.4
  drawing 6.3
  indention 1.7
  number 1.4
  styles 6.15
LINE STYLE 6.15
Link for input and output 10.1
LIST 2.8, 1.5
Listing
  files 10.8
  names of procedures K.168
  program lines 1.5
  to a printer 10.2
LN K.108
LOAD 1.12
Loading files 10.5
LOG K.111
Logarithm
  base 10 K.111
  natural K.108
Logical operations 4.5
Logical operators B.5, 3.14
  AND 4.5
  in expressions 4.7
  OR 4.6
LOOKUP 10.8
Loop structures 1.7
Loss of synchronization (sound) 9.7
Low resolution
  colour table 6.10
  graphics 6.4

Magnifying characters 6.20
Mathematical equations 4.1
MEM K.115
Memory available K.115
Menu programming 8.11
MERGE K.116
MERGEGO K.117
Metronome settings 9.6
MID$ K.118
Middle C note 9.2
MIX 6.28
MKDIR 10.8
MOD 4.8, K.122
MODE 6.4
Mode
  for screen graphics 6.2
  for screen text 7.1
Modern western music 9.12
Modes of graphics 6.2
Modifying
  files 10.6
  program lines 2.1, 2.7
  the colour tables 6.27
Monitors 6.1
Monochrome monitor 6.1
MOUSE 8.2
Mouse 8.2
  buttons 8.4
  finding its current position 8.2
  initial position 8.6
  programming it 8.2
  sensitivity 8.6
  state of the button(s) 8.5
  tracking movement 8.3
  using buttons 8.5
MOVE K.45, 7.3
Movement in graphics 6.21
Movement sensitivity (mouse) 8.6
Moving
  the cursor 7.2
  the cursor to screen top-left 1.9
  the x and y axes (graphics) 6.8
MS-DOS commands 10.8
Multiple file-handling 10.7
Multiplication 4.8
  operator 4.8
Music 9.1
Musical
  intervals 9.12
  scale 9.3

Naming
  a procedure 5.2
  arrays 3.9
  variables 3.7
Natural logarithm K.108
Negative binary numbers B.3
Negative number exponentiation 4.10
NEW 1.6
NEXT 1.7
NOISE 9.13
Noise
  pitch 9.13
  production 9.12
Non-destructive overdrawing (XOR) 6.20
Non-destructive overwriting 7.10
NOT B.8
NOTE 9.1
Note lengths 9.7
Num lock key 2.2
Number of characters per line 7.1
Numbering lines 1.6
  automatically 1.11
Numbers
  16-bit B.2
  absolute value K.8
  binary B.1
  floating point 3.3
  floating point precision 3.3
  fractional 3.3
  hexadecimal B.2, 3.4
  how RM Basic writes them 3.5
  integer 3.2
  integer or floating point 3.4
  scientific notation 3.4
  spaces when displayed 3.5
  square root K.196
  TAN K.207
  to string value K.199
  with fractional part 3.3

Numeric
  constants 3.2
  keypad 2.2
  value of a string K.214
  variable 3.6

Obtaining
  information from a file 10.5
  movement 6.21
Octaves 9.2
ON BREAK K.133
ON EOF 10.7, K.134
ON ERROR A.2, K.135
OPEN 10.5
Opening a channel 10.2
Operand in integer division 4.8
Operating system
  finding the date K.52
  reporting time K.209
  updating date K.208
  updating the date K.51
  utility 10.2
  version number K.215
Operators 4.2
  arithmetic 4.8, 3.16
  comparison B.5, 4.2
  interestingly equal 4.3
  logical B.5, 3.14
Options
  graphics 6.13
  in Keywords section K.4
OR B.7, 4.6
Order for expression operations 4.9
ORIGIN 6.8
Origin
  changing 6.8
  of graphics coordinates 6.3
Origin of graphics coordinates 6.3
Outline drawing 6.17
Output to a device 10.9
Output to a printer 10.2
OVER 6.21
Overdrawing
  destructive 6.20
  non-destructive 6.20
Overwriting modes 7.10

PAPER 7.8, 6.11
Paper colour 7.8, 6.11
Parameters 5.3
  using several 5.5
  with functions 5.7
Partial line clearing 7.4
Passing arrays by reference 5.6
PATTERN 6.13
PEN 7.8, 6.11
Pen colour 7.8, 6.11
Permanent storage of programs 1.12
Pg dn key 2.4
Pg up key 2.4
PI K.149
Pi constant K.149
piano 9.11
Piconet 10.9
Pictures 6.1
Pink noise 9.12
PITCH 9.2
Pixels 6.2
PLOT 6.19, 5.1
PLOT DIRECTION 6.20
PLOT SIZE 6.20
Plotting
  direction 6.20
  options 6.20
  string (Grpahics) 6.19
Point
  drawing 6.9
  styles 6.14
POINTS 6.9
POINTS STYLE 6.14
POS K.160
POSX 8.8
POSY 8.8
Pre-defined dithered patterns 6.13
Pre-defined functions 4.8
Precision of numbers 3.3
Preventing
  cursor flashing (mouse) 8.4
  errors stopping programs A.2
  note blockage 9.14
PRINT 1.3
Printer
  features 10.2
  output 10.1
Printing blank lines 5.5
Priority of operations 4.9
Procedure 5.1
  calling 5.2
  calling other procedures 5.10
  defining 5.2
  leaving it 5.9
  name 5.2
  parameters 5.3
  recursion 5.10
  returning values 5.15
PROCEDURE/ENDPROC 5.1
PROCS K.168
Producing movement 6.21
Program 1.4
  interruption 1.12
  remarks 1.8
  tracing K.211
Programming
  a joystick 8.7
  a mouse 8.2
  for mouse and joystick 8.9
Prompt 1.2
PUT 10.3
Putting values in variables 3.5

Quarter semitones 9.12
QUEUE 9.5
Queue in sound 9.4

Quote marks 1.4
Quotes 3.1
Quotient in integer division 4.8

RAD K.172
Radians K.172
Random number generation 4.4
Re-dimensioning in graphics 6.24
READ 1.9
Reading
  colour at points on screen K.186
  files 10.5
Recalling program lines 2.9
Recorded information 10.2
Recursion 5.10
Redefining the colour tables 7.9
Relative cursor movement 7.3
Release time 9.9
REM 1.8, K.174
Remainder of integer division K.122
Remarks 1.8
  in Keywords section K.4
Removing a directory 10.8
RENAME . . . TO 10.8
Renaming a file 10.8
RENUMBER 2.10
Renumbering program lines 2.10
Repeat action of keys 2.4
REPEAT/UNTIL 1.8
Repeating program instructions 1.7
Replacing default colours 6.27
Reporting
  current directory 10.8
  free disk space 10.8
  operating system date K.52
  operating system time K.209
Repositioning coordinates 6.23
Representation in binary B.2
Rereading data instructions K.179
Resetting colours 6.12, 7.9
Resolution of graphics characters 6.5
Rest in sound 9.4
RESTORE K.179
Restoring a line 2.10
RESULT 5.8
RESUME K.181
Resuming program execution K.42
Retaining a variable value 8.10
RETURN and RECEIVE 5.15
Returning
  values from functions 5.8
  values from procedures 5.15
Right key 2.3
RIGHT$ K.183
RM Basic
  cursor 1.2
  editor 2.2
  introductory message 1.2
  prompt symbol 1.2
  version number K.215
RMDIR 10.8
RND K.185, 4.4
Rows (screen text handling) 7.1
RPOINT K.186
Rules
  for character comparison 4.3
  of keyword use K.3
  summary K.5
RUN 1.5
Running
  Basic 1.2
  programmes 1.5

SAVE 1.12
Saving programs 1.11
SCALE K.3, 6.24
Scale in music 9.3
Scaling the drawing size 6.24
Scientific notation 3.4
Screen
  40-character mode 7.2
  80-character mode 7.1
  areas 6.1
  border 6.1
  clearing it 1.5, 7.3
  drawing areas 6.26
  drawing area 6.1
  graphics coordinates 6.3
  graphics origin 6.3
  line clearing 7.4
  output 10.1
  part clearance 7.3
  prompt with INPUT 1.6
  reading colour of points K.186
  writing area 6.1, 7.4
Screen character modes 7.1
Searching for a file 10.8
Selecting
  a drawing area 6.26
  a writing area 7.5
  an envelope 9.9
  dithered patterns 6.13
  features with SET 1.10
  from options with mouse 8.11
  graphics resolution 6.4
  line styles 6.15
  text background colour 7.8
  the text colour 7.8
Sensitivity
  of a joystick 8.7
  of a mouse 8.6
Separating instructions on same line 1.9
Sequential storage 10.6
SET 1.9
SET DEG B.3
Sets of characters 7.10
Shapes 6.1
Simulating error occurrence A.2
SIN K.190
Sine of an angle K.190
Size or range problem errors A.6
SLICE 6.7
Slice drawing 6.7
Sliding pitch 9.3

Sorting 5.15
SOUND 9.1
Sound
  defining an envelope 9.9
  duration of notes 9.2
  envelopes 9.9
  glissandos 9.3
  holding and releasing notes 9.6
  intervals less than a semitone 9.12
  loss of synchronization 9.7
  middle C 9.2
  noise production 9.12
  octaves 9.2
  pitch of notes 9.2
  preventing note blockage 9.14
  quarter semitones 9.12
  queue 9.4
  rest 9.4
  scale 9.3
  selecting an envelope 9.9
  sliding pitch 9.3
  synchronization 9.6
  tempo 9.6
  using ENVELOPE with NOTE 9.11
  voices 9.7
  volume 9.3
Spaces
  in displayed numbers 3.5
  in the sound queue 9.5
Speaker 9.1
Specifying a writing area 7.6
speed of music 9.6
SQR K.196
Square root of a number K.196
Standard character set 7.10
Starting Basic 1.2
Starting position of coordinates 6.23
Stick direction of a joystick 8.7
STOP K.197
Stopping a program 1,13, K.197
Storing
  an area of graphics 6.22
  information 10.2
  instructions 1.5
  with arrays 3.8
STR$ K.199
String 3.1
  arrays 3.9
  code of the first character K.12
  comparison 4.3
  enclosed quotes 3.1
  find numerical value K.214
  functions for handling them 5.15
  joining 4.4
  MID$ K.118
  number of characters K.78
  RIGHT$ K.183
  STR$ K.199
  STRING$ K.198
  variable 3.6
STRING$ K.198
Styles in graphics
  area filling 6.17
  line 6.15
  point 6.14
Subscripts 3.10
Subtraction 4.8
Subtraction operator 4.8
Summary of RM Basic rules K.5
Superscript printing 10.2
Sustain 9.9
Symbols used in keywords section K.2
Synchronization 9.6
Syntax
  errors A.5
  of graphics instructions K.3
  of keywords K.1
  rules 1.1

TAN K.207
Tangent of an angle K.207
Tempo 9.6
Temporary
  file 10.7
  origin 6.23
  scaling 6.23
  selection (graphics) 6.12
Testing
  for equality 3.6
  for true or false condition 1.8
  values 4.2
  with logical operators 4.5
Text 7.1
  colour 7.8
  deletion 2.6
  writing 7.1
Tilde 7.6
TIME K.208
TIME$ K.209
TONE 9.12
TRACE K.211
Tracing execution of a program K.211
Tracing mouse movement 8.3
TRUE B.4, 4.2
Truncation of quotients 4.8
Type identifier 3.6
  for arrays 3.9
Typing
  errors 1.2
  mistakes 2.1

Unary
  minus operator 4.8
  plus operator 4.8
UNDERLINE 7.10
Underlined characters 7.10
UNTIL 1.8
Up key 2.5
Updating
  operating system date K.51
  operating system time K.208
Using
  a printer 10.2
  arrays in graphics 6.29

brackets in expressions 4.9
ENVELOPE with NOTE 9.11
loops to check data 4.7
mouse buttons 8.5
quote marks 1.4
several channels 10.7
several parameters 5.5

VAL K.214
Variables 3.1, 3.5
assignment rules 3.7
creating 3.6
floating point 3.6
global 5.11
integer 3.6
keeping alive 5.14
naming 3.7
numeric 3.6
string 3.6
VERSION K.215
Voices for sound 9.7
Volume of notes 9.3

WARN K.217
Warning messages A.8, K.217
When to use integers 3.4
Word display on the screen 1.4
WRITING 7.4
Writing
an area of graphics 6.22
area clearing commands 7.6
area selection 7.6
areas 7.4
areas with drawing areas 7.7
modes 7.10
programs 1.4
text 7.1
to a disk file 10.2
to writing areas 7.6

XOR B.8
XOR drawing 6.20
XOR writing 7.10
xylophone 9.10

Seal with self-adhesive tape (not staples) along this edge.

Fold along this line.

Postage will be paid by licensee

Do not affix Postage Stamps if posted in Gt Britain, Channel Islands, N Ireland or the Isle of Man

1

BUSINESS REPLY SERVICE
Licence No OF32.

RESEARCH MACHINES LTD
MILL STREET
OXFORD
OX2 0BR

User's comments help us to improve our products. If you would like to make any comments, please use this reply-paid form.

Your comments:

Research Machines may use this information in any way believed to be appropriate and without obligation.

Model: PC / AX / VX

Supplier: .....

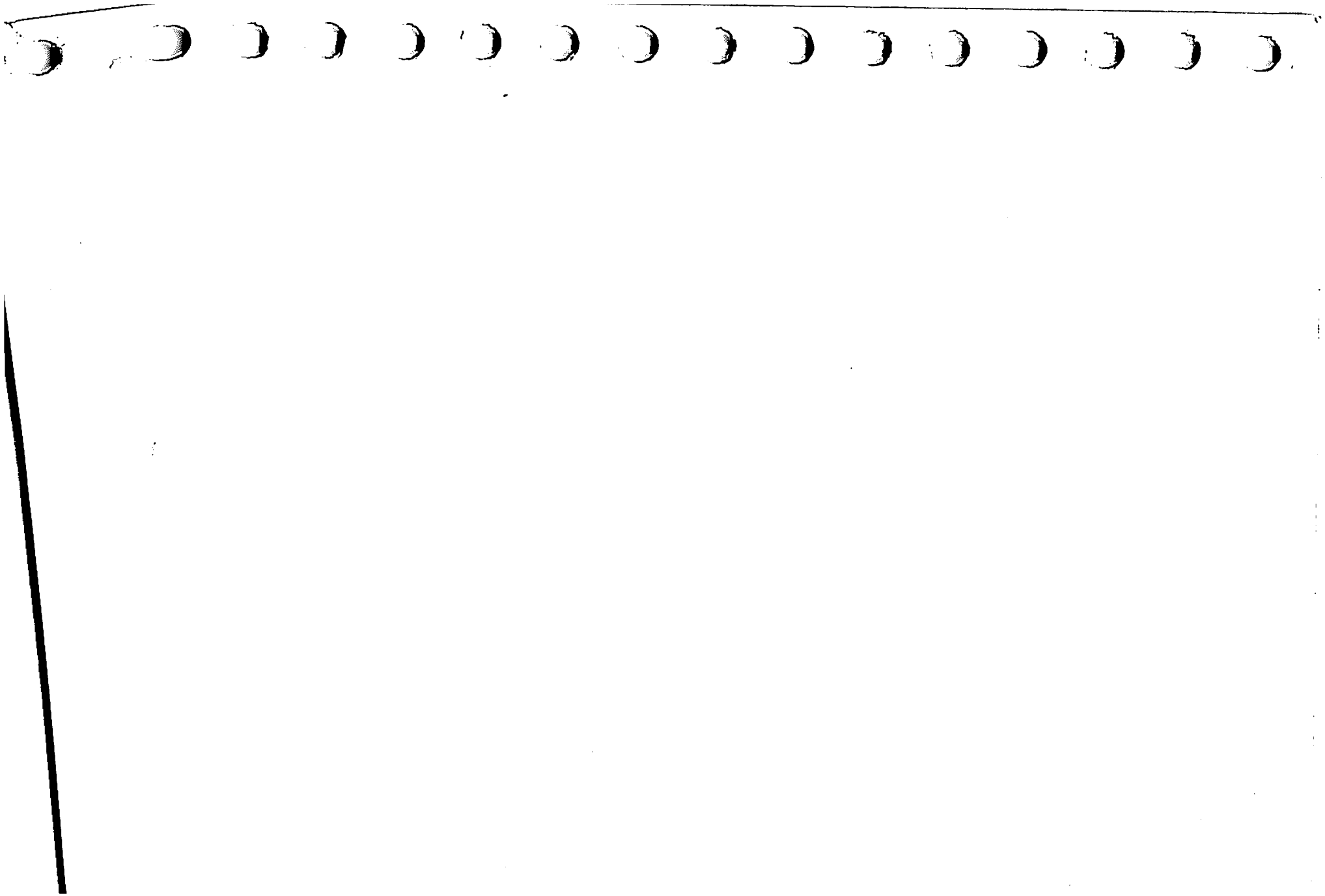
It will be helpful if you supply the following information:

Name .....

Organisation .....

Address .....

.....



RESEARCH MACHINES LIMITED
Mill Street Oxford OX2 0BW